



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Gautham P Atreyas

Compiler Design

Unit 1 Compilers

Preet Kanwal

Department of Computer Science &
Engineering

In this unit, you will learn about:

- Compilers, history and variants of compilers
- The language processing system
- The phases of a compiler
- Writing grammar for C Language
- Parsing code in C Language as per the grammar
- Lexical Analysis
- Design of a Lexical Analyzer Generator
- Specification and recognition of Tokens
- Implementation of a lexer
- Syntax Analysis
- Top-Down Parsers
- Error Recovery

After studying this unit, you will be able to:

- Explain how GCC transforms source files to an executable file
- Construct a symbol table
- Parse an input in C Language containing Variable Declaration (and initialization) Statements, Expressions (involving binary, unary, logical and relational operators), if, if-else, while, do-while, for-loop
- Draw a syntax tree for each of the above mentioned statements
- Use and write lex and yacc programs
- Create a lexer for C language using the lex tool
- Implement Recursive Descent Parser
- Write FIRST and FOLLOW sets for Table driver parser
- Understand Error Recovery

Compiler Design

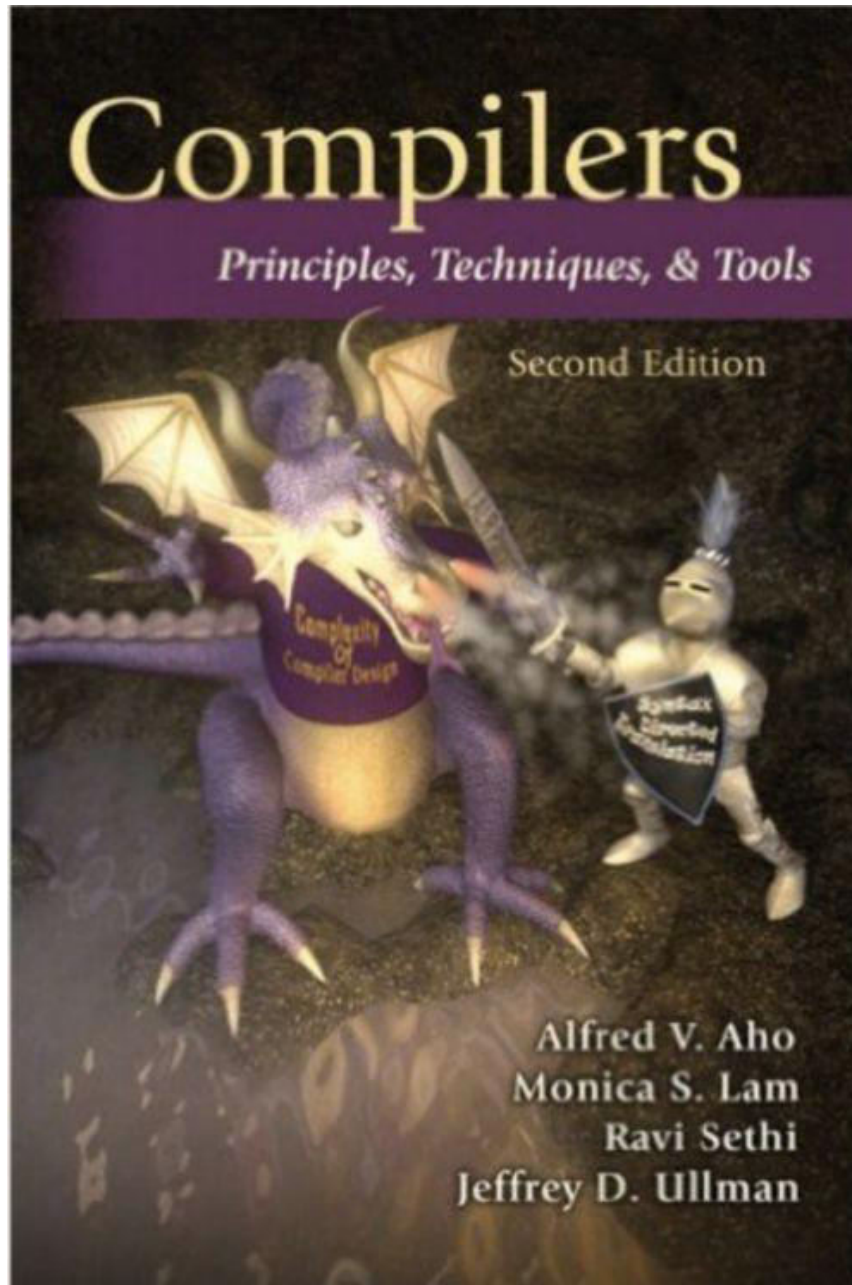
Lecture Overview

In this lecture, you will learn about:

- Introduction to the Course
- Introduction to Compilers
- History of Compilers
- Variants of Compilers:
 - Self-hosting Compilers and Bootstrapping
 - Native and Cross Compiler
 - Transpiler
 - Decompiler
 - Just-in-time Compiler
 - Compiler-compiler

Compiler Design

Course Textbooks



T1



R1

Textbook (T1):

“Compilers—Principles, Techniques and Tools” Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffery D. Ullman; 2nd Edition

Reference book (R1):

“Modern Compiler Design”, Dick Grune, Kees van Reeuwijk, Henri E. Bal, Criel J.H. Jacobs, Koen Langendoen; 2nd Edition

- Programming languages are notations for describing computations to people and to machines.
- But, before a program can be run, it must be translated into a form which can be executed by a computer.
- Machines do not understand programs in natural language or high-level programming languages.
- The software systems that do this translation are called compilers.
- A compiler is a program that can read another program in one language (the source) and translate it into an equivalent program in another language (the target).
- An interpreter differs from a compiler, where instead of producing a target program, it appears to directly execute the operations specified in the source program on inputs supplied by the user. While an interpreted program is slower to execute, they provide better error diagnostics compared to a compiler.

converts the whole program in one go

does it line by line to make debugging possible simultaneously

Compiler Design

Why study Compilers?

- Learn about the tools used by a programmer in everyday life. ✓
- Build a large, ambitious software system; see theory come to life. ✓
- Learn how to build programming languages. ✓
- Learn how programming languages work.
- Learn tradeoffs in language design. ✓



The GNU project and GCC compiler



LLVM - Clang compiler

Compiler Design

History of Compilers

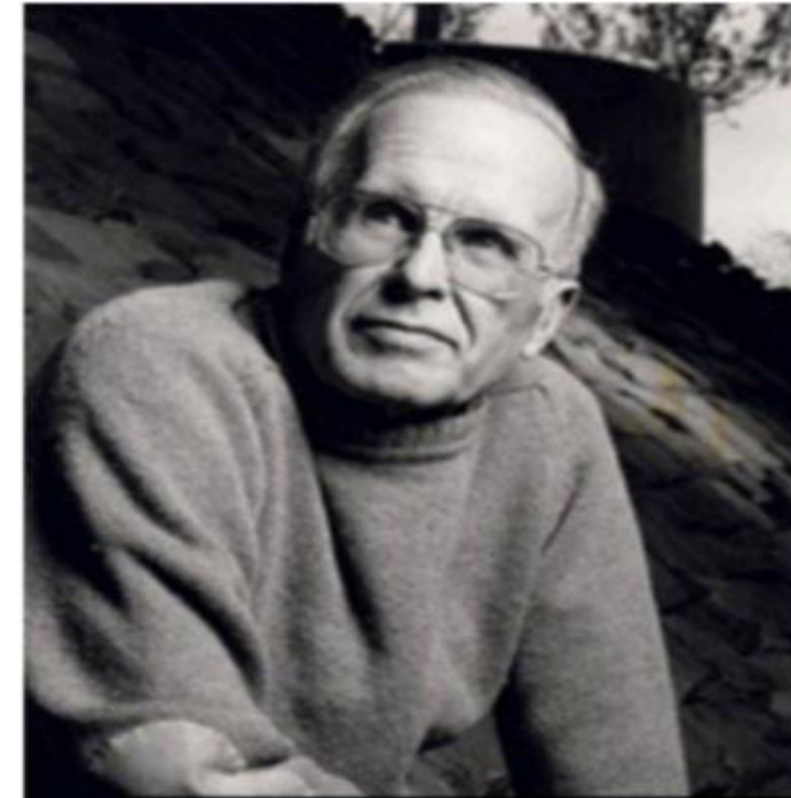
- Software for early computers was primarily written in machine code, and then later in assembly.
- Programming was expensive; 50% of costs for machines went into programming.
- The first practical compiler was written by Corrado Böhm in 1951 : PhD thesis.
- The first implemented compiler was written by Grace Hopper in 1952, who also coined the term “compiler”. The A-0 System functioned more as a loader/linker than a modern compiler.
- The FORTRAN team led by John W. Backus at IBM introduced the first commercially available compiler in 1957.
- The GNU C Compiler (now known as the GNU Compiler Collection) was first released in 1987 by Richard Stallman, and handled only the C programming language.

Compiler Design

History of Compilers



Rear Admiral Grace Hopper:
Inventor of A-0(1952) and
COBOL(1959) Coined the term “compiler.”
Coined the term “compiler”



John Backus,
team lead on
FORTRAN.

FORTRAN (FORMula TRANslation) :
First widely used high-level programming
language and the first unambiguously
complete compiler. (1954-57)

Bootstrapping is the process of writing a compiler (or assembler) for a programming language in that same programming language.

- C came into being in the years 1969-1973, in parallel with the early development of the Unix operating system.
- Ken Thompson used BCPL (Basic Combined Programming Language) to write a compiler for the B language, which was the immediate ancestor of C.
- Later, Thompson rewrote the B compiler in B itself (bootstrapping).
- This new compiler was gradually tweaked by Ken Thompson and Dennis Ritchie, leading to the first C compiler.
- TMG (TransMoGrifier) was the compiler definition tool used by Ken Thompson to write the compiler for the B language on his PDP-7 (an assembler) in 1970.

- A self-hosting compiler is a compiler that can compile its own source code. It is written in the same language which it compiles.
- Before a self-hosting compiler for a language can be created, the initial compiler must be implemented in a different language, or by bootstrapping.
- Once the language is developed enough, the compiler can be shifted to the native language, allowing the compiler to build itself.
- Example : C, C++ , C#, Java, Pascal, Python, VisualBasic
- Bootstrapping is the technique for producing a self-hosting compiler.

this is why B lang was first written in BCPL and later written in B itself.

- The problem of compiling a self-hosting compiler is analogous to the chicken-or-egg problem: which came first, the language or the compiler?
- An initial core version of the compiler - the bootstrap compiler - is generated in a different language (which could be assembly); successive expanded versions of the compiler are developed using this minimal subset of the language.
- Example : BASIC, ALGOL, C, D, Pascal, Lisp, Java, Rust, Python.
- Assemblers were the first language tools to bootstrap themselves!
- Why Bootstrap? "If the compiler for language X isn't written in language X, why should I risk using it?"
- C compiler was first written in B, C++ in C (Cfront) and C# in C++
- Here a few interesting reads on this topic:
 - Bootstrapping
 - The need for self-hosting compilers

Compiler Design

Variants of Compilers - Native and Cross Compiler

- A Native Compiler generates code the same platform or hardware architecture on which it runs. It converts the high-level language into the computer's native language.

Example: Turbo C

- A Cross Compiler is capable of creating executable code a platform other than the one on which the compiler is running. This enables developing embedded systems.

Example: A compiler that runs on Linux/x86 is building a program that will run on a separate Arduino/ARM.

- A cross compiler is for cross-platform software development of binary code.

1. Native Compiler (The Standard)

This is what you probably use most of the time on your laptop.

- **Definition:** A compiler that runs on **Machine A** and produces a program that will also run on **Machine A**.
- **The Logic:** The "Host" (where you compile) and the "Target" (where you run the app) are the exact same architecture.
- **Example from Slide: Turbo C.** If you run Turbo C on a Windows PC with an Intel processor, it creates an `.exe` file that is designed to run on that same Windows/Intel architecture.

2. Cross Compiler (The Specialist)

This is essential for mobile development, game consoles, and IoT devices.

- **Definition:** A compiler that runs on **Machine A** but produces a program that is meant to run on **Machine B**.
- **The Logic:** The "Host" and "Target" have different architectures.
- **Why do we need this?**
 - Imagine you are writing code for a tiny chip inside a microwave or a smartwatch (an **Embedded System**).
 - That tiny chip is too weak to run a heavy compiler program itself.
 - So, you write and compile the code on your powerful PC (Linux/x86), but the compiler translates it into the specific language of the tiny chip (Arduino/ARM).
- **Example from Slide:** Building an Arduino program (ARM architecture) using a Linux computer (x86 architecture).

Transpiler: Also called as source-to-source compiler, it translates from a source high-level language to a target high-level language that operates at approximately the same level of abstraction.

Example: Emscripten which transpiles C/C++ to JavaScript

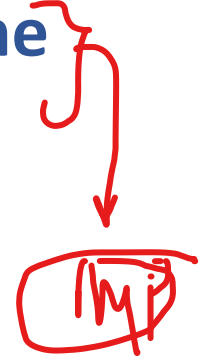
→ compiler -> high to low, decompiler -> low to high

Decompiler: Translates from a source low-level language to a target high-level language. The task is inherently difficult, due to the loss of critical information during compilation. Example: Ghidra decompiler (written in C++)

Just-in-time(JIT) compiler: Runs after the program has started, and compiles the code “just-in-time”. It is the combination of the traditional “ahead-of-time” compiler and an interpreter.

Example: Java virtual machine (JVM)

Compiler runs before runtime at compile time i.e compiles everything in one go to an exe file and then shuts down and at runtime exe file runs.
Interpreter runs at runtime one line at a time
JIT is hybrid and makes the compilation faster

- A programming tool that creates a parser, interpreter, or compiler from some form of formal description of a programming language and machine.
- The most common type of **compiler-compiler** is more precisely called a parser generator, and only handles syntactic analysis: 
 - its input is a grammar, typically written in Backus–Naur form (BNF) or extended Backus–Naur form (EBNF) that defines the syntax of a programming language
 - its output is source code of a parser for the programming language.
- Parser generators do not handle the semantics of the programming language, or the generation of machine code for the target machine 

1. Syntax (The Form)

Syntax refers to the set of rules that defines the **structure** of the language. It governs how symbols, keywords, and punctuation must be arranged to form a valid program.

- **The Key Question:** "Is this sentence grammatically correct?"
- **What it checks:** Spelling, parentheses matching, semicolons, and the order of words.
- **Error Example:**
 - *English:* "Boy the run fast." (Violates English syntax/word order).
 - *Code (C/Java):* `int x = 10` (Missing a semicolon `;` at the end).
 - *Code (Python):* `if x > 5 print(x)` (Missing a colon `:`).

2. Semantics (The Meaning)

Semantics refers to the **logic and meaning** behind the statement. It dictates what the code actually *does* when it runs. A statement can be syntactically perfect but semantically invalid.

- **The Key Question:** "Does this sentence make sense?"
- **What it checks:** Type compatibility, variable declarations, and logical operations.
- **Error Example:**
 - *English:* "The bicycle ate a stone." (Syntactically perfect—Noun + Verb + Noun—but semantically nonsense because bicycles don't eat).
 - *Code:* `int result = "Hello" / 5;`
 - **Syntax:** Correct. (Variable = Value / Value;).
 - **Semantics:** Incorrect. You cannot mathematically divide a text string by the number 5.

of Computer Science & Engineering
[@pes.edu](mailto:pes.edu)



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Gautham P Atreyas

Compiler Design

Unit 1

The Language Processing System

Preet Kanwal

Department of Computer Science &
Engineering

Compiler Design

Lecture Overview

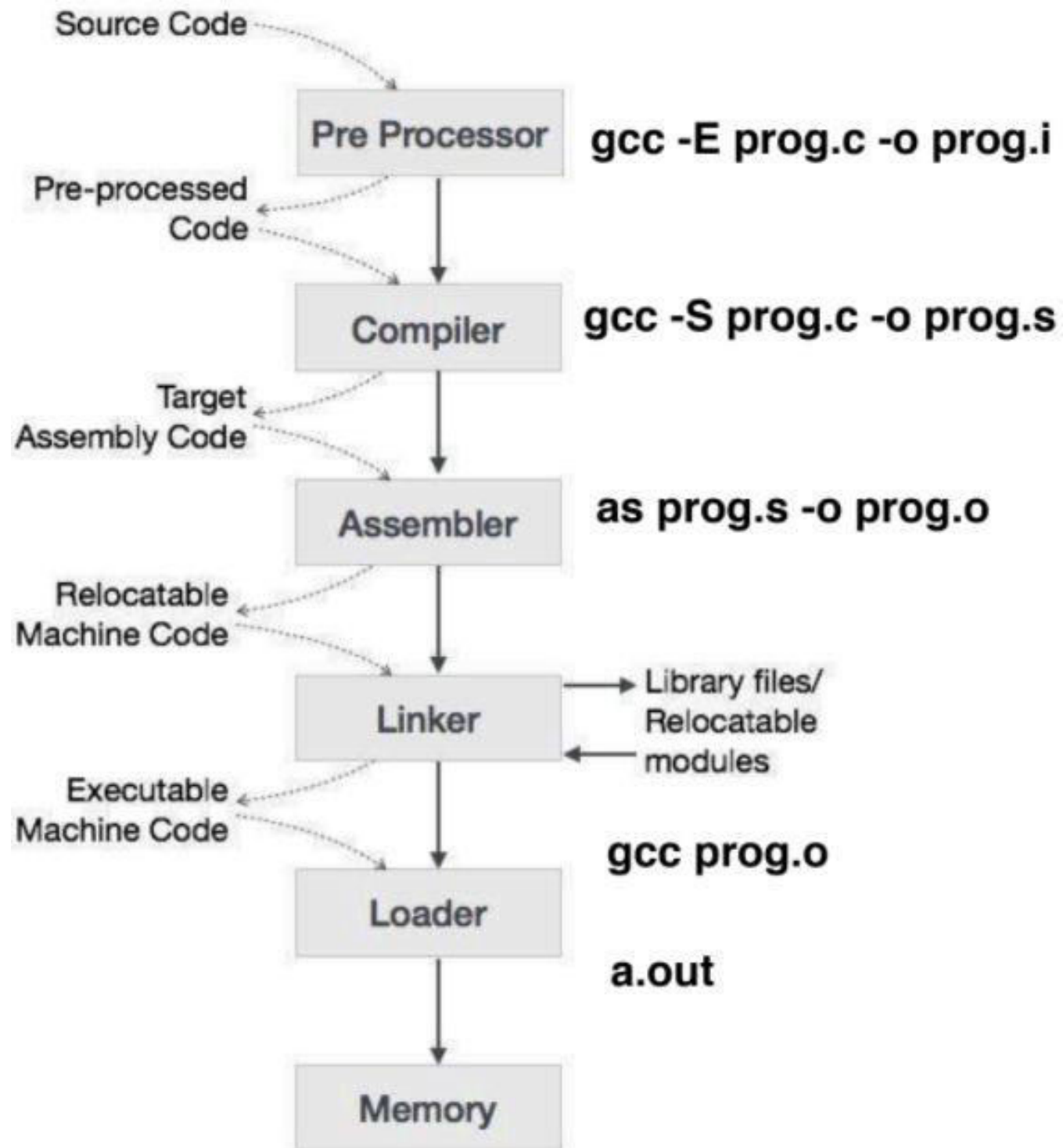
In this lecture, you will learn about:

- Introduction to GCC
- Language Processing System
- GCC Compiler Framework
- Phases of a Compiler
- Grouping of Phases into Passes
 - Single pass
 - Traditional two-pass
 - Three pass

- The acronym GCC is used to refer to the "**GNU Compiler Collection**". Over time GCC has been extended to support many additional languages, including Fortran, ADA, Java and ObjectiveC.
- GCC is written in C with a strong focus on portability, and can **compile itself**, so it can be adapted to new systems easily.
- GCC is a **portable compiler**: it runs on most platforms available today, and can produce output for many types of processors.
- GCC is not only a native compiler, it can also cross-compile any program, producing executable files for a different system from the one used by GCC itself.
- GCC has multiple language frontends, for parsing different languages. Programs in each language can be compiled, or cross-compiled, for any architecture. For example, an ADA program can be compiled for a microcontroller, or a C program for a supercomputer.

Compiler Design

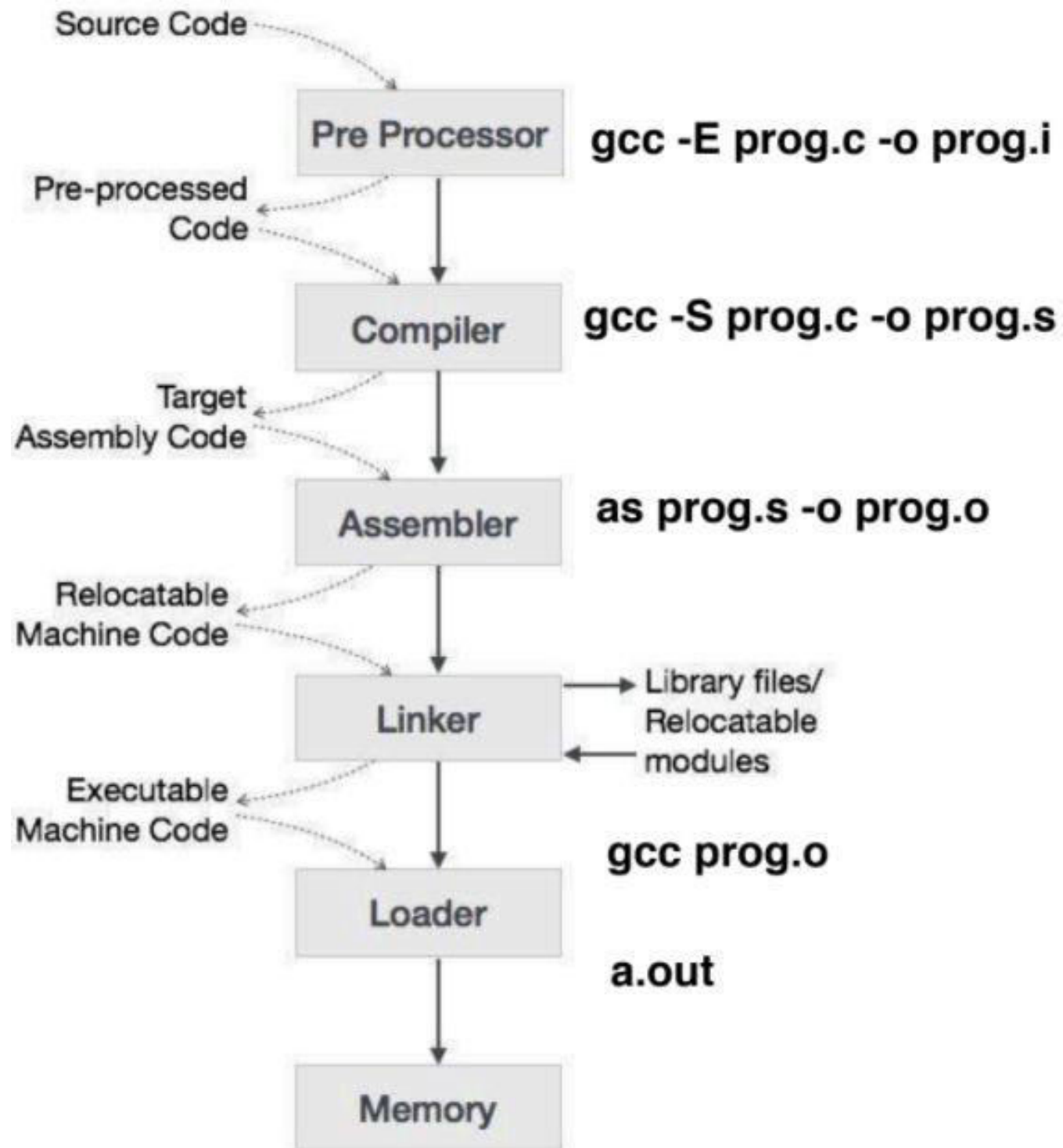
Language Processing System



- A source program can be divided into modules stored in separate files.
- Preprocessor collects the source program and performs tasks like expanding macros.
- The modified source program is fed to the Compiler, which may produce an assembly language program. Assembly language is easier to produce and debug.
- The Assembler produces Relocatable Machine Code from the assembly code. This is the machine code that can be adjusted to run at different memory addresses.

Compiler Design

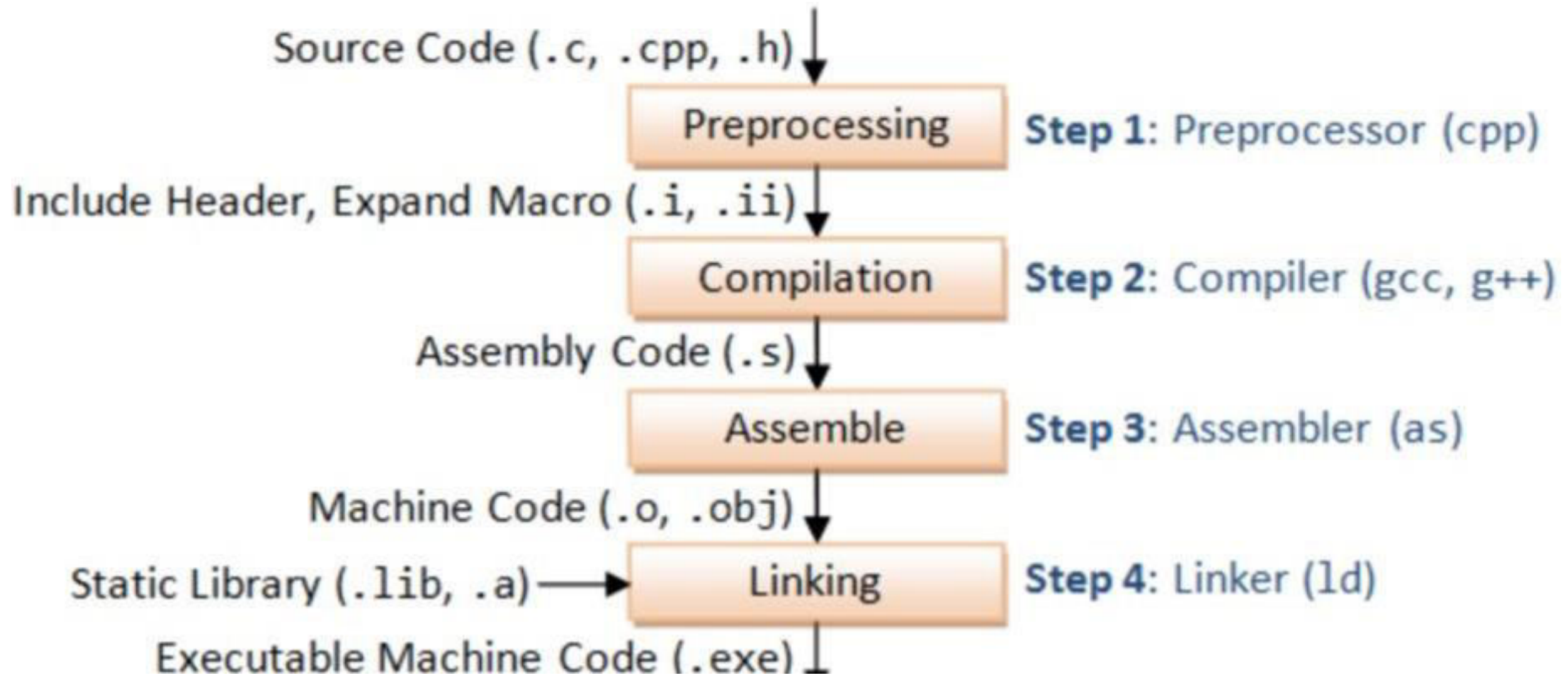
Language Processing System

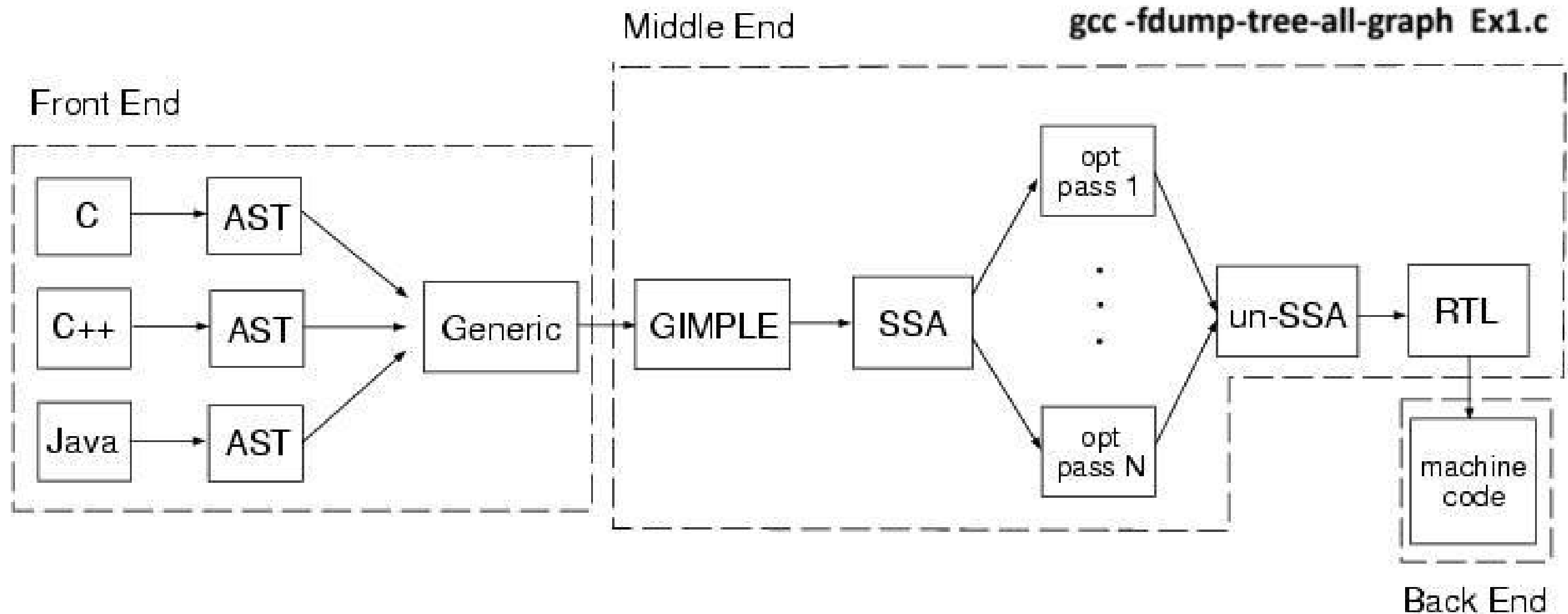


- The Linker takes care of linking the relocatable object files of all the modules, and library files. It resolves external memory addresses, where the code in one file may refer to a location in another file.
- The Loader loads all the executable object files into memory for execution. The operating system allocates memory for the program, and the loader copies the program's instructions into that memory.

- A **library** is a collection of pre-compiled object files that can be linked into your programs via the linker. Examples are the system functions such as `printf()` and `sqrt()`.
- There are two types of external libraries: **static library** and **shared library**.
- A **static library** has file extension of ".a" (archive file) in Unixes or ".lib" (library) in Windows.
- When your program is linked against a static library, the machine code of external functions used in your program is copied into the executable. A static library can be created via the archive program "ar.exe".
- A **shared library** has file extension of ".so" (shared objects) in Unixes or ".dll" (dynamic link library) in Windows.
- When your program is linked against a shared library, only a small table is created in the executable.

- Static libraries increase the size of code in the executable and are always loaded.
- The version of the static library with which it was compiled, will be the version which will run.
- Shared libraries (also called as Dynamic Linked Library in Windows) are stored and versioned separately.
- Before the executable starts running, the operating system loads the machine code needed for the external functions - a process known as *dynamic linking*. The dynamic library is usually loaded only when it is first called.
- Furthermore, most operating systems allows one copy of a shared library in memory to be used by all running programs, thus, saving memory.
- The version of the shared library can be upgraded, and the latest version can be run without the need for recompilation.
- Dynamic libraries had the problem of DLL Hell in early versions of Windows.

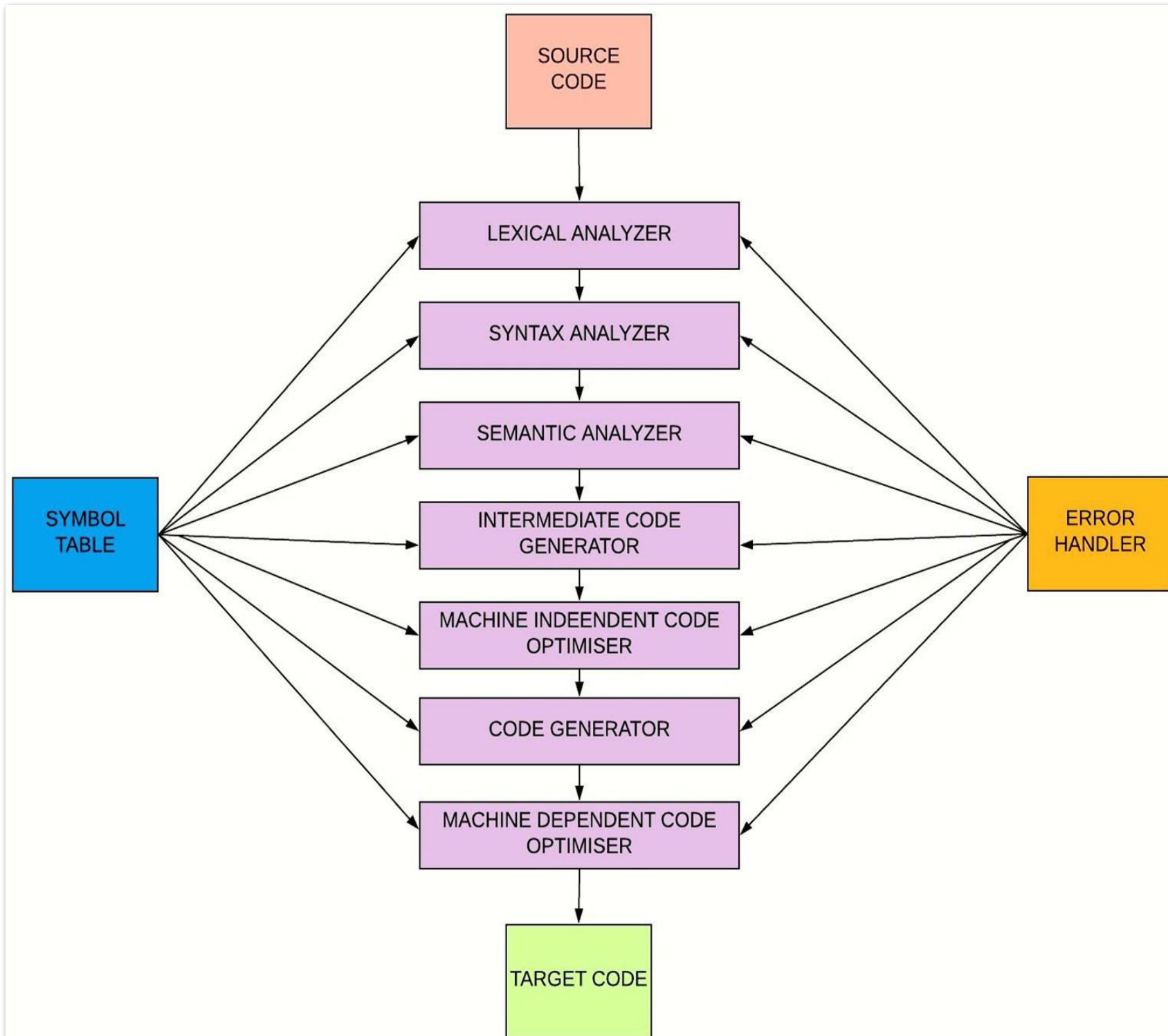




Refer to the following link for further information

<http://gcc.gnu.org/onlinedocs/gccint/>

Picture taken from grc/iitb



- The compiler does its work in seven different phases.
- Each of them have access to the entities known as “Symbol Table” and “Error Handler”
- They will be explained in upcoming slides.
- Each phase transforms one representation of the source program into another.

Single pass Compiler: All phases are grouped into one part.

Source program (HLL)



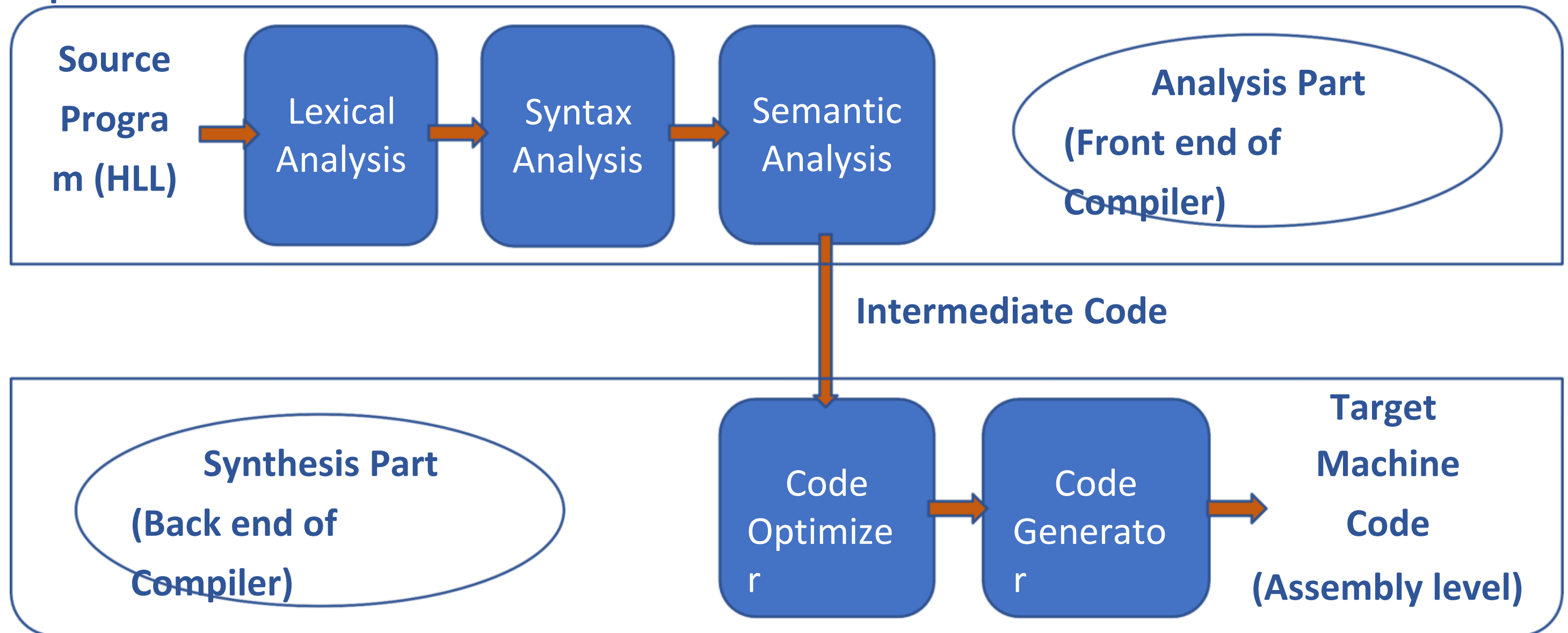
Lexical Analysis
Syntax Analysis
Semantic
Analysis
Intermediate Code
Generation Code Optimization
Code Generation



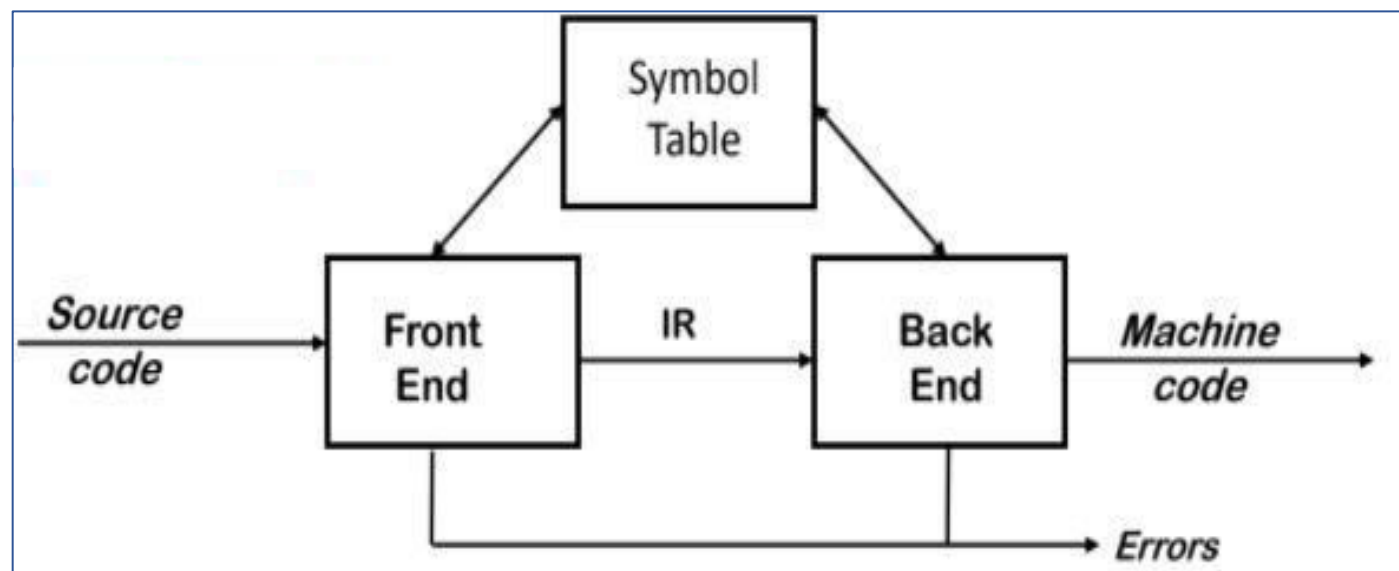
Target Machine code

/Assembly language

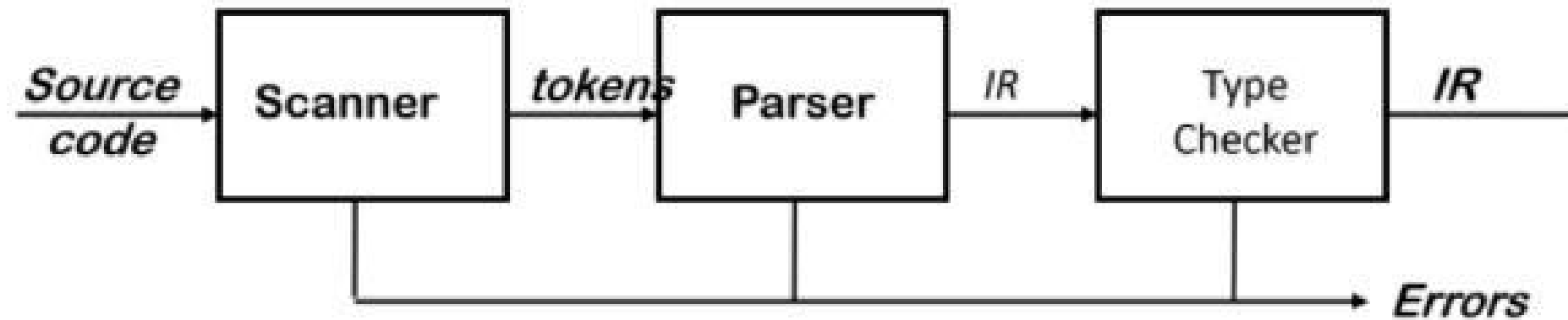
Two pass Compiler- The phases are grouped into two parts.



- The analysis part (or the front end) breaks up the source program into constituent pieces and imposes a grammatical structure on them.
- It also collects information about the source program and stores it into the Symbol Table.
- The analysis part creates the Intermediate Representation (IR).
- The synthesis part (or the back end) constructs the desired target program from the IR and information in the Symbol Table.

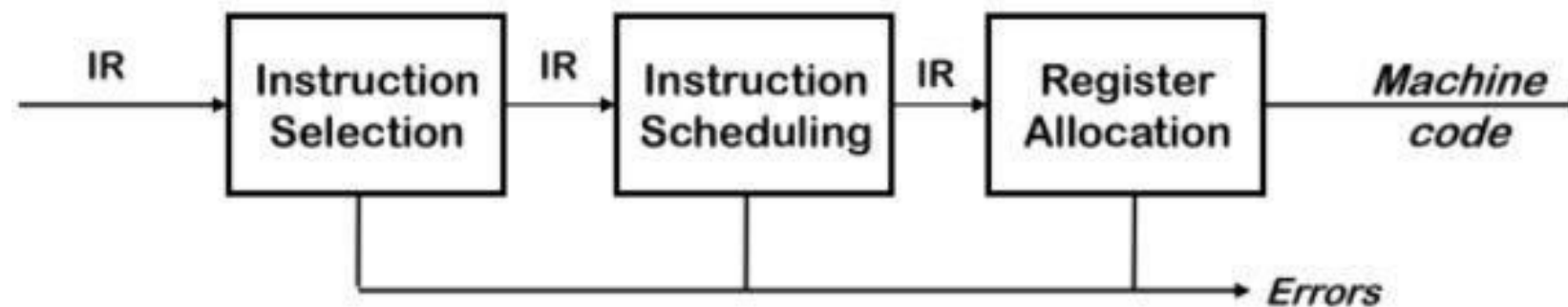


- There can be multiple front ends, and multiple passes.
- Different front ends can be combined with different back ends. Example: GCC frontend with LLVM backend.
- Typically, the front end is $O(n)$ or $O(n \log n)$, while the back end is NP-complete.



Responsibilities

- Recognize legal programs
- Report errors for the illegal programs in a useful way
- Produce IR and construct the symbol table
- Much of front end construction can be automated

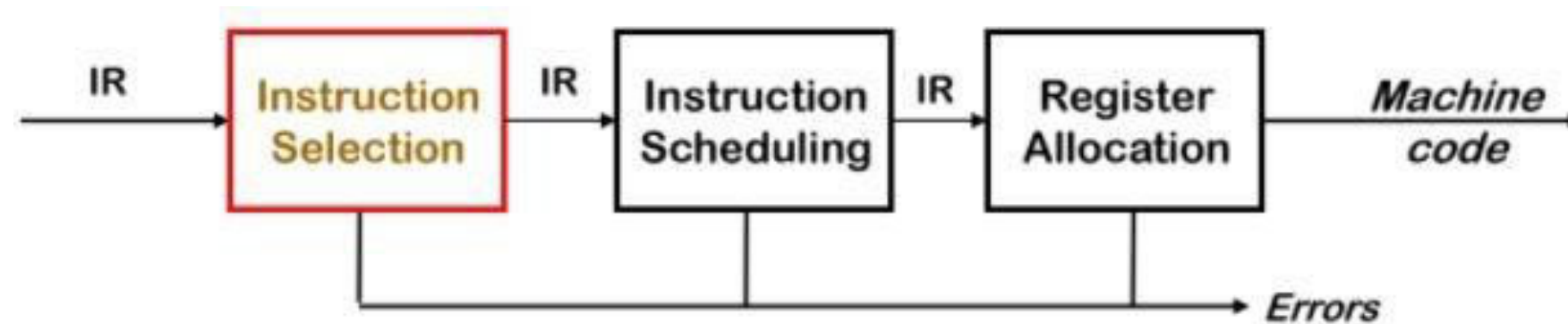


Responsibilities

- Translate IR into target machine code
- Choose instructions to implement each IR operation
- Decide which values to keep in registers
- Schedule the instructions for instruction pipeline

Automation has been *much less* successful in the back end

We can not run these “IR” files since it is machine independent and the code gets transferred into another form



- Produce fast, compact code.
- Take advantage of target language features such as addressing modes.
- Usually viewed as a pattern matching problem. This was the problem of the future in late 70's when instruction sets were complex.
- RISC architectures simplified this problem.

Source

code a = b +
c;

d = a + e;

Target

code MOV

b,R0 ADD

MOV R0,a

MOV a,R0

ADD e,R0

MOV

R0,d

Code for first
statement

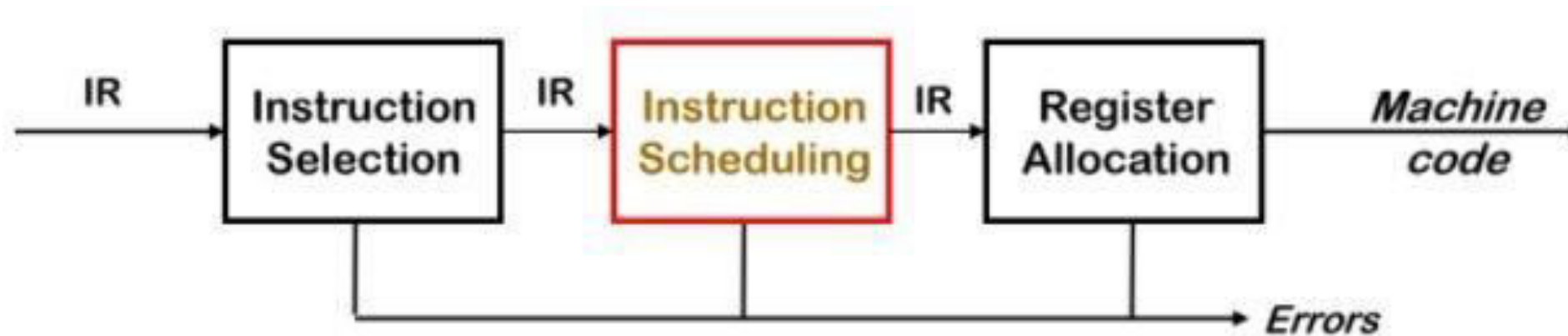
Redundant

Code for second
statement

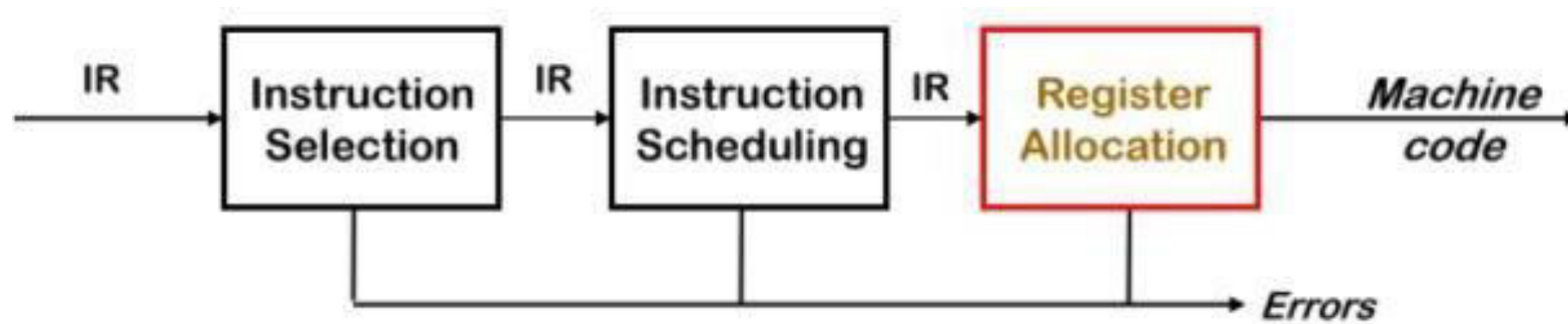
If we generate code for each statement separately,
we will not generate efficient code.

If c = 1, then:-

MOV
b,R0 INC
R0 ADD
e,R0 MOV
R0,d

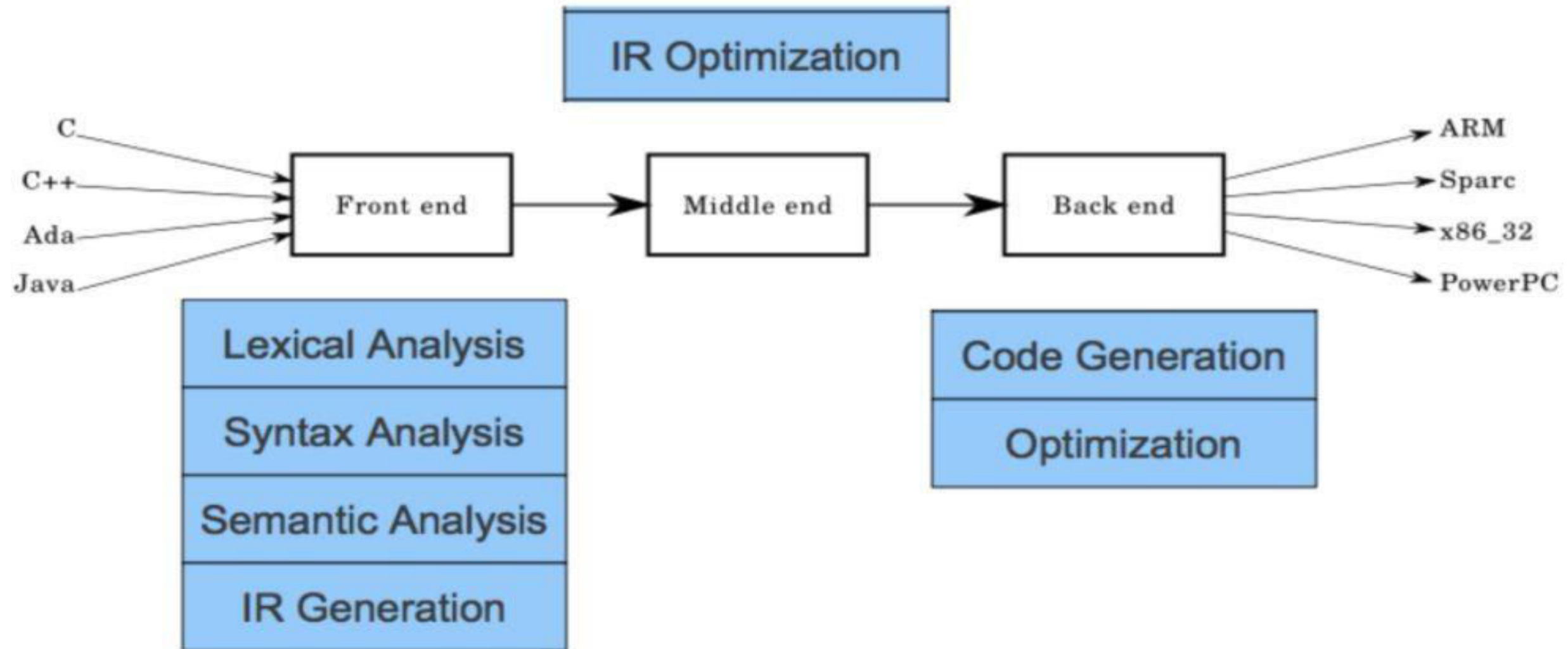


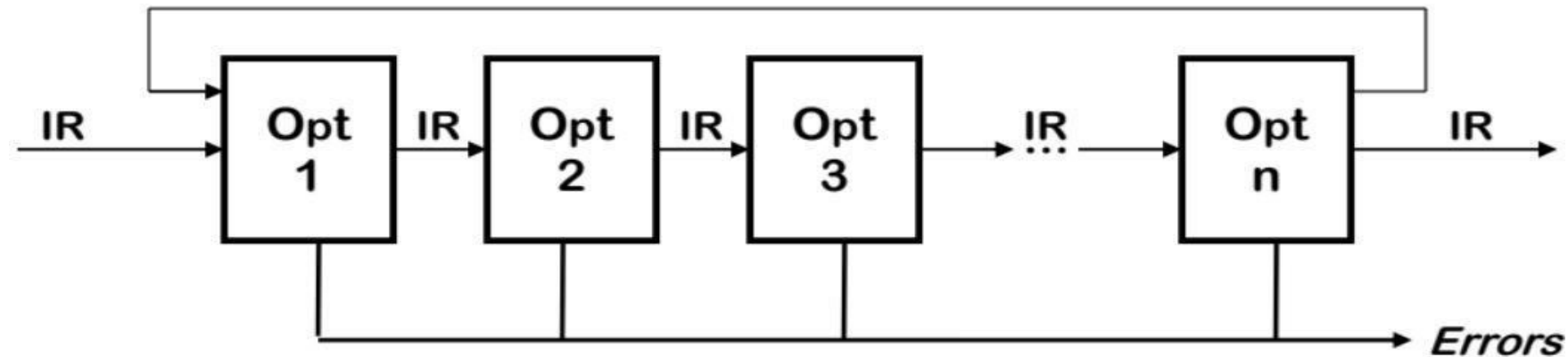
- Avoid hardware stalls (keep pipeline moving)
- Use all functional units productively
- Optimal scheduling is the focus



- Have each value in a register when it is used
- Manage a limited set of registers
- Can change instruction choices, and insert LOADs and STOREs
- Optimal allocation is the main focus

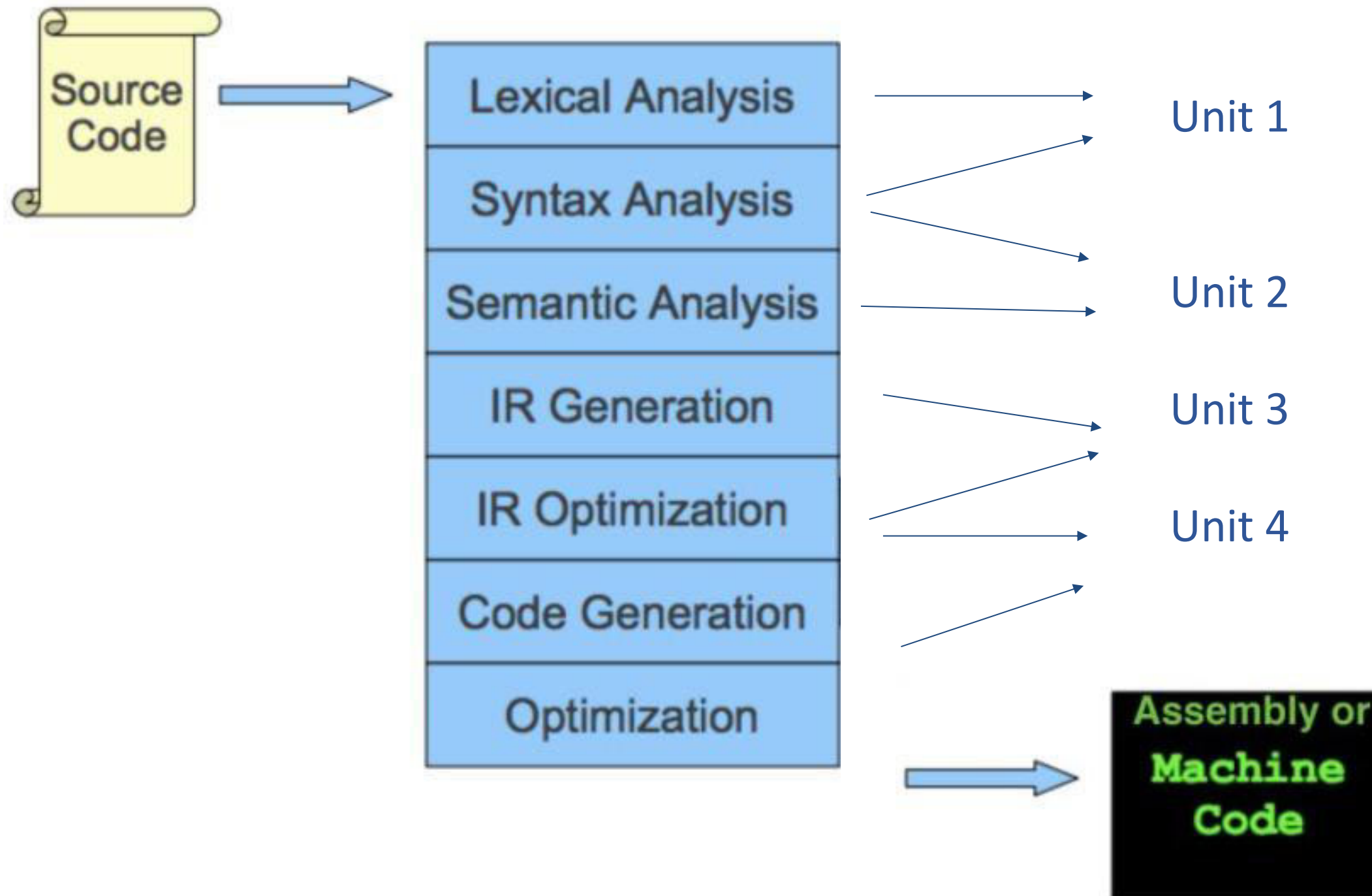
Three pass Compiler- It also includes a middle-end that involves optimization of intermediate code





Modern optimizers are structured as a series of passes

- Analyzes and transforms IR
- Primary goal is to reduce running time of the compiled code
- May also improve space and power consumption (essential for mobile computing)
- Must also preserve the “meaning” of the code
- Typical transformations include:
 - Discover and propagate constant values
 - Discover and remove redundant computation or unreachable code





**THANK
YOU**

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Gautham P Atreyas

Compiler Design

Unit 1

The Phases of a Compiler

Preet Kanwal

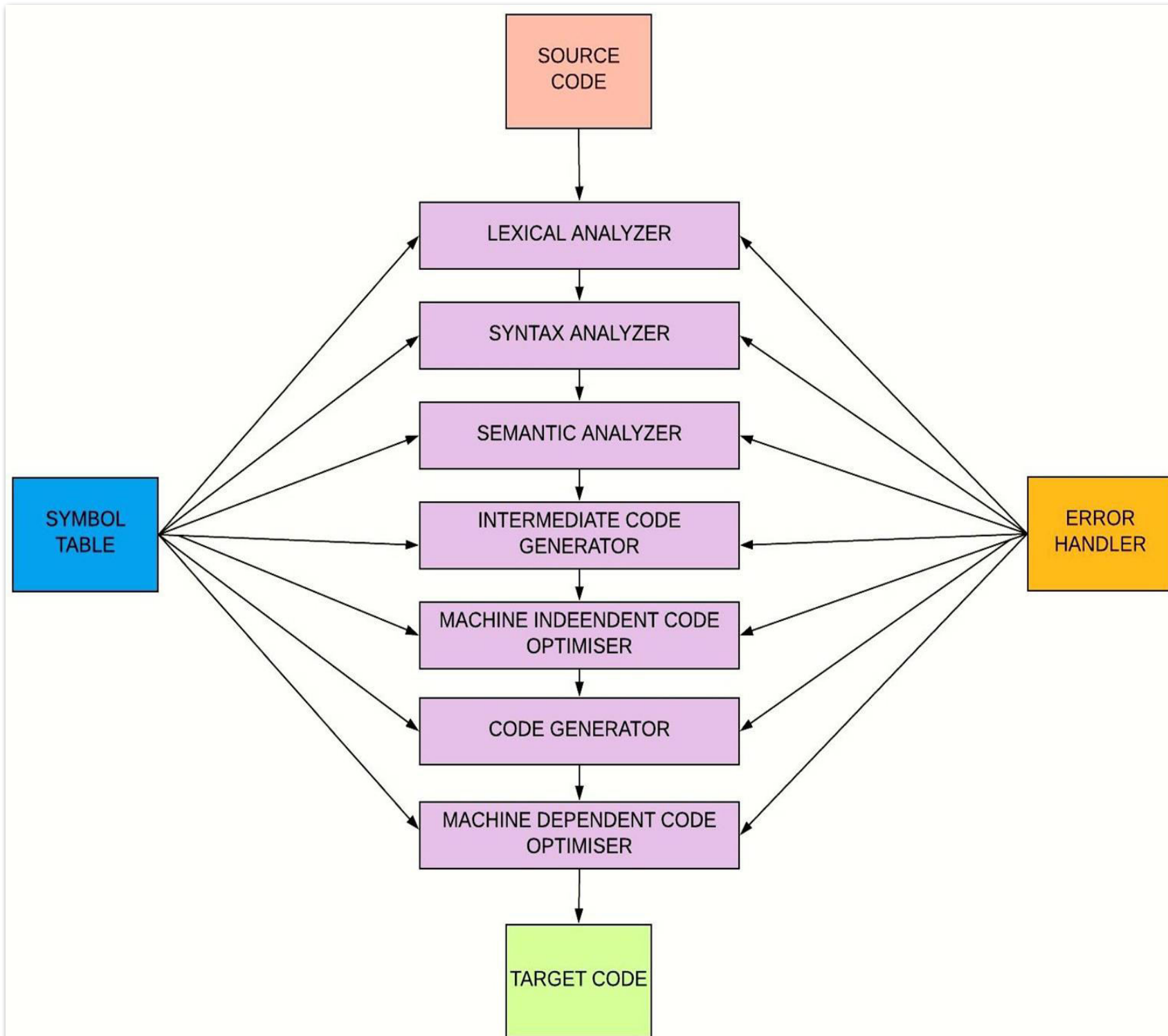
Department of Computer Science &
Engineering

Compiler Design

Lecture Overview

In this lecture, you will learn about:

- Symbol Table
- Construction of Symbol Table
- Symbol Table and Scope



- The compiler does its work in seven different phases.
- Each of them have access to the entities known as “Symbol Table” and “Error Handler”
- They will be explained in upcoming slides.
- Each phase transforms one representation of the source program into another.

What is a symbol table?

Data structure containing a record for each variable name with fields for the attributes of the name. The information is collected incrementally by analysis phase of a compiler and used by the synthesis phase to generate the target code.

How does it look like?

Attributes provides the information about:

- Storage allocation
- Type
- Scope (where in the program its value may be used)
- Procedure names
 - Number and types of its arguments
 - Method of passing arguments (call by value or reference)

Why symbol table?

- Used during all phases of compilation
- Maintains information about many source language constructs
- Used directly in the code generation phases
- Efficient storage and access important in practice
- Typically need to support multiple declarations of the same identifier

Who creates it?

- In some cases, the lexical analyzer can create an entry when it identifies a lexeme
- More often, the lexical analyzer can only return a token with a pointer to the lexeme, to the parser. Only the parser can decide whether to use a previously created symbol-table entry or create a new one
- Note: lexemes and tokens are explained in later topics

When and where is it used?

- **Lexical Analysis Time**
 - Lexical Analyzer scans program
 - Finds Symbols
 - Adds Symbols to symbol table
- **Syntactic Analysis Time**
 - Information about each symbol is filled in/updated
- **Semantic Analysis Time**
 - Used for type checking

More about symbol tables

- Each piece of info associated with a name is called an **attribute**.
- Attributes are language dependent.
- They include:
 - Actual characters of the name
 - Type
 - Storage allocation info (number of bytes).
 - Line number where declared
 - Lines where referenced
 - Scope

Different Classes of Symbols have different **attributes**

- Variable, Type, Constant, Parameter, Record:
 - Type, storage, name, scope.
- Procedure or Function:
 - Number of parameters, parameters themselves, result type.
- Array:
 - # of Dimensions, array bounds.
- File:
 - record size, record type

Other attributes:

- A scope of a variable can be represented either by
 - A number (Scope is just one of attributes)
 - Constructing different symbol tables for different scopes
- Object Oriented Languages have symbol classes like
 - Method names, class names, object names.
 - Scoping is VERY important (inheritance).

There are three main operations to be carried out on the symbol table:

- determining whether a string has already been stored
- inserting an entry for a string
- deleting a string when it goes out of scope

This requires three functions:

- **lookup(s)**: returns the index of the entry for string s, or 0 if there is no entry
- **insert(s,t)**: add a new entry for string s (of token t), and return its index
- **delete(s)**: deletes s from the table (or, typically, hides it)

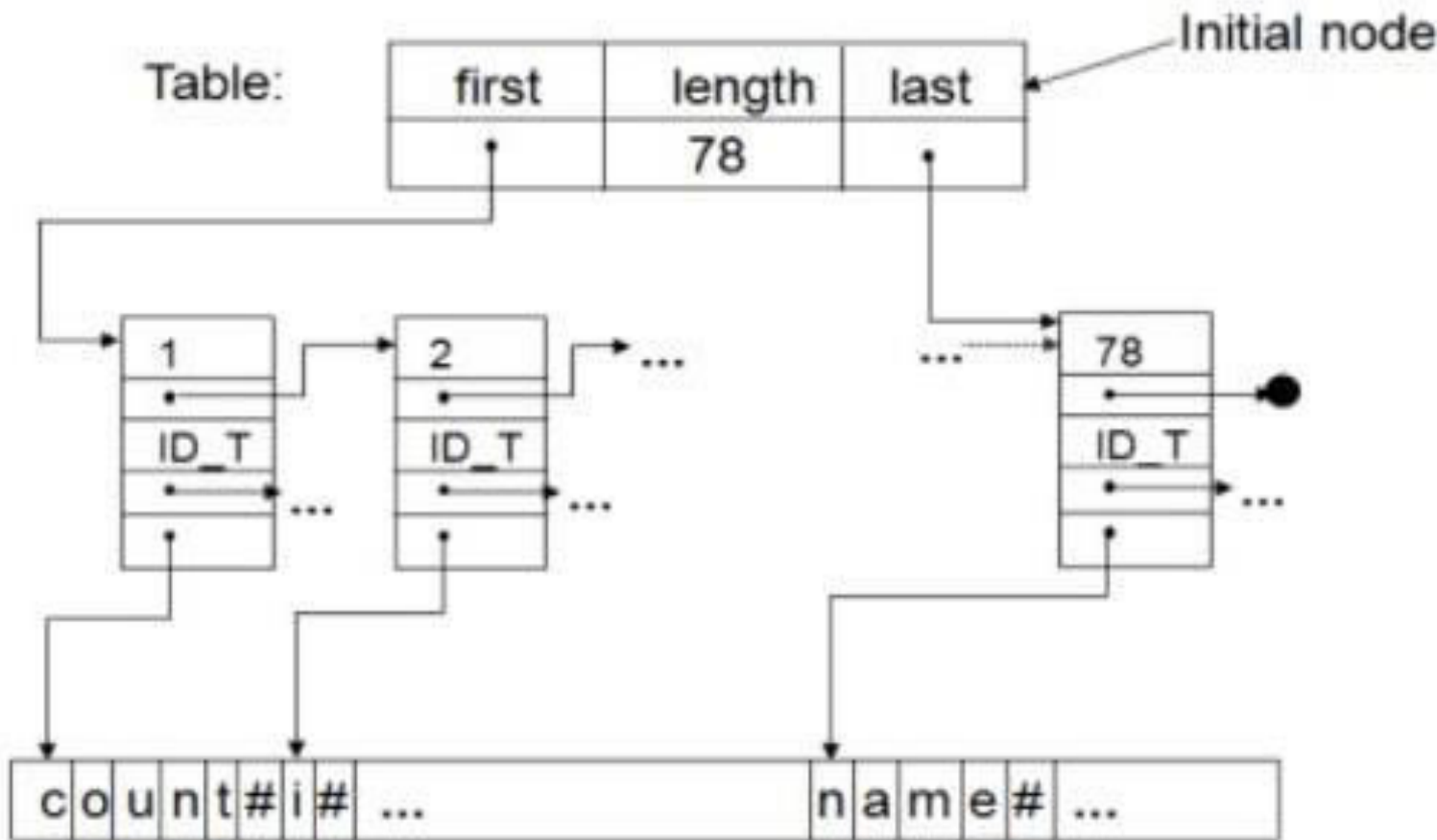
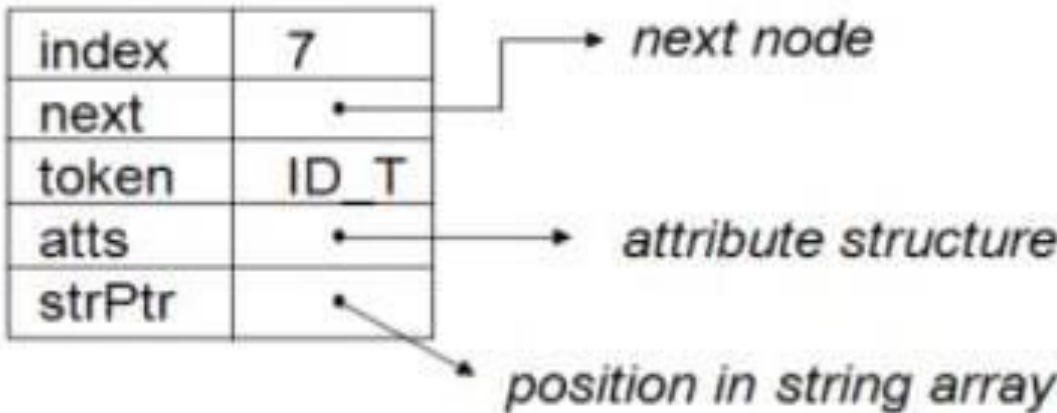
The symbol table can be constructed using 2 mechanisms:

Linear lists and Hash tables

- Each scheme is evaluated on the basis of time required to add `n` entries and make `e` inquiries.
- A linear list is the simplest to implement, but its performance is poor when `n` and `e` are large.
- Hashing schemes provide better performance for greater programming effort and space overhead.

```
1. float Fun2(int l, float j,){  
2. int k,e;  
3. float z;  
4. ;  
5. e=0;  
6. k = l *j+k;  
7. z = fun1(k,e,);  
8. *z; }
```

Name	Token	Dtype	Value	Size	Scope	Other Attribute		
						Declared	Referred	Other
Fun2	TK_ID	procname				1		
l	TK_ID	Int		4	1	1	6	parameter
j	TK_ID	Int		4	1	1	6	parameter
k	TK_ID	Int		4	0	2	6,7	argument
e	TK_ID	Int	0	4	0	2	7	argument
z	TK_ID	Float		4	0	3	7,8	return
fun1	TK_ID	procname				7		proccall



The scope of a declaration is the portion of a program to which the declaration applies. The scope of identifier `x` refers to the scope of that particular declaration of `x`.

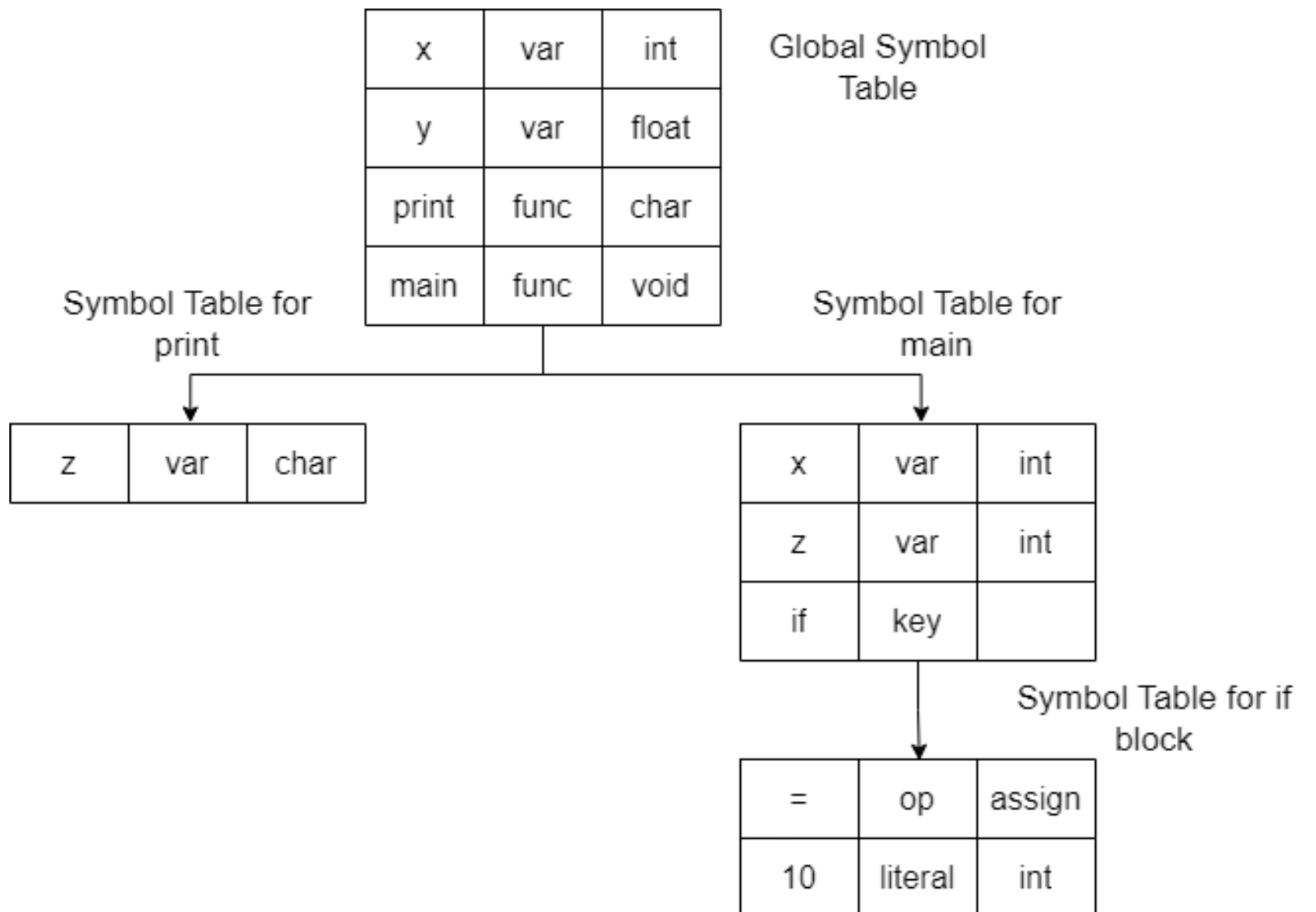
- Symbol tables typically need to support multiple declarations of the same identifier within a program.
- Subclasses can redeclare a method name to override a method in the superclass.
- If blocks can be nested, several declarations of the same identifier can appear within a single block.

We shall implement scopes by setting up a **separate symbol table for each scope**

Compiler Design

Symbol Table and Scope

```
int x;  
float y;  
  
char print() {  
    char z;  
}  
  
void main() {  
    int x, z;  
    if ( x ) {  
        y = 10;  
    }  
}
```





**THANK
YOU**

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Gautham P Atreyas

Compiler Design

Unit 1

The Phases of a Compiler

Preet Kanwal

Department of Computer Science &
Engineering

Compiler Design

Lecture Overview

In this lecture, you will learn about:

- Challenges in scanning
- C Language Grammar

Syntactic Sugar : Syntax is designed to make things easier to read or express. Example:

- In C : $a[i]$ is a syntactic sugar for $*(a+i)$
- In C : $a+=b$ is equivalent to $a = a + b$
- In C# : `var x = expr` (compiler deduces type of x from expr)

Compilers expand sugared constructs into fundamental constructs (Desugaring).

Example 1: In FORTRAN, whitespace is irrelevant. For eg:

```
DO 5 I = 1,25  
DO5I = 1.25
```

This makes it difficult to decide where to partition the input.

Example 2 : Different uses of the same characters `>` and `<` in C++

Template syntax: `a<b>`

Stream syntax: `cin>>var;`

Binary right shift syntax:

`a>>4` Nested Template :

`A<B<C>>D;`

Example 3: When keywords are used as identifiers.

```
IF THEN THEN THEN = ELSE; ELSE ELSE = IF
```

This makes it difficult to name
lexemes.

C and C++ Lexers require lexical feedback to differentiate between
typedef names and **identifiers**.

```
int foo;  
typedef int foo;  
foo x;
```

Example 4 : Scanning in Python: scope is handled using whitespace

This requires **whitespace tokens** - Special tokens inserted to indicate changes in levels of indentation.

- NEWLINE marks the end of a line.
- INDENT indicates an increase in indentation.
- DEDENT indicates a decrease in indentation.

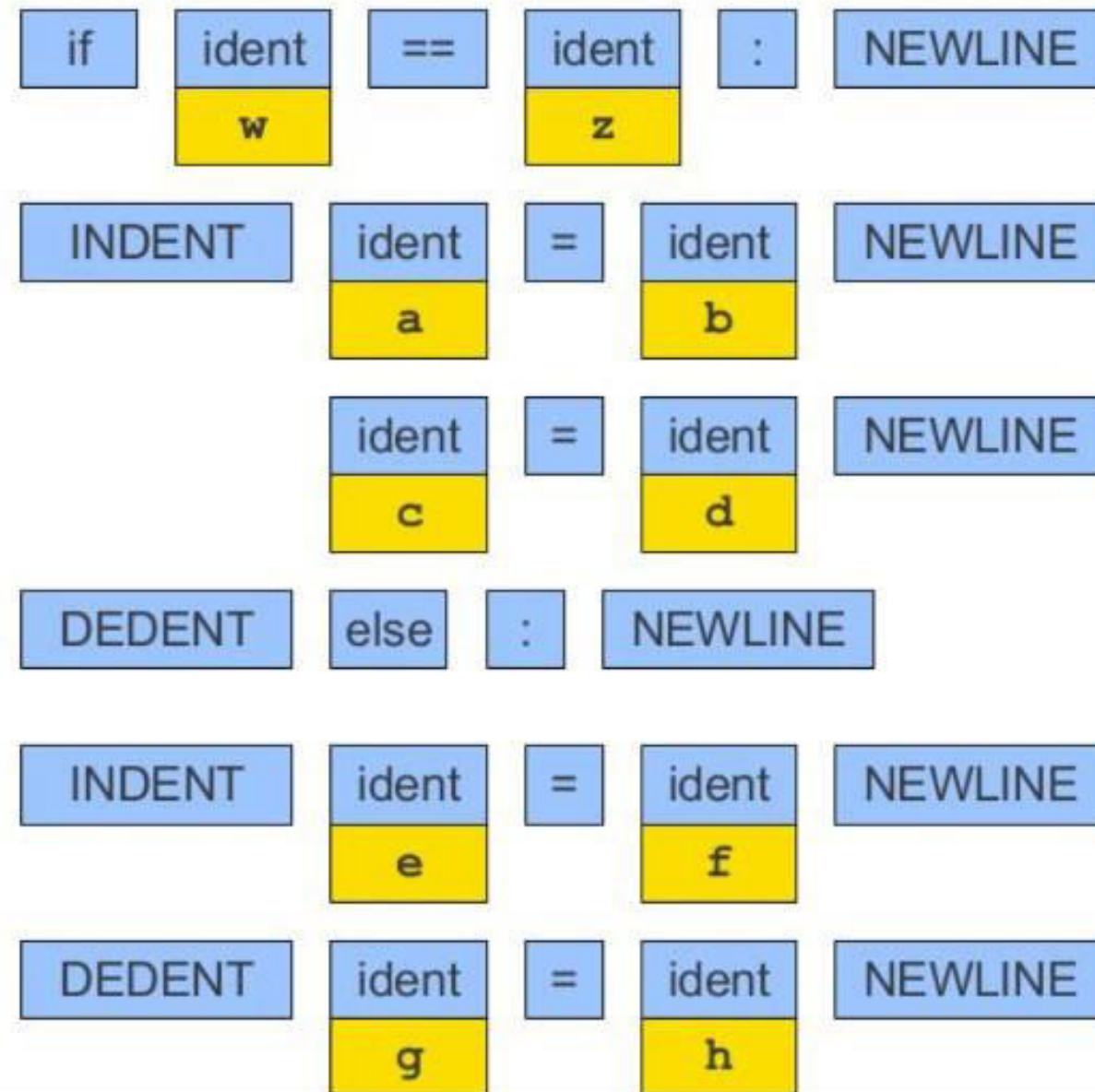
Note that INDENT and DEDENT encode only the change in indentation, and not the total amount of indentation.

Reference:

https://docs.python.org/3/reference/lexical_analysis.html

Example 4: Scanning in python (cont.)

```
if w == z:
    a = b
    c = d
else:
    e = f
g = h
```



Compiler Design

C Language Grammar

To define the C Language Grammar, let us first define the following:

P : **Program Beginning**

S : **Statement**

Declr : **Declaration**

Assign : **Assignment**

Cond : **Condition**

UnaryExpr : **Unary Expression**

Type : **Data type**

ListVar : **List of variables**

X : **can take any identifier
or assignment**

Compiler Design

C Language Grammar

P	→	S
S	→	Declr; S Assign; S if (Cond) {S} S while (Cond) {S} S if (Cond) {S} else {S} S for (Assign; Cond; UnaryExpr) {S} S return E; S λ
Declr	→	Type ListVar
Type	→	int float
ListVar	→	X ListVar, X
X	→	id Assign
Assign	→	id = E
Cond	→	E RelOp E
RelOp	→	< > <= >= == !=
UnaryExpr	→	E + E - E * E /

Recall that

$E \rightarrow E + T \mid E - T \mid T$

$T \rightarrow T * F \mid T / F \mid F$

$F \rightarrow G \wedge F \mid G$

$G \rightarrow (E) \mid id \mid num$



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Gautham P Atreyas

Compiler Design

Unit 1

The Phases of a Compiler

Preet Kanwal

Department of Computer Science & Engineering

Compiler Design

Lecture Overview

In this lecture, you will learn about:

- C Language Grammar
- Running an input through the different phases of a compiler
- Simple IR Optimizations
- Syntax Tree vs Parse Tree

Compiler Design

C Language Grammar

P	→	S
S	→	Declr; S Assign; S if (Cond) {S} S while (Cond) {S} S if (Cond) {S} else {S} S for (Assign; Cond; UnaryExpr) {S} S return E; S λ
Declr	→	Type ListVar
Type	→	int float
ListVar	→	X ListVar, X
X	→	id Assign
Assign	→	id = E
Cond	→	E RelOp E
RelOp	→	< > <= >= == !=
UnaryExpr	→	E + E - E * E /

Recall that

$E \rightarrow E + T \mid E - T \mid T$

$T \rightarrow T * F \mid T / F \mid F$

$F \rightarrow G \wedge F \mid G$

$G \rightarrow (E) \mid id \mid num$

Example 1 : Consider the following piece of code:

```
while (number > 0)
{
    factorial = factorial * number;
    --number;
}
```

We will now run this code through the different phases of a compiler and see what the output will be at each stage.

Before we start, there are a few terminologies that should be known. These will be explained in the later slides and units, but are important for this exercise.

- **Token**: a structure, that represents a sequence of characters which cannot be broken down further
- **Lexeme**: the actual sequence of characters in the source program
- **Pattern**: rule that defines the structure for a lexeme to be an instance of a token
- **Syntax**: describes the proper form of a programming language
- **Semantics**: defines what the program means
- **Syntax Tree**: tree-like representation of the syntax of a source program

Before we start, there are a few terminologies that should be known. These will be explained in the later slides and units, but are important for this exercise.

- **3 Address Code**: an intermediate representation, where the instructions are of the form ``x = y op z``. There can be at most one operator per instruction. Complex instructions are broken up using temporary variables ``t0, t1, ... tn``
- **Basic Block**: sequence of instructions that are always executed sequentially, i.e. no jumps or interruptions
- **DAG Optimization**: optimization of basic blocks using Directed Acyclic Graphs. It is used to determine common subexpressions

Phase 1 : Lexical Analysis or Linear Analysis or Scanner

Pattern	Lexemes matched
Keyword	while
Identifier	number, factorial, factorial, number, number
RelOp (<, <=, >, >=, ==, !=)	>
LogOp (&&, , ~)	
Assign (=)	=
ArithOp (*, +, -, /)	*
Punctuation ((,), {, }, [,], ;)	({ ; ; }
	)
Number	0
Literal – anything in double quotes	
Inc_op	

```
while (number > 0)
{
    factorial = factorial *
    number;

    --number;
}
```


Output: # of tokens : 17

< keyword, while >
< punctuation, (> or < (>
< ID, 3 >
< RelOp, > > or <
> >
< Number, 0 >
< punctuation,) > or <) >
< punctuation, { > or < { >
< ID, 4 >
< Assign, = > or <
= >
< ID, 4 >
< ArithOp, * > or <
* >
< ID, 3 >
< punctuation, ; > or < ; >
< Dec_op, -- > or <
-- >

Symbol
Table

	name	type	...
1	...		
2			
3	number		
4	factorial		

< ID, 3 > corresponds
to the third
identifier in the

while (number > 0)

{

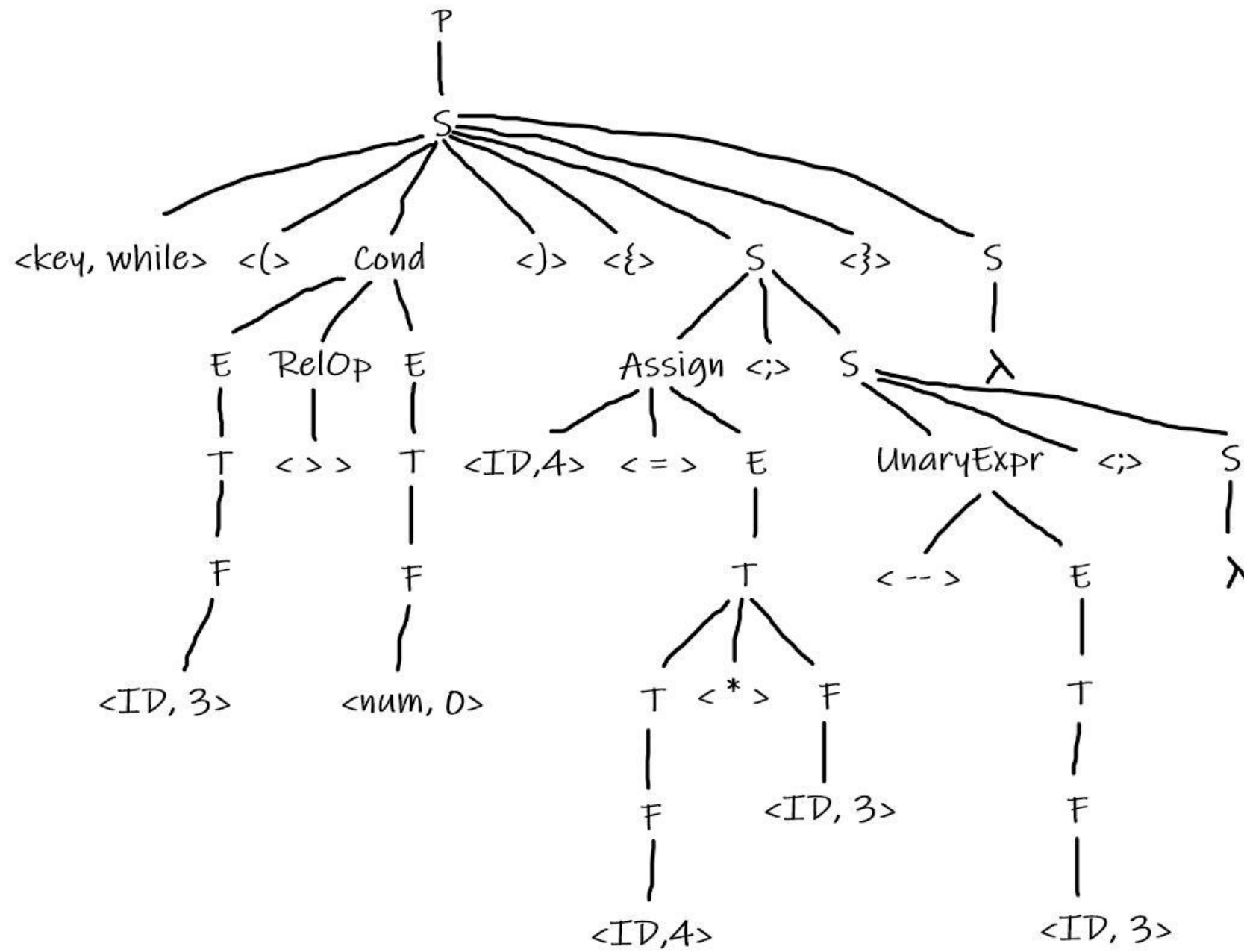
factorial = factorial *
number;

--number;

}

Phase 2 : Syntax Analysis or Parser

Output: **Parse Tree or Syntax Tree**

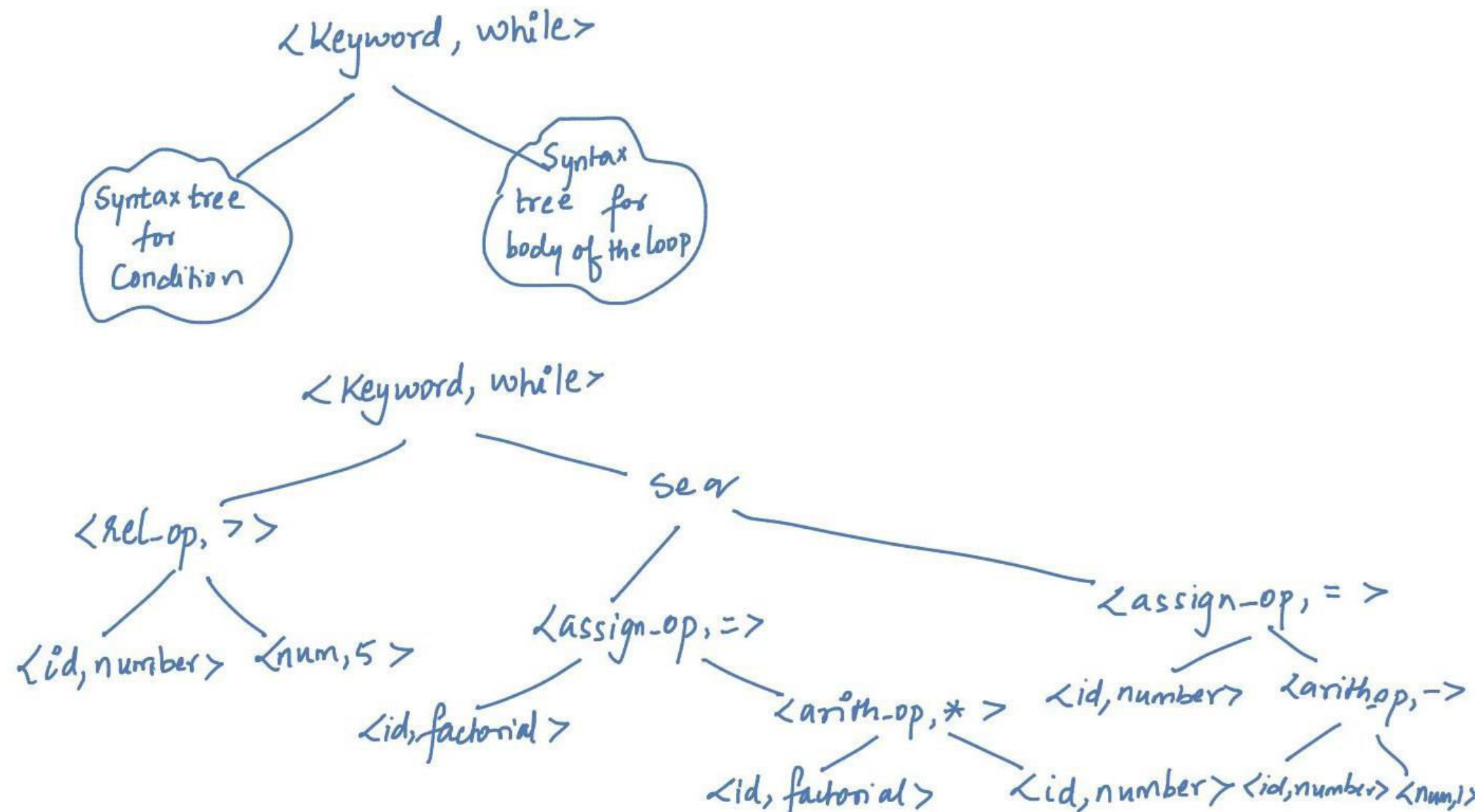


Use the grammar
we have defined
earlier

```
while (number > 0)
{
    factorial = factorial *
    number;
    --number;
}
```

Phase 3 : Semantic Analyzer

Output: **Semantically checked Syntax Tree**



```
while (number > 0)
```

```
{
```

```
    factorial = factorial *  
    number;
```

```
    --number;
```

```
}
```

Phase 4 : Intermediate Code Generator

Output: **Three Address Code (3AC or TAC)**

```
L0:    if number > 0 goto L1
      goto L2

L1:    t1 = factorial * number
      factorial = t1
      t2 = number - 1
      number = t2
      goto L0
```

```
L2: ...
```

```
while (number > 0)
{
    factorial = factorial *
    number;
    --number;
}
```

Phase 5 : Machine Independent Code Optimization

Output: **DAG Optimization on the basic block L1**

```
L0 : if number > 0 goto L1
```

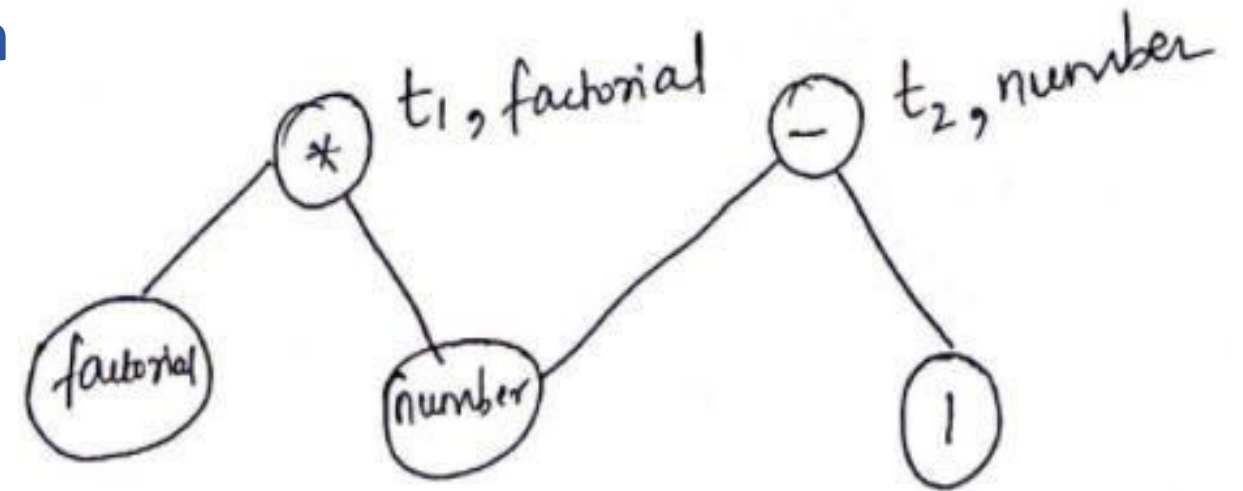
```
    goto L2
```

```
L1 : factorial = factorial * number
```

```
    number = number - 1
```

```
    goto L2
```

```
L2 : ...
```



```
while (number > 0)
```

```
{
```

```
    factorial = factorial *  
    number;
```

```
    --number;
```

```
}
```

Phase 6 : Code Generator

Output: **Assembly Code**

```
LD      R1, #0
L1: LD   R4, number
      SUB R2, R1, R4
      BLTZ R2, L2
      LD   R3, factorial
      MUL  R3, R3, R4
      ST   factorial, R3
      SUB  R4, R4, #1
      ST   number, R4
      BR   L1
L2:
```

```
while (number > 0)
{
    factorial = factorial *
    number;
    --number;
}
```

Phase 7 : Machine Independent Optimized Code

Output: **Optimized Target Code**

Constant folding is the process of recognizing and evaluating constant expressions at compile time rather than computing them at runtime.

Constant propagation is the process of substituting the values of known constants in expressions at compile time.

Common Subexpression Elimination (CSE) is a compiler optimization that searches for instances of identical expressions (i.e., they all evaluate to the same value), and analyzes whether it is worthwhile replacing them with a single variable holding the computed value.

Parse tree is a concrete representation of the input. It retains all the information from the input.

Syntax Tree is an abstract representation of the input. It only contains the information required to generate IR or machine code.

Parse Tree (Concrete Syntax Tree)	(Abstract) Syntax Tree
It is huge and focuses on all lexeme	It focuses only on the syntax
Consists of both Non-terminals (P, S, E, Cond, RelOp, etc.) and Terminals (leaf nodes like tokens)	All nodes are tokens; It also has dummy nodes (<body>) to maintain proper syntax
Complicated	No non-terminals; Easy
Has punctuation (; , { })	No punctuation symbols

Example 2 : Consider the following piece of code:

```
int rhs,  
lhs = 0;  
rhs = 2;  
if (lhs <= rhs)  
{  
    lhs = lhs - 2 * rhs;  
}
```

Run this code through the different phases of a compiler and produce the output at each stage.



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Gautham P Atreyas

Compiler Design

Unit 1

The Phases of a Compiler

Preet Kanwal

Department of Computer Science &
Engineering

Compiler Design

Lecture Overview

In this lecture, you will practise:

- Running an input through the different phases of a compiler

Example 2 : Consider the following piece of code:

```
int rhs,  
lhs = 0;  
rhs = 2;  
if (lhs <= rhs)  
{  
    lhs = lhs - 2 * rhs;  
}
```

We will now run this code through the different phases of a compiler and see what the output will be at each stage.

Phase 1 : Lexical Analysis or Linear Analysis or Scanner

Pattern	Lexemes matched
Keyword	int if
Identifier	rhs lhs rhs lhs rhs lhs lhs rhs
RelOp (<, <=, >, >=, ==, !=)	<=
LogOp (&&, , ~)	
Assign (=)	= = =
ArithOp (*, +, -, /)	- *
Punctuation ((,), {, }, [,], ;)	; ; ({ ; })
Number	0 2 2
Literal – anything in double quotes	
Inc_op	
Dec_op	

```
int rhs, lhs = 0;

rhs = 2;

if (lhs <= rhs)
{
    lhs = lhs – 2 * rhs;
}
```

Output: # of tokens : 27

< keyword, int >
< ID, 3 >
< punctuation, , >
< ID, 4 >
< Assign, = >
< Number, 0 >
< punctuation, ; >
< ID, 3 >
< Assign, = >
< Number, 2 >
< punctuation, ; >
< keyword, if >
< punctuation, (>
< ID, 4 >
< RelOp, <= >
< ID, 3 >
< punctuation,) >

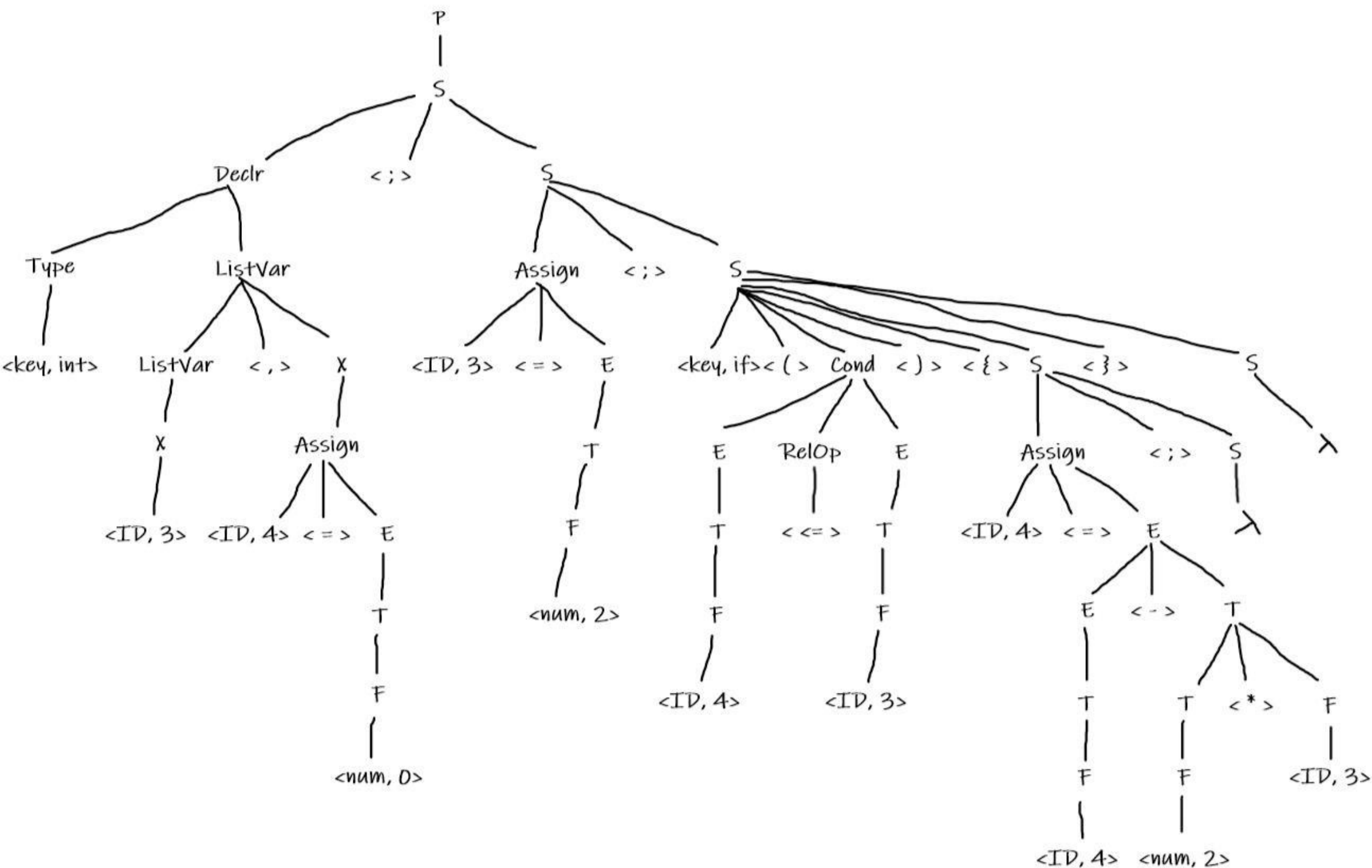
Symbol
Table

	name	type	...
1	...		
2			
3	rhs		
4	lhs		

< punctuation, { >
< ID, 4 >
< Assign, = >
< ID, 4 >
< ArithOp, - >
< Number, 2 >
< ArithOp, * >
< ID, 3 >
< punctuation, ; >
< punctuation, } >

```
int rhs, lhs = 0;  
  
rhs = 2;  
  
if (lhs <= rhs)  
{  
  
    lhs = lhs - 2 * rhs;  
  
}
```


Phase 2 : Syntax Analysis or

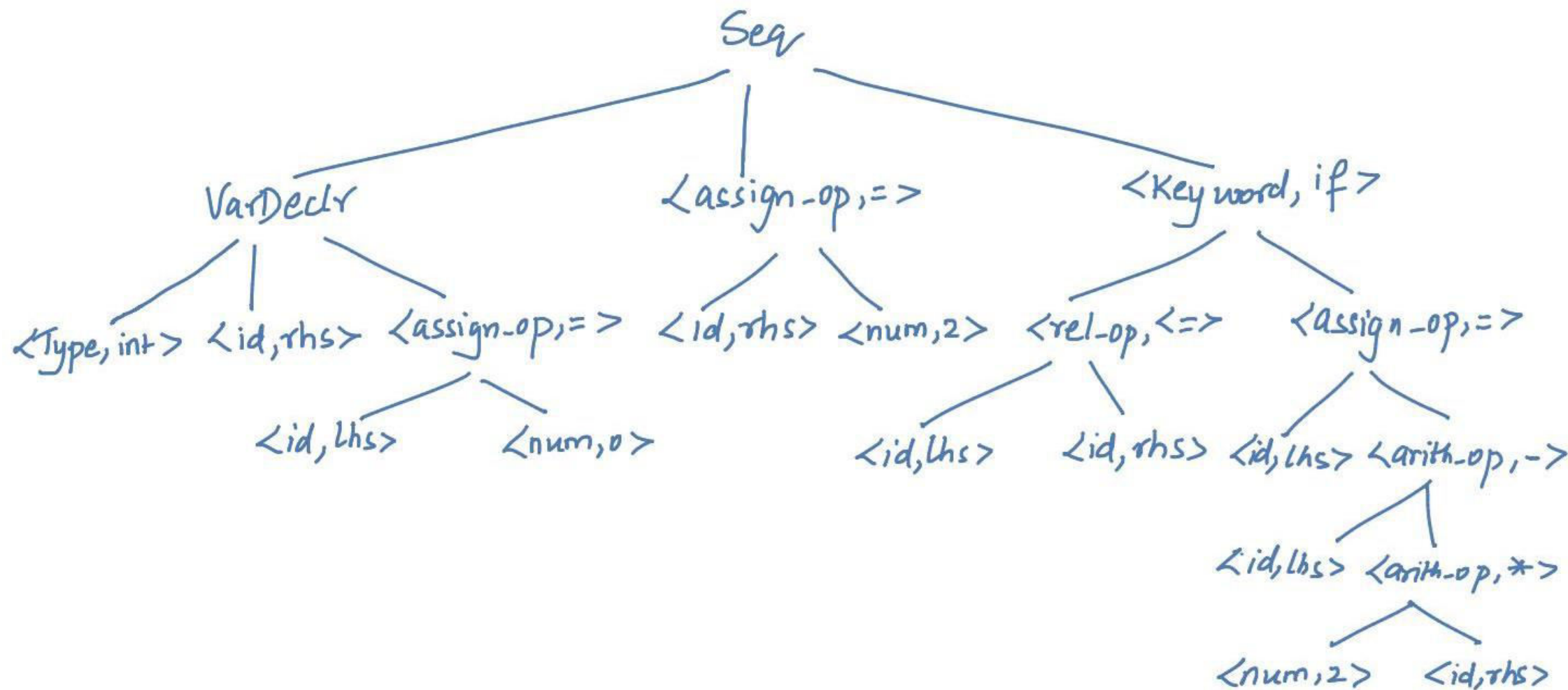


Use the grammar
we have defined

```
int rhs, lhs = 0;  
rhs = 2;  
if (lhs <= rhs)  
{  
  lhs = lhs - 2 * rhs;  
}
```


Phase 3 : Semantic Analyzer

Output: Semantically checked Syntax Tree



```
int rhs, lhs = 0;
rhs = 2;
if (lhs <= rhs)
{
    lhs = lhs - 2 * rhs;
}
```

Phase 4 : Intermediate Code Generator

Output: **Three Address Code (3AC or**

TAC)

```
lhs = 0
rhs = 2
t0 = lhs <= rhs
if t0 goto L1
goto L2
L1 :   t1 = 2 * rhs
      t2 = lhs - t1
      lhs = t2
L2 : next
```

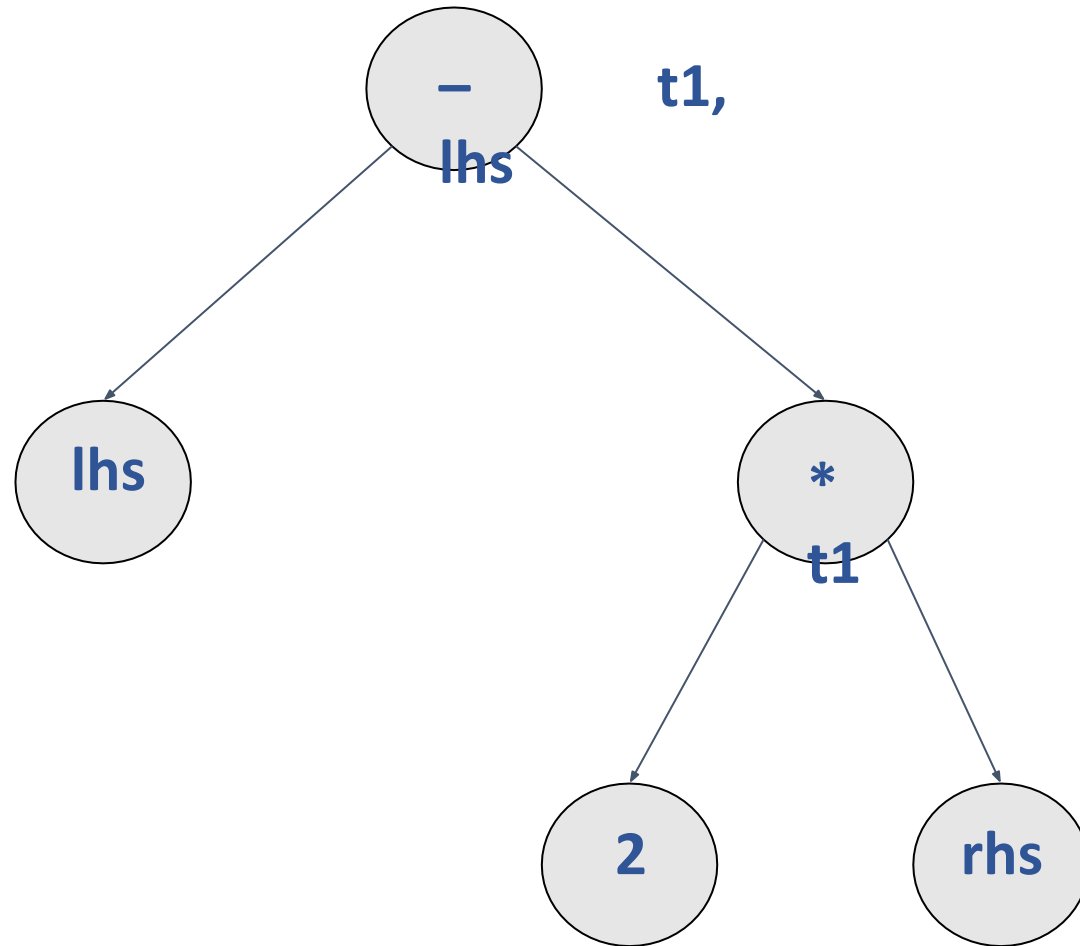
```
int rhs, lhs = 0;

rhs = 2;

if (lhs <= rhs)
{
    lhs = lhs - 2 * rhs;
}
```

Phase 5 : Machine Independent Code

Optimization Output: **Optimized IR**



```
lhs = 0  
rhs = 2  
t0 = lhs <= rhs  
if t0 goto L1  
goto L2  
L1: t1 = 2 * rhs  
    lhs = lhs - t1  
L2: next
```

```
int rhs, lhs = 0;  
rhs = 2;  
if (lhs <= rhs)  
{  
    lhs = lhs - 2 * rhs;  
}
```

Phase 6 : Code

Generator Output:

Assembly Code

```
MOV    R1, #0
ST      lhs, R1
MOV    R2, #2
ST      rhs, R2
SUB     R3, R1, R2
BLTZ   R3, L2
LD      R4, #2
MUL     R4, R2
SUB     R5, R1, R4
ST      R5, lhs
L2 : ...
```

```
int rhs, lhs = 0;

rhs = 2;

if (lhs <= rhs)
{
    lhs = lhs - 2 * rhs;
}
```

Example 3 : Consider the following piece of code:

```
n = 23;  
for (i = 0; i < n; i++)  
{  
    sum = sum * i;  
}
```

We will now run this code through the different phases of a compiler and see what the output will be at each stage.

Phase 1 : Lexical Analysis or Linear Analysis or Scanner

Pattern	Lexemes matched
Keyword	for
Identifier	n i i n i sum sum i
RelOp (<, <=, >, >=, ==, !=)	<
LogOp (&&, , ~)	
Assign (=)	= = =
ArithOp (*, +, -, /)	*
Punctuation ((,), {, }, [,], ;)	; (;) { ; } ;
Number	23 0
Literal – anything in double quotes	
Inc_op	++

```
n = 23;
for (i = 0; i < n; i++)
{
    sum = sum * i;
}
```

Output: # of tokens : 25

< ID, 1 >
< = >
< Number, 23 >
< Keyword, for >
< (>
< ID, 2 >
< assign_op, =>
< Number, 0 >
< ; >
< ID, 2 >
< < >
< ID, 1 >
< ; >
< ID, 2 >
< Inc_op, ++ >
<) >

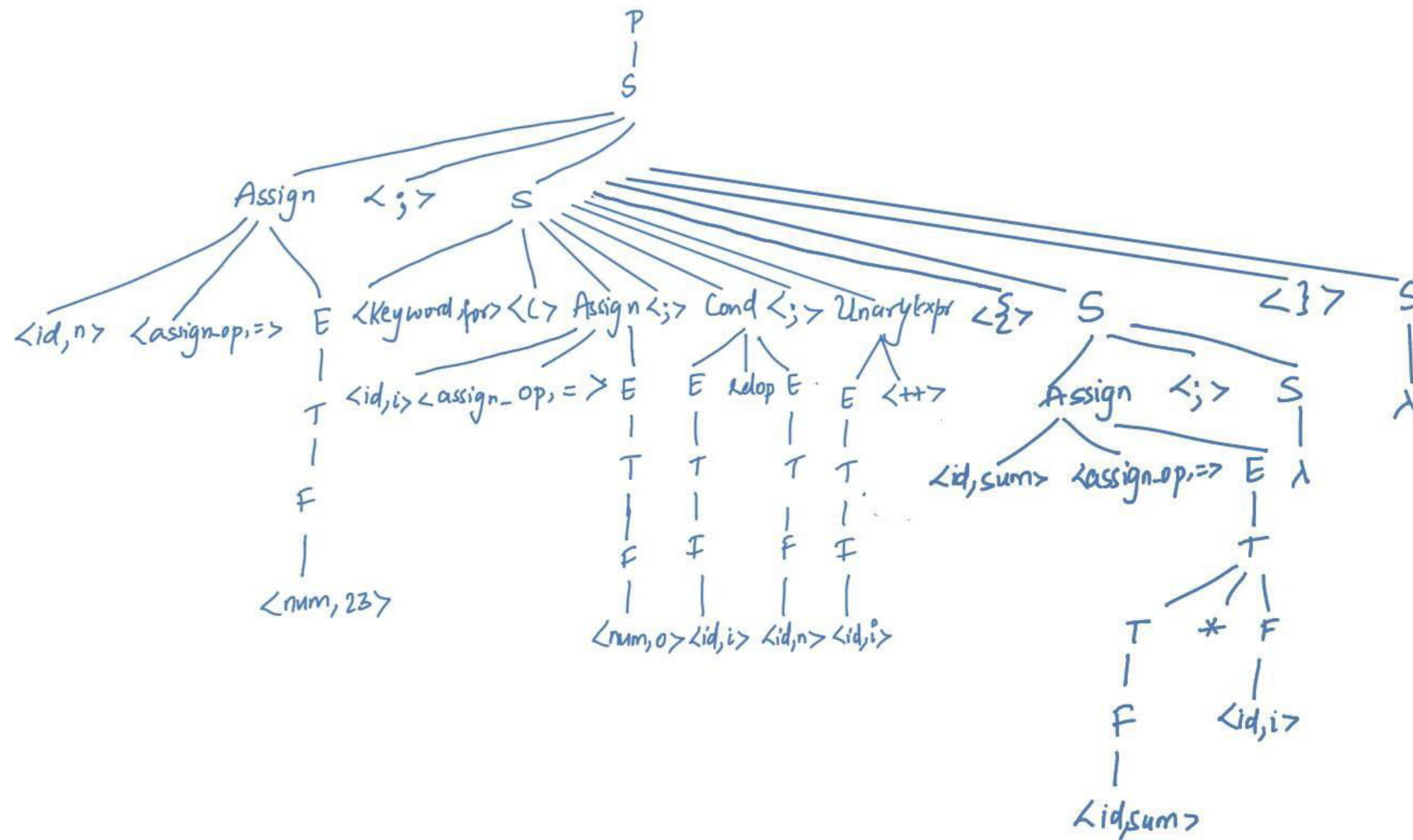
Symbol
Table

	name	type	...
1	n		
2	i		
3	sum		
	...		

< { >
<ID, 3>
<assign_op, =>
<ID, 3>
<arith_op, *>
<ID, 2>
< ; >
< } >

```
n = 23;  
  
for (i = 0; i < n; i++)  
{  
    sum = sum * i;  
}
```


Phase 2 : Syntax Analysis or

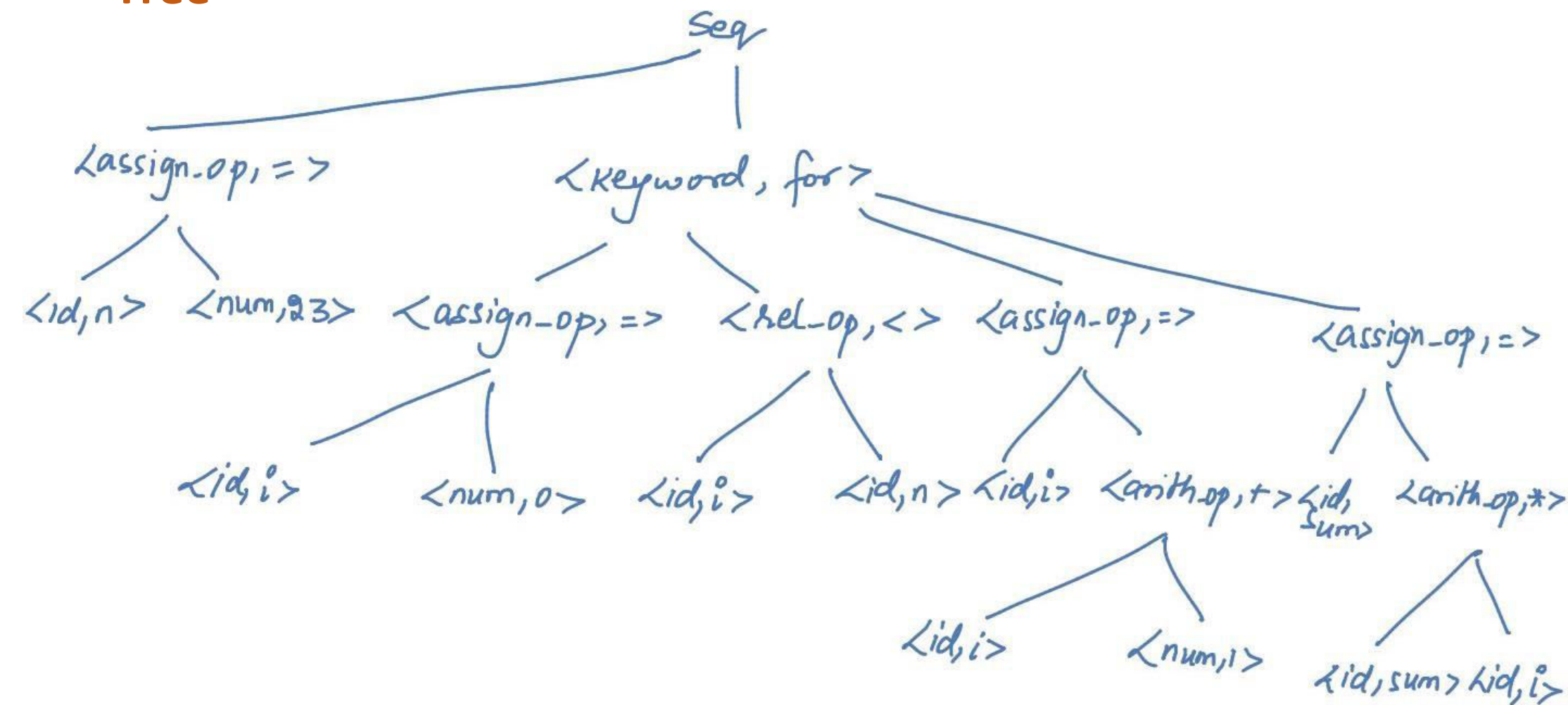


Assume slightly
modified
grammar (with
functions)

```
n = 23;
for (i = 0; i < n; i++)
{
    sum = sum * i;
}
```


Phase 3 : Semantic Analyzer

Output: Semantically checked Syntax Tree



```
n = 23;
for (i = 0; i < n; i++)
{
    sum = sum * i;
}
```

Phase 4 : Intermediate Code Generator

Output: **Three Address Code (3AC or**

TAC)

```
number = 23
```

```
i = 0
```

```
L0:    if i < n goto L1
```

```
      goto L2
```

```
L1:    t1 = sum * i
```

```
      sum = t1
```

```
      t2 = i + 1
```

```
      i = t2
```

```
      goto L0
```

```
L2 : ...
```

```
n = 23;
```

```
for (i = 0; i < n; i++)
```

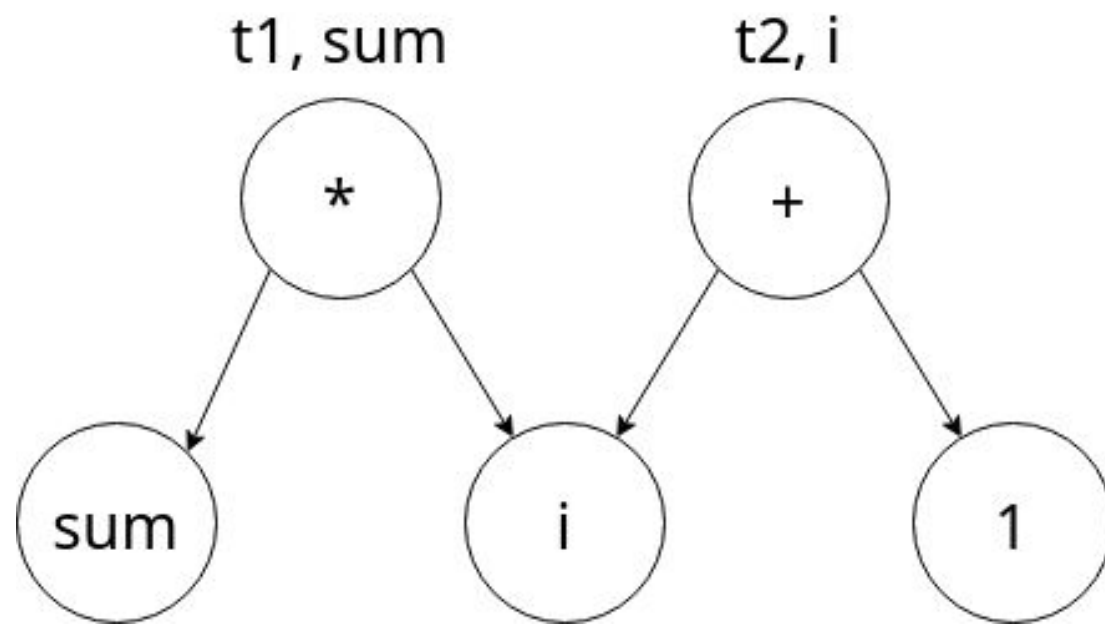
```
{
```

```
    sum = sum * i;
```

```
}
```

Phase 5 : Machine Independent Code

Optimization
Output: **Optimized IR**



```
number = 23
i = 0
L0: if i < n goto L1
    goto L2
L1: sum = sum * i
    i = i + 1
    goto L0
L2: ...
```

```
n = 23;
for (i = 0; i < n; i++)
{
    sum = sum * i;
}
```

Phase 6 : Code

Generator Output:

Assembly Code

```
MOV R1, #23          // R → n
MOV R2, #0           // R2 → i
MOV R4, sum          // R4 = contents(sum)
L0:  SUB R3, R1, R2  // i - n
     BLZ R3, L1
     BR L2
L1:  MUL R4, R4, R2
     BR L0
L2:  ...
```

```
n = 23;
for (i = 0; i < n; i++)
{
    sum = sum * i;
}
```

Exercise Problem : Consider the following piece of code:

```
int n = 3;
do
{
    if( (n/2)*2 == n )
    {
        n = n / 2;
    }
    else
    {
        n = 3 * n + 1;
    }
}
while(n != 1 || n != 2 || n != 4); return n;
```

This problem is for the understanding of the concept by testing you on if-else and do-while constructs.

Note: The Parse Tree may become large, try to do the if-else part separately

Hint: Consider $S \rightarrow \text{do } \{S\} \text{ while } (S); S$ in the grammar



**THANK
YOU**

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Gautham P Atreyas

Compiler Design

Unit 1 Lexical Analysis

Preet Kanwal

Department of Computer Science &
Engineering

Compiler Design

Lecture Overview

In this lecture, you will learn about:

- Tokens, Lexemes and Patterns
- The Lexical Analyzer
- Role of a Lexer
- Lexical Analyzer and Parser
- Speed of Lexical Analysis
 - Buffer Pairs
 - Sentinels
- Lexical Errors

Token : A token is a structure that holds a token name, and an optional attribute value. The token name is an abstract symbol representing a type of lexical unit (keywords, identifiers, etc). A token represents a sequence of characters that cannot be broken down further. It can be denoted as `**<token name, attribute value>**`.

Lexeme: A lexeme is a sequence of characters in the source program which is identified by the lexical analyzer as an instance of a token.

Pattern: A pattern is the description of the form that the lexemes of a token may take. It is a rule that defines the structure a token must follow. When a sequence of characters in a source program matches this rule, it is identified as a lexeme (an instance) of that token.

- For a keyword token, the pattern is just the string of characters that forms the keyword
- For an identifier token, the pattern is a more complex structure that is matched by many strings

The attribute value of a token is a pointer to a symbol table entry, and contains additional information that is required by a parser. It is more likely for a token to contain the starting and ending position of the lexeme, rather than the sequence of characters itself. This is also used for tasks such as error reporting.

regex101 is a very useful tool for writing and testing patterns

Token	Informal Description	Pattern	Lexemes
if	Characters `i` and `f` together as a whole word	if	if
else	Characters `e`,`l`,`s` and `e` together as a whole word	else	else
Comparison	`<` or `>` or `<=` or `>=` or `==` or `!=`	< > <= > = == !=	<=,!=
id	Letter followed by letters and digits	[a-zA-Z_]+[a-zA-Z0-9_]*	Sum, avg1, pi

regex101 is a very useful tool for writing and testing patterns

Token	Informal Description	Pattern	Lexemes
Number	Any numeric constant	digit: [0-9] exponent: [eE][-+]?{digit}+ {digit}+(\.{digit}+)?{exponent}?	10001, 3.14

Exercise 1: Write the pattern for matching a String Literal. It should match everything that is surrounded by `""`, except `""`.

Exercise 2: Enhance the pattern to handle escape sequences appropriately.

Exercise 3: Does the order of defining patterns matter? What happens if the pattern for matching a String Literal appears before the pattern for matching keywords like if or else?

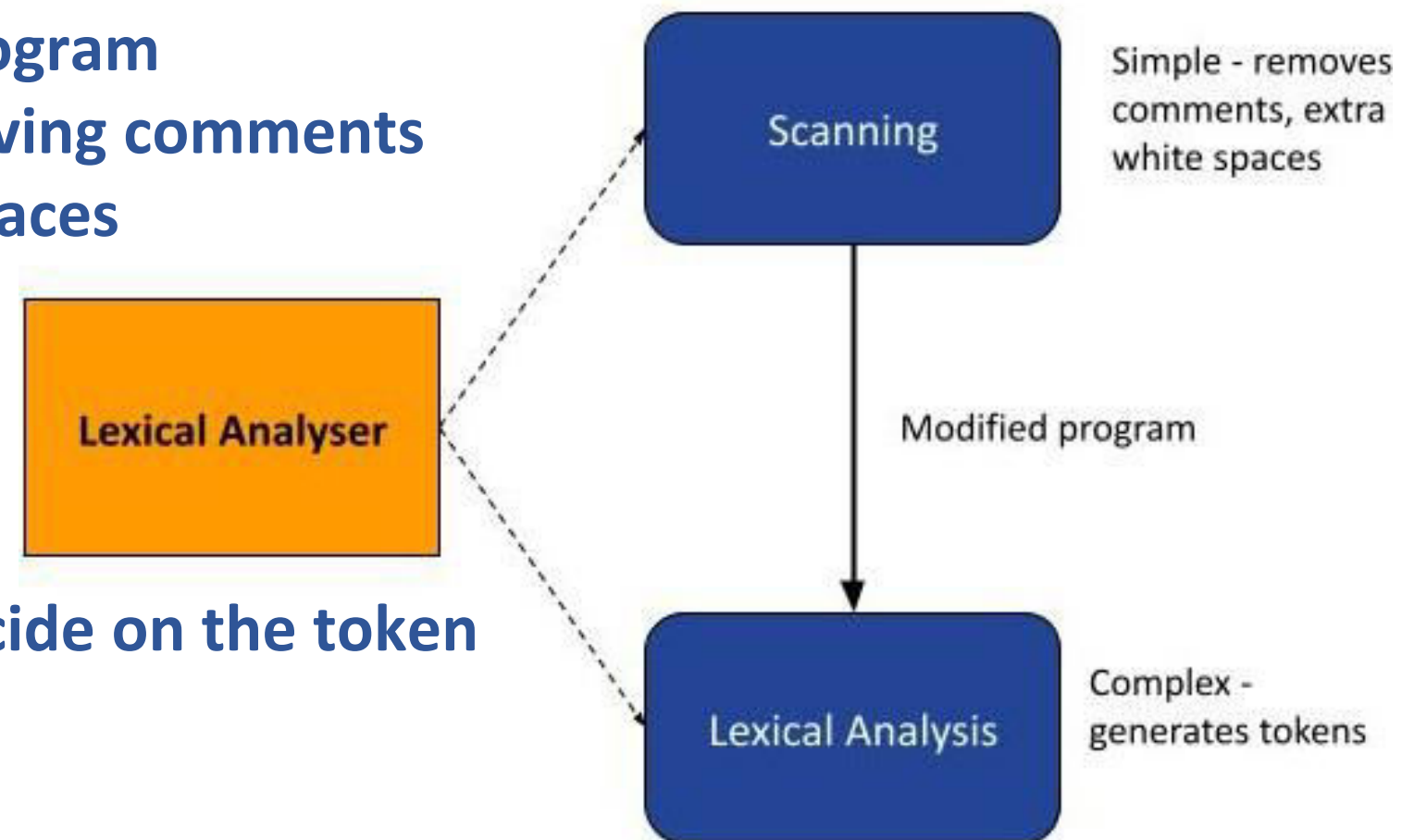
- The Lexical Analyzer is the first phase of the compiler, and is language dependent.
- It performs two tasks:

1. Scanning:

- Reading the characters of the source program
- Performing simple operations like removing comments and compacting consecutive whitespaces

1. Lexical Analysis:

- Produce tokens as output
- A lexical analyzer may need to read ahead some characters before it can decide on the token to be returned to the parser



The Lexical Analyzer has the goal of converting the source program into a sequence of tokens, by grouping the characters of the source program into lexemes.

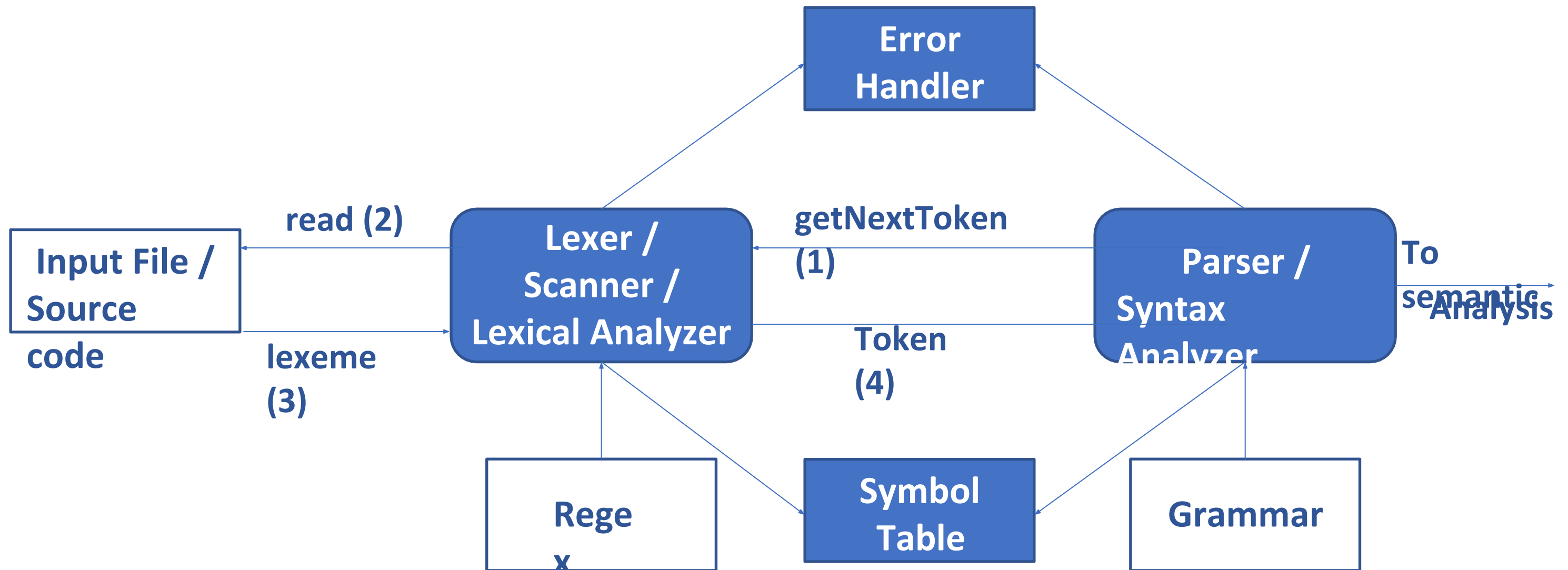
- Each token represents one logical piece of the source file - a keyword, the name of a variable etc.
- Each token is associated with a lexeme, i.e, the actual text of the token: "137," "int," etc.
- Each token can have optional attributes - extra information derived from the text, perhaps a numeric value.

The token sequence will be used in the Parser for Syntax Analysis and to recover the program structure. When a lexeme representing an identifier is discovered, it is entered into the symbol table.

The Lexical Analyzer also performs the following secondary tasks:

- Stripping out comments and white space like blank, new line, tab and other characters that are used to separate tokens in the input.
- Correlating error messages generated by the compiler with the source program file.
- Keeping track of the number of the newline characters and associate each error message with its line number.
- Making a copy of the source program with error messages inserted at appropriate positions.
- Expanding macros

There is tight coupling between the Lexical Analyzer and the Parser: the parser works as the master program and directs the lexical analyzer.



Then why are they separate?

1. **Simplicity of design**: for example, comments are removed by the lexer, and the parser need not worry about them (if the parser needs to handle comments, it will increase the complexity of the parser). Separating lexical and syntactical concerns leads to a cleaner design.
2. **Efficiency** is improved: specialized buffering schemes can be utilized for reading characters, which improves the overall speed of the compiler.
3. **Compiler portability** is enhanced: input specific peculiarities can be restricted to the lexical analyzer.

This post goes over several other reasons in detail, and is a great read.

Lexical Analyser	Parser
Scan Input program	Perform syntax analysis
Identify Tokens	Create an abstract representation of the code
Insert tokens into Symbol Table	Update symbol table entries
It generates lexical errors	It generates a parse tree of the source code

- The lexical analyzer reads characters of the source program one at a time to discover tokens. Thus, speed is a concern.
- In many languages, there are scenarios when the lexer needs to look ahead at least one character before a match can be announced.
Example: single-character operators like ``-``, ``=``, or ``<`` could also be the beginning of a two-character operator like ``->``, ``==``, or ``<=``

To solve these issues, the lexical analyzer utilizes **Input Buffers**. There are many schemes that can be used to buffer input, but in this course we will discuss:

1. Buffer Pairs

1. Sentinels

- This scheme makes use of two buffers of the same size(N). Usually, $N = \text{size of disk block} = 4096 \text{ bytes}$.
- These buffers are alternately reloaded. Reloading here means to read in the next N characters into the buffer.
- A single system read can be used to read N characters into the buffer, instead of one system call per character.
- If fewer than N characters remain in the input file, EOF is read into the buffer, marking the end of the source file.
- Two pointers are maintained to the input buffer:
 - lexeme_begin** - marks the beginning of the current lexeme
 - forward** - scans ahead till a pattern is found.

- Initially, both pointers point to the first character of the next lexeme to be found.
- The forward pointer scans ahead, till it crosses the end of the current lexeme as determined by the pattern.
- The string of characters between the two pointers is the current lexeme.
- The forward pointer is then retracted back by one position, which will result in the pointer being at the end of the current lexeme.
- After the lexeme is processed, both pointers are set to the character immediately after the lexeme.

The algorithm of handling the forward pointer when it reaches the end of a buffer:

if *forward* at end of first buffer **then begin**

 reload second buffer;

forward := *forward* + 1

end

else if *forward* at end of second buffer **then begin**

 reload first half;

 move *forward* to beginning of first buffer

end

else

forward := *forward* + 1;

Consider the following statement

- **abc = pqr * xyz ;**

: a : b : c : = : p :

abc = pqr * xyz ;

Initially, both pointers are set to the beginning of the buffer.



abc = pqr * xyz ;

Forward pointer advances till a pattern is matched.



abc = pqr * xyz ;

Forward pointer advances till a pattern is matched.



abc = pqr * xyz ;

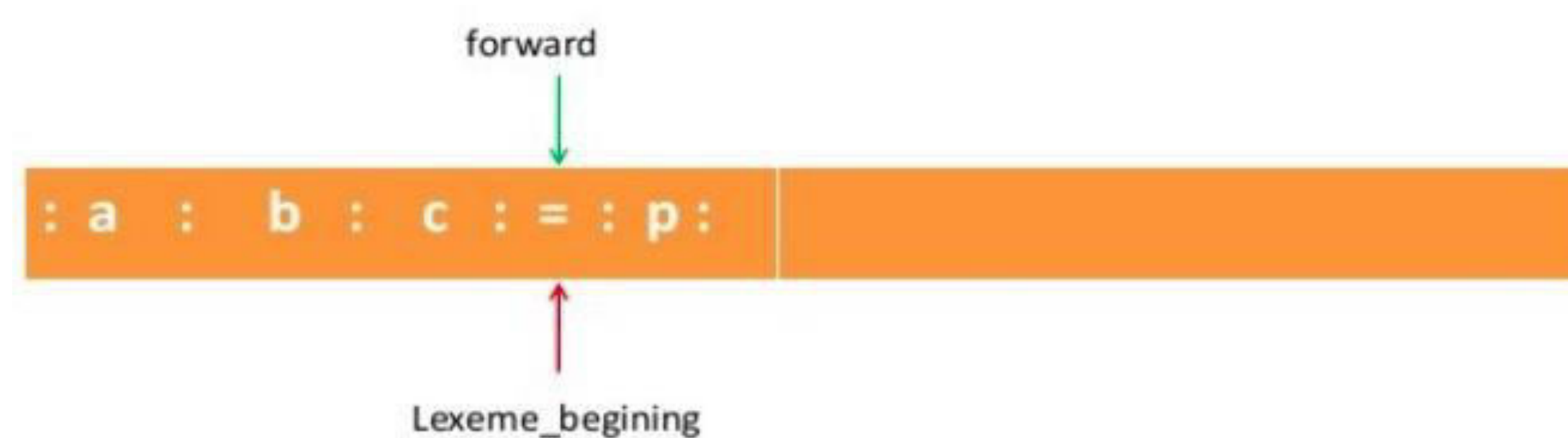
Here, a pattern is matched. The current lexeme lies between the two pointers.

Current lexeme: **abc**



abc = pqr * xyz ;

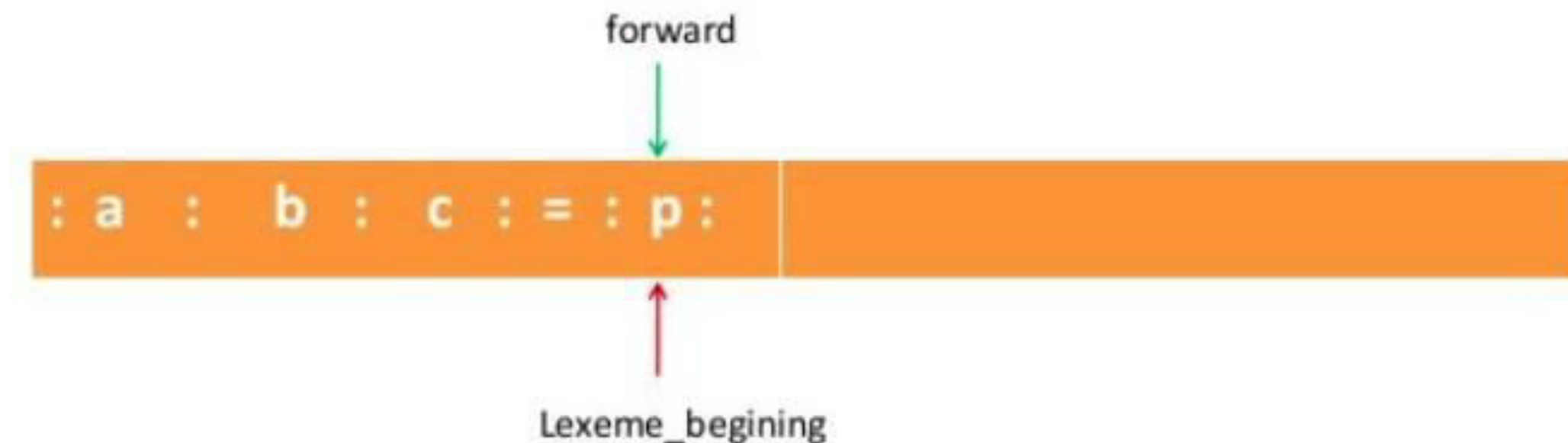
Both **forward** and **lexeme_begin** are set to the next character after the lexeme.



abc = pqr * xyz ;

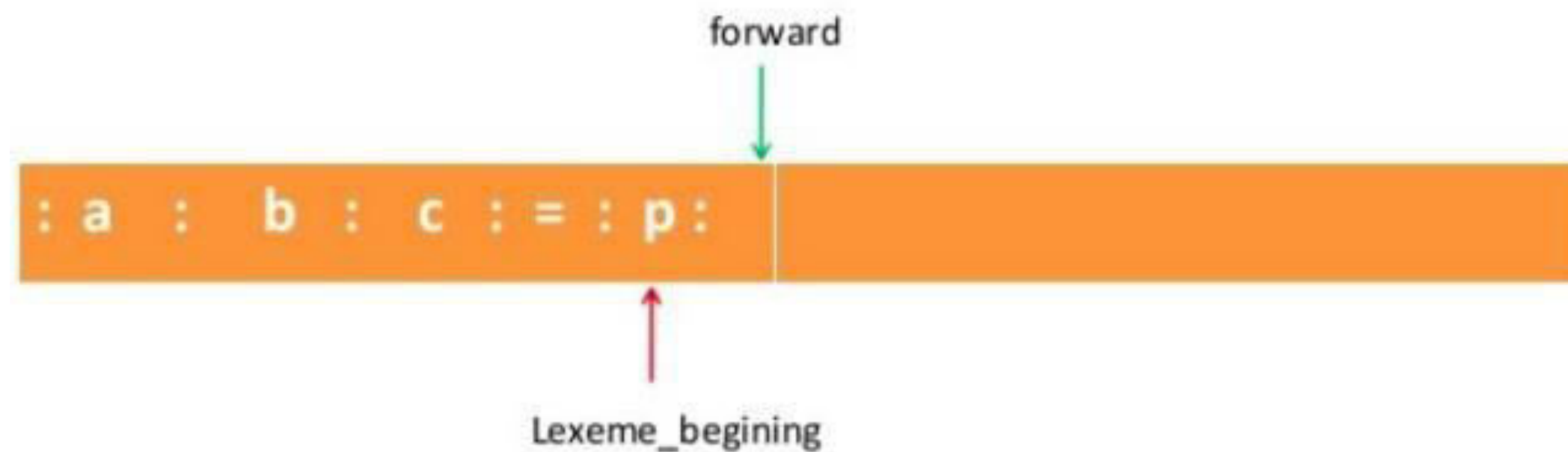
The current lexeme is =

Both forward and lexeme_begin are moved to the next character after the current lexeme.



abc = pqr * xyz ;

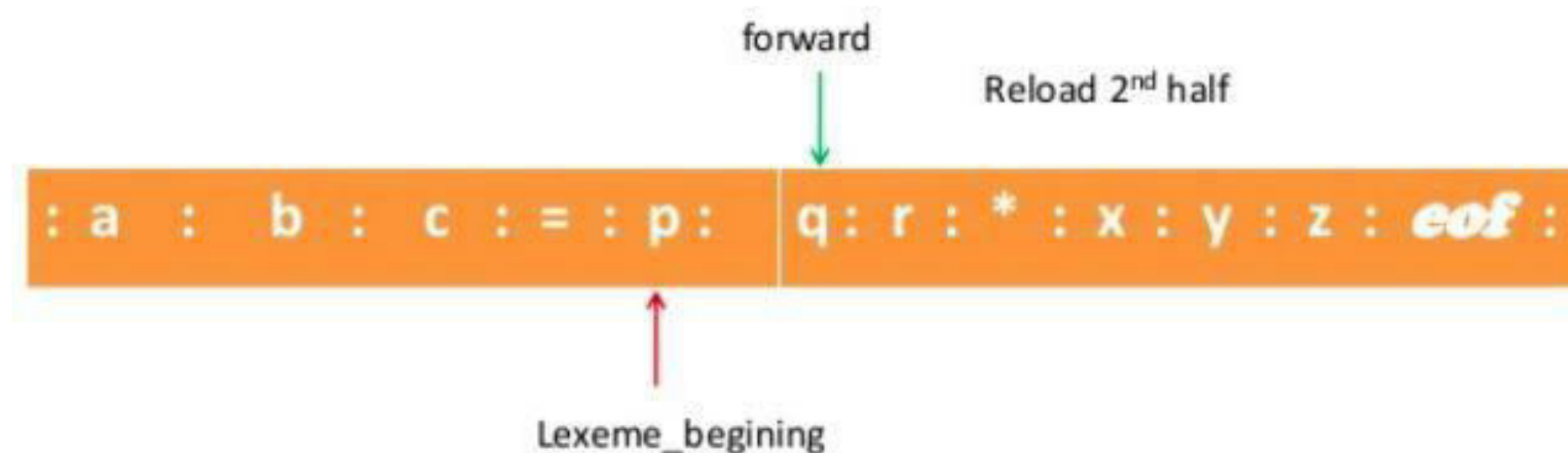
The forward pointer has reached the end of the first buffer.



abc = pqr * xyz ;

The second input buffer is reloaded. Forward pointer moves ahead till the next pattern is matched.

This process continues till EOF is reached.



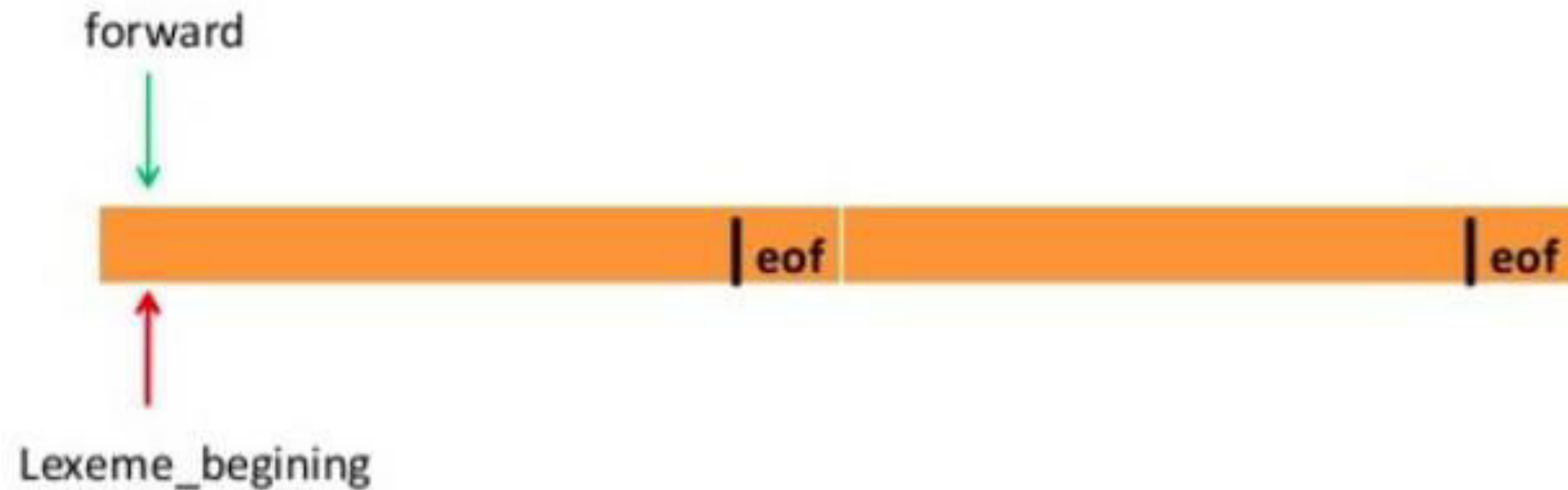
The drawbacks of the buffer pairs algorithm is that each time we advance the forward pointer, there are two checks that need to be performed:

1. For End of Buffer
2. To determine which character is read

Solution: Combine the End of Buffer check with the check for current character.

This can be done by extending each buffer to hold a **Sentinel** character (EOF) at the end. We only determine what character is read, and if it is a Sentinel, we reload the next buffer.

Sentinels (eof) are added to the end of each input buffer.



```
forward = forward + 1;  
if forward = eof then begin  
    if forward at end of first buffer then begin  
        reload second half;  
        forward := forward + 1  
    end  
    else if forward at end of second buffer then begin  
        reload first half;  
        move forward to beginning of first half  
    end  
    else  
        terminate lexical analysis  
        /* eof within buffer, i.e, end of input */  
end
```

Suppose a situation arises in which the lexical analyzer is unable to proceed because none of the patterns for tokens matches any prefix of the remaining input. Some possible scenarios are:

1. Spelling error
2. Unmatched String
3. Appearance of illegal characters
4. Exceeding the length of identifiers

In such cases, when writing an error message, it is important that the message satisfies the following properties:

- It must pinpoint the error correctly
- It must be understandable
- It must be specific
- It must not have any redundancy

1. **Spelling error**
2. Unmatched string
3. Appearance of illegal characters
4. Exceeding length of identifiers

```
di {...  
.....}  
while() ;
```

1. Spelling error
2. Unmatched string
3. Appearance of illegal characters
4. Exceeding length of identifiers

```
printf("hi) ;  
  
printf("hi") ;
```

1. Spelling error
2. Unmatched string
3. Appearance of illegal characters
4. Exceeding length of identifiers

```
printf("hi") $$;
```


1. Spelling error
2. Unmatched string
3. Appearance of illegal characters
4. Exceeding length of identifiers

```
int  
this_is_a_very_long_global  
_identifier_and_is_not_legal  
in_c;
```

External and global identifiers in C cannot exceed 31 characters.



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Kavya P

K

Compiler Design

Lex Tutorial

Preet Kanwal

Department of Computer Science &
Engineering

Compiler Design

Lecture Overview

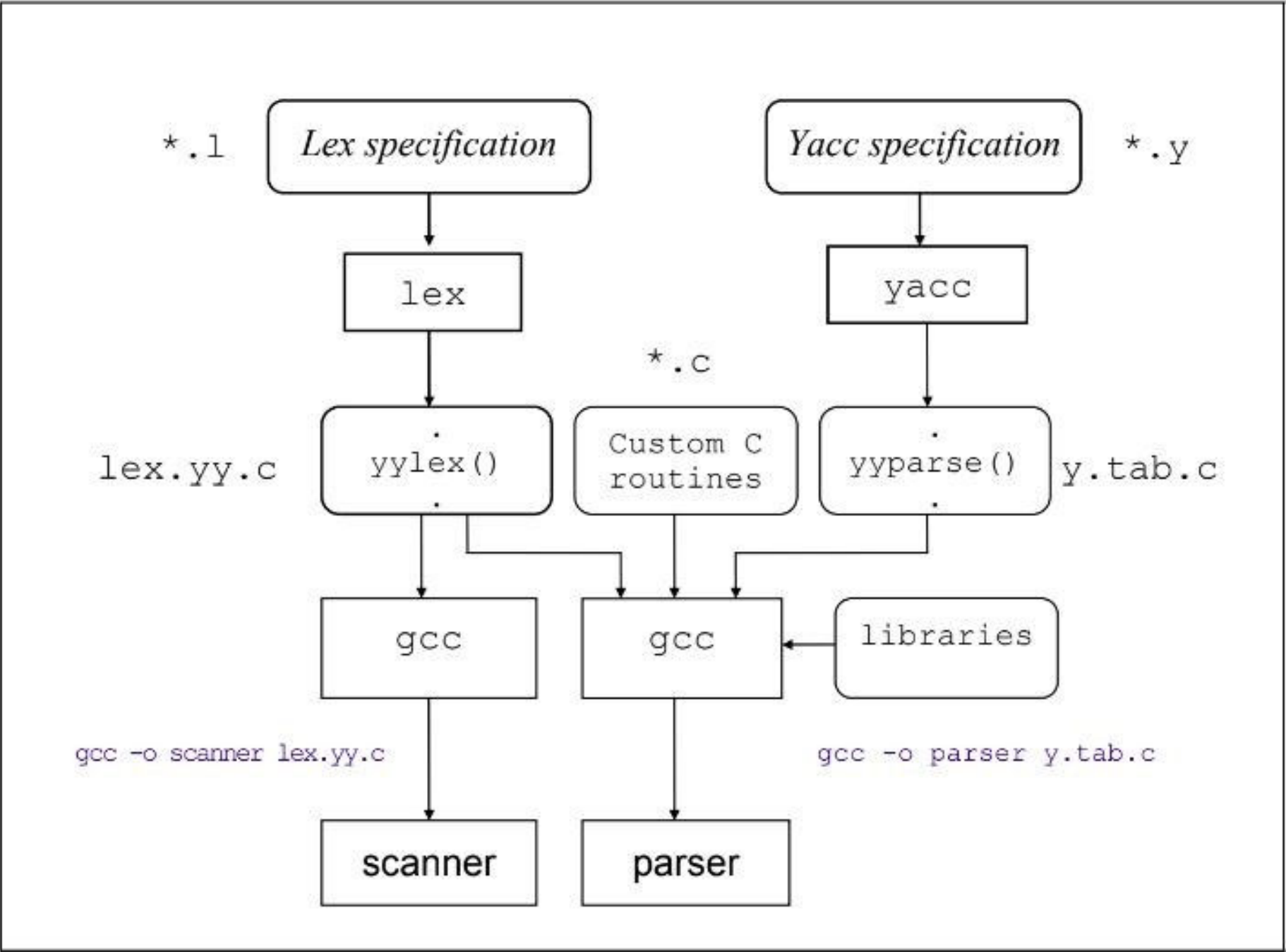
In this lecture, you will learn about:

- What is Lex and YACC?
- Flow of code
- How to write a Lex Specification file?
- Example 1 - A simple program
- Predefined variables in lex
- Example 2 - Prepend Line Numbers
- Example 3 - Use of definitions
- Example 4 - Word count
- Example 5 - Removing Comments
- Start conditions
- Commonly used Regular expressions in lex specification file

- **Lex** - reads a specification file containing regular expressions and generates a C routine that performs lexical analysis. Matches sequences that identify tokens.
- **YACC** - reads a specification file that codifies the grammar of a language and generates a parsing routine.

Compiler Design

Flow of Code



- The Lex specification file is divided into three sections with **%%** dividing the sections.
- The structure of a lex specification file is as follows:

```
definitions
%%
regular expressions and associated actions (rules)
%%
user routines
```

- The Lex specification file has a `.l` prefix

- Input is copied to output one character at a time.
- The first **%%** is always required as there must always be a rules section.
- However, if we don't specify any rules, then the default action is to match everything and copy it to output.
- Defaults for input and output are stdin and stdout.
- Any source code not intercepted by lex is copied into the generated program.
 - A line that is not part of a lex rule or action, which begins with a blank or tab, is copied out as above (useful for global declarations)
 - Anything included between lines containing only **%{** and **%}** which is useful for preprocessor statements that must start in col 1
 - Anything after the second **%%** delimiter is copied out after the lex output, which is useful for local function definitions

- Definitions intended for lex are given before the first `%%`. Anyline in this section that does not begin with a blank or tab, or is not enclosed by `%{...%}`, is assumed to be defining a lex substitution string of the form
name translation , for e.g, **letter [a-zA-Z]**
- `yytext` : a character array that contains the actual string that matches a pattern.
- `yytext` : the no. of characters matched.
- Values associated with the token are returned by lex in a union variable **yyval**.
- Local variables can be defined.
- Do not leave extra spaces and/or empty lines at the end of the lex specification file.

```
[0-9]+ { yyval = atoi(yytext);  
return INTEGER;}
```

- Here, two patterns have been specified in the rules section.
- Each pattern must begin in column one.
- This is followed by whitespace (space, tab or newline) and an optional action associated with the pattern.
- The action may be a single C statement, or multiple C statements enclosed in braces.
- Anything not starting in column one is copied to the generated C file. This behavior can be to specify comments in the lex file.
- In this example there are two patterns, "." and "\n", with an ECHO action associated for each pattern.

```
%%  
. ECHO;  
\n ECHO;  
%%  
int yywrap() {  
    return 1;  
}  
int main(void) {  
    yylex();  
    return 0;  
}
```

Program file - **echo.l**

- Several macros and variables are predefined by lex.
 - **ECHO** is a macro that writes code matched by the pattern. This is the default action for any unmatched strings. Typically, ECHO is defined as:
#define ECHO fwrite(yytext, yyleng, 1, yyout)
- Function **yywrap** is called by lex when input is exhausted. Return 1 if done, 0 if more processing is required.
- Every C program requires a main function. In this case we simply call **yylex** which is the main entry-point for lex.
- Some implementations of lex include copies of main and yywrap in a library thus eliminating the need to code them explicitly.

```
%%  
. ECHO;  
\n ECHO;  
%%  
int yywrap() {  
    return 1;  
}  
int main(void) {  
    yylex();  
    return 0;  
}
```

Program file - **echo.l**

Table : Lex Predefined Variables	
Name	Function
<code>int yylex(void)</code>	call to invoke lexer, returns token
<code>char *yytext</code>	pointer to matched string
<code>yylen</code>	length of matched string
<code>yyval</code>	value associated with token
<code>int yywrap(void)</code>	wrapup, return 1 if done, 0 if not done
<code>FILE *yyout</code>	output file
<code>FILE *yyin</code>	input file
<code>INITIAL</code>	initial start condition
<code>BEGIN condition</code>	switch start condition
<code>ECHO</code>	write matched string

```
$ lex echo.l      # generate C file which implements the scanner
$ gcc lex.yy.c    # produce an executable scanner
$ ./a.out
$ ./a.out < input.txt # you can redirect input to read in files
```

- This example prepends line numbers to each line in a file.
- Some implementations of lex predefine and calculate `yylineno`.
 - **%option yylineno** is added to the definitions
 - Check program file : **start_conditions.l**
- Here, the input file for lex is **yyin** (which defaults to `stdin`), which is initialised in the main function.
- Variable **yytext** is a pointer to the matched string (NULL-terminated) and **yyleng** is the length of the matched string.
- Variable **yyout** is the output file and defaults to `stdout`.

```
%{
    int yylineno;
}%
%%
(.*)\n    printf("%4d\t%s",
yylineno++, yytext);
%%
int yywrap() { return(1);
} int main() {
    yyin = fopen("input.txt",
    "r"); yylex();
    fclose(yyin);
}
```

Program file - **line_numbers.l**

- This example demonstrates the use of the definitions section.
- The definitions section is composed of substitutions, code, and start states.
- Code in the definitions section is simply copied as-is to the top of the generated C file and must be bracketed with **%{** and **%}** markers.
- Substitutions simplify pattern-matching rules.

```
digit    [0-9]
letter [_A-Za-z]
%{
#include<stdio.h>
%}
%%
{letter}({letter}|{digit})* printf("Identifier : %s",yytext);
. ;
\n ;
%%
int yywrap(){return(1);}
int main() {
    yyin = fopen("input.txt", "r");
    yylex();
    fclose(yyin);
    return 0;
}
```


- This example counts the number of characters, words, and lines in a file (similar to Unix wc)
- Whitespace must separate the defining term and the associated expression.
- References to substitutions in the rules section are surrounded by braces ({letter}) to distinguish them from literals.
- When we have a match in the rules section the associated C code is executed.

```
%{
    int nchar, nword, nline;
}%
%%
\n      { nline++; nchar++; }
[^ \t\n]+ { nword++, nchar += yyleng; }
.       { nchar++; }
%%
int yywrap(){return(1);}
int main(void) {
    yyin = fopen("input.txt", "r");
    yylex();
    printf("%d\t%d\t%d\n", nchar, nword, nline);
    return 0;
}
```

Program file - **counts.l**

- This example removes single line and multi line comments.

```
%%  
\\(.*) ;  
\\\"(.*)\".*\\\" ;  
%%  
int yywrap(){return(1);}  
int main()  
{  
    yyin=fopen("input.txt","r");  
    yylex();  
    return 0;  
}
```

Program file - **removing_comments.l**

- Start conditions are a mechanism for conditionally activating patterns.
- This is useful for handling -
 - Conceptually different components of an input
 - Situations where the lex defaults (e.g., “longest possible match”) don’t work, like in comments or quoted strings.
- Declare a set of start condition names using
%Start name1 name2 . . .
- If **scn** is a start condition name, then a pattern prefixed with **<scn>** will only be active when the scanner is in start condition **scn**.
- The scanner begins in start condition **INITIAL**, of which all non-<scn>-prefixed rules are members.

- Start conditions such as these are inclusive: i.e., being in that start condition adds appropriately prefixed rules to the active rule set.
- flex also allows exclusive start conditions (declared using %x), which are sometimes more convenient.
 - %x scn

Example: Detecting multi-line comments using start conditions

Program File - **start_conditions.l**

- This program makes use of start conditions to detect the beginning and end of a multiline comment.

Exercise: Count the number of digits after the decimal in a floating point number using start conditions. Is it possible to do without using start conditions?

Program File - **start_conditions_floats.l**

Table 1: Special Characters	
Pattern	Matches
.	any character except newline
\.	literal .
\n	newline
\t	tab
\\	matches character \
^	beginning of line
\$	end of line

Table 2: Operators	
Pattern	Matches
?	zero or one copy of the preceding expression
*	zero or more copies of the preceding expression
+	one or more copies of the preceding expression
a b	a or b (alternating)
(ab)+	one or more copies of ab (grouping)
abc	abc
abc*	ab abc abcc abccc ...
"abc*"	literal abc*
abc+	abc abcc abccc abccccc ...
a(bc)+	abc abcbcb abcbcbcb ...
a(bc)?	a abc

Table 3: Character Class	
Pattern	Matches
[abc]	one of: a b c
[a-z]	any letter a through z
[a\ -z]	one of: a - z
[-az]	one of: - a z
[A-Za-z0-9]+	one or more alphanumeric characters
[\t\n]+	whitespace
[^ab]	anything except: a b
[a^b]	one of: a ^ b
[a b]	one of: a b



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Gautham P Atreyas

Compiler Design

Unit 1

Design of a Lexical Analyzer Generator

Preet Kanwal

Department of Computer Science &
Engineering

Compiler Design

Lecture Overview

In this lecture, you will learn about:

- Maximal Munch Rule
- Lexical Analyzer with Lex
- Constructing Automata - RE to NFA to DFA
- Lookahead Operator in Lex (//)
- Program to identify keywords and identifiers

Given:

abb

printf("1")

aba

printf("2")

a

printf("3")

Questions:

Provide output for the String : a

Provide output for the String : ababa

A code-generation grammar is usually highly ambiguous, and some care needs to be given to how conflicts are resolved. Lex follows the principle of **Maximal Munch Rule**:

When more than one pattern can match the input, lex chooses as follows:

1. The longest match is preferred.
2. Among rules that match the same number of characters, the rule that occurs earliest in the list is preferred.

However, it is important to note that some lexical analyzers that follow this principle produce the longest possible translation at each step, even if it results in an incorrect program. For example, in Java and C

Given:

abb	printf("1")
aba	printf("2")
a	printf("3")

Questions:

- Provide output for the String : **a**
 - **Answer - 3**
- Provide output for the String : **ababa**
 - **Answer--2b3**
b
a - 3

Recall Panic
Mode
Recovery

Given -

$a*b$ `printf("1")`

$(a|b)*b$ `printf("2")`

c^* `printf("3")`

Questions:

- Provide output for the String : **cbabc**
- Provide output for the String : **cbbbbac**
- Provide a String such that the Output is **132**

a*b printf("1")

(a|b)*b printf("2")

c* printf("3")

- **Provide output for the String : cbabc**

- **Answer: 323**

- c - 3**

- bab - 2**

- c - 3**

- **Provide output for the String :
cbbbbac**

- **Answer - 32a3**

- c - 3**

- bbbb - 2**

- a - a**

- c - 3**

- **Provide a String such that the Output is
132**

- **Answer: abccbb**

- ab - 1**

- cc - 3**

- bb - 2**

- **Other answers are possible**

Given -

aa printf("1")

b?a+b? printf("2")

b?a*b? printf("3")

Questions:

- Provide output for the String : **bbbaabb**
- Provide a String such that the Output is **123**
- Provide a String such that the Output is **321**

- aa printf("1")
- b?a+b? printf("2")
- b?a*b? printf("3")
- Provide output for the String : **bbbaabb**
 - Answer - 323
 - bb - 3
 - baab - 2
 - b - 3
- Provide a String such that the Output is **123**
 - Not possible
- Provide a String such that the Output is **321**
 - bbbabaa
 - bb - 3
 - bab - 2
 - aa - 1

Give an example of such a set of regular expressions and an input string such that :

1. The string can be broken apart into substrings, where each substring matches one of the regular expressions BUT,
 1. The longest-prefix algorithm will fail to break the string in a way where each piece matches one of the regex.

Answer -

Suppose the patterns are as follows

a* **print("1")**

ab **print("2")**

bb **print("3")**

Let the pattern be **aabbb**

Here, **a** matches 1, **ab** matches 2 and **bb** matches 3

However, due to maximal munch rule, the longest match is found, and

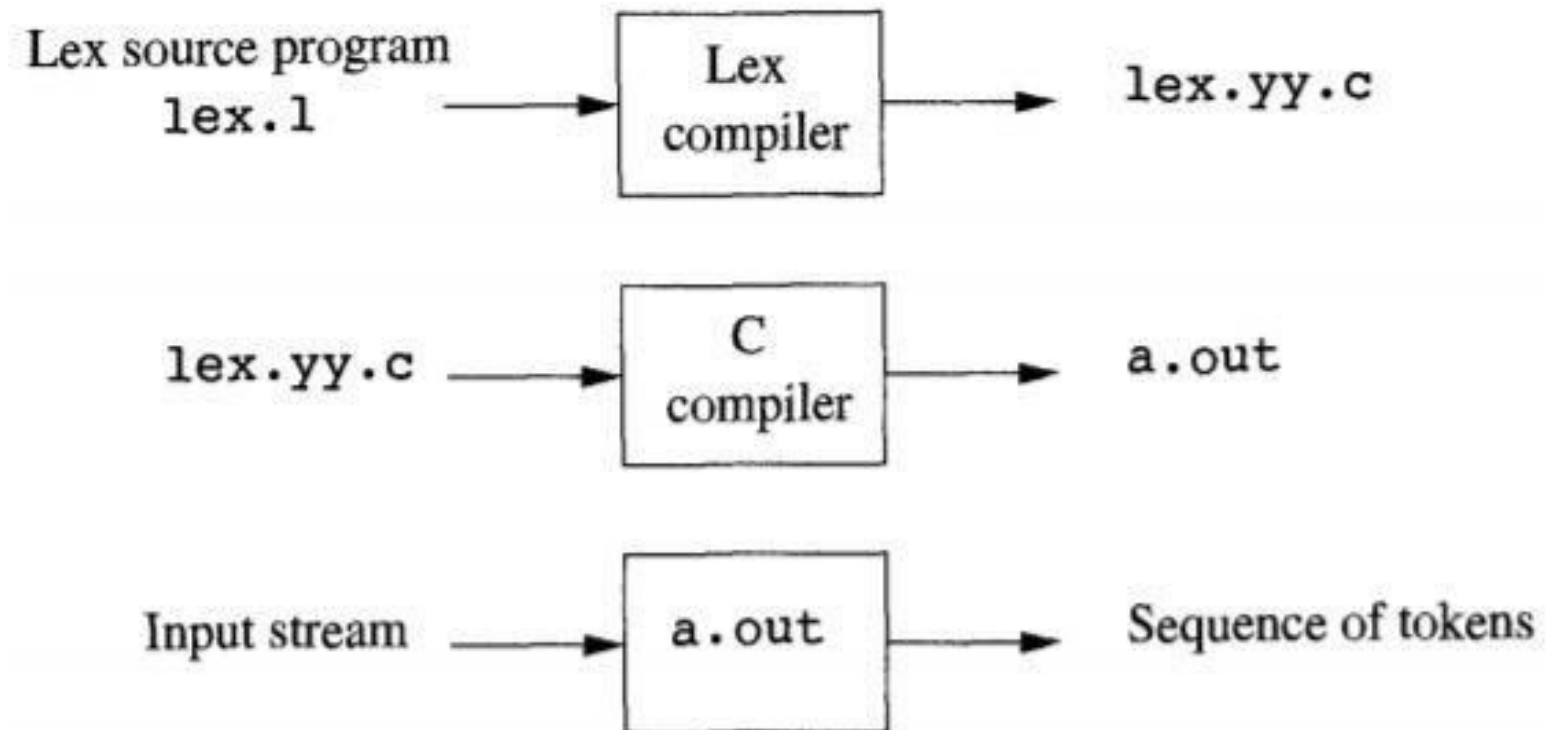
aa matches 1, **bb** matches 3 and last **b** is left unmatched. So output is
13b

Verify the answers of the previously solved examples using Lex.

- Example 1 - **example1.l**
- Example 2 - **example2.l**
- Example 3 - **example3.l**
- Example 4 - **example4.l**
 - In this program, lex shows a warning - **“rule cannot be matched”**, because the first rule always matches the lexemes matched by the other 2 rules.

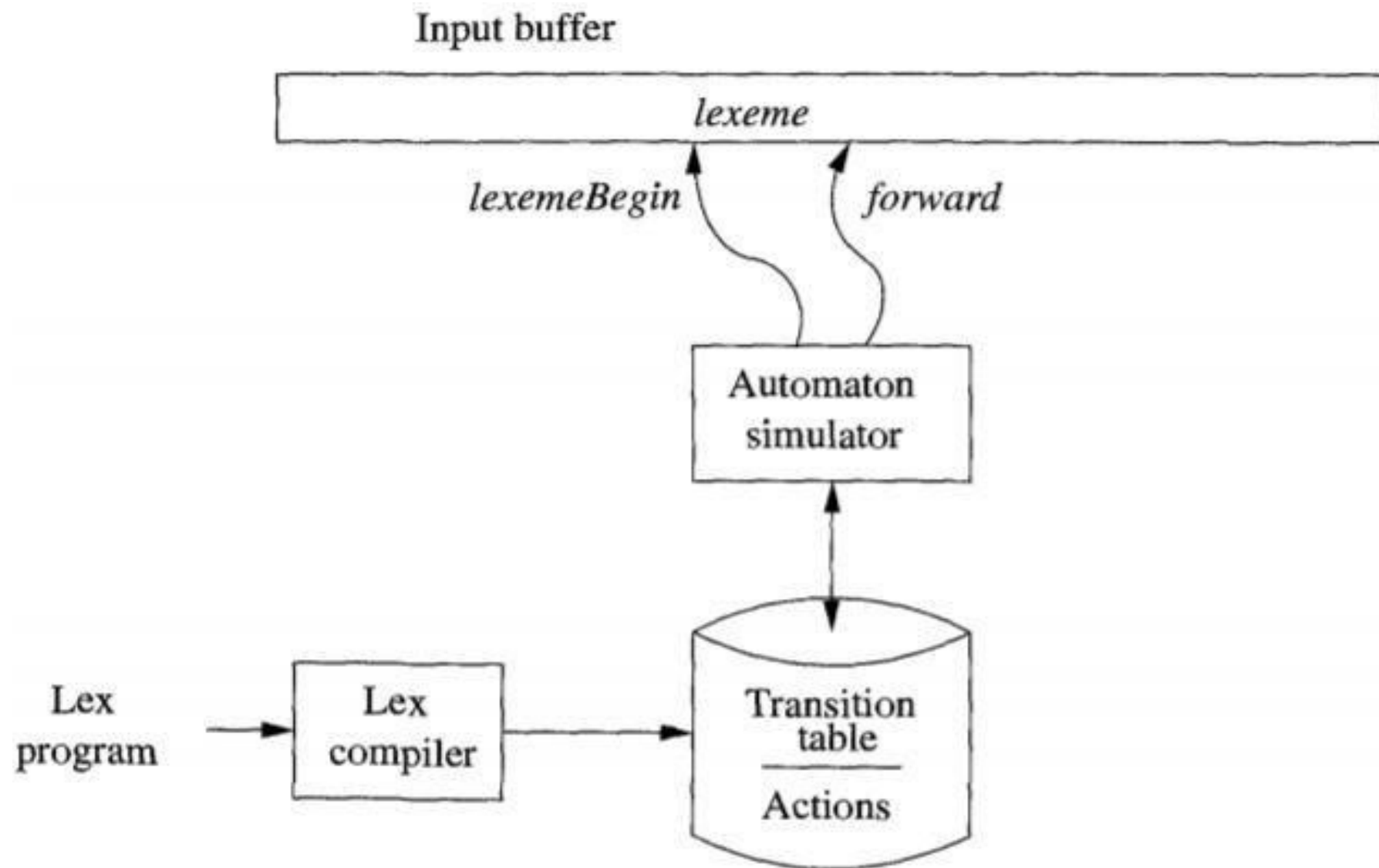
Compiler Design

Lexical Analyzer with Lex



Compiler Design

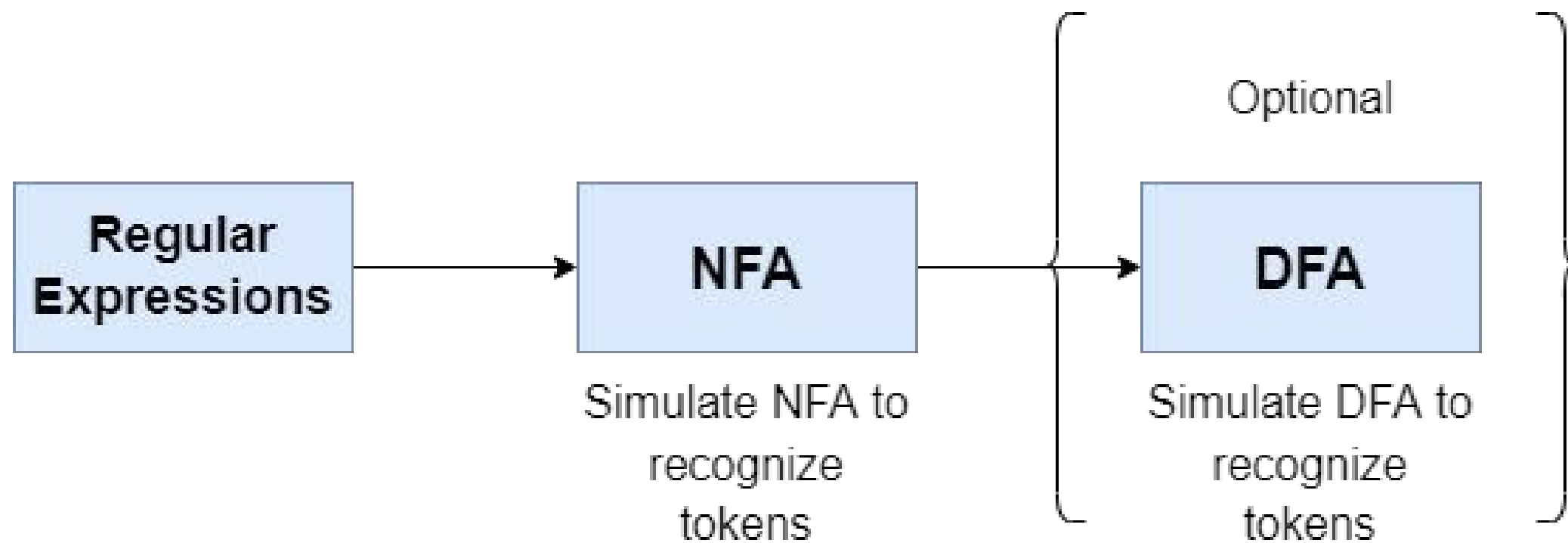
Lexical Analyzer with Lex



The generated lexical analyzer consists of components that are created from the lex specification file.

These components are -

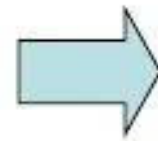
- **Transition Table** - Created for all patterns defined in the lex program.
- **Actions** - Fragments of code defined to their corresponding patterns.
 - Actions are invoked by the Automata Simulator at the appropriate time.
- **Functions** - Defined in the auxiliary functions section of the lex program, these are passed directly through lex to the output file.
- **Automata Simulator** - The program that serves as the lexical analyzer and uses the components mentioned above.
 - It simulates an automata (NFA or DFA)



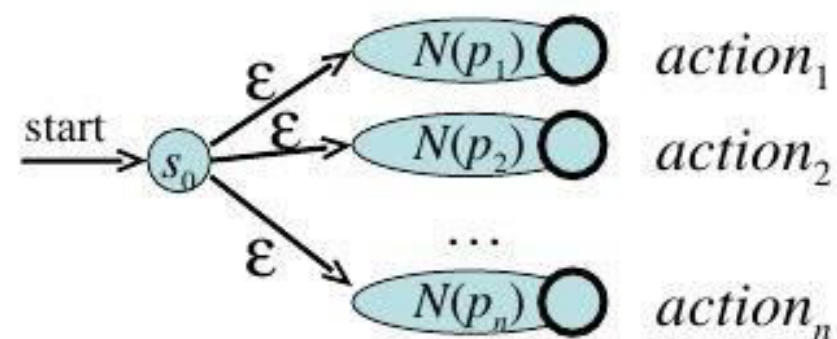
- Convert each pattern (regular expressions) in the lex program to an NFA.
- Combine all NFAs into one using a new start state, with ϵ transactions to each of the start states of the NFAs.
- Convert the combined NFA to DFA.

Lex specification with
regular expressions

p_1	$\{ action_1 \}$
p_2	$\{ action_2 \}$
...	
p_n	$\{ action_n \}$



NFA



Subset construction

DFA

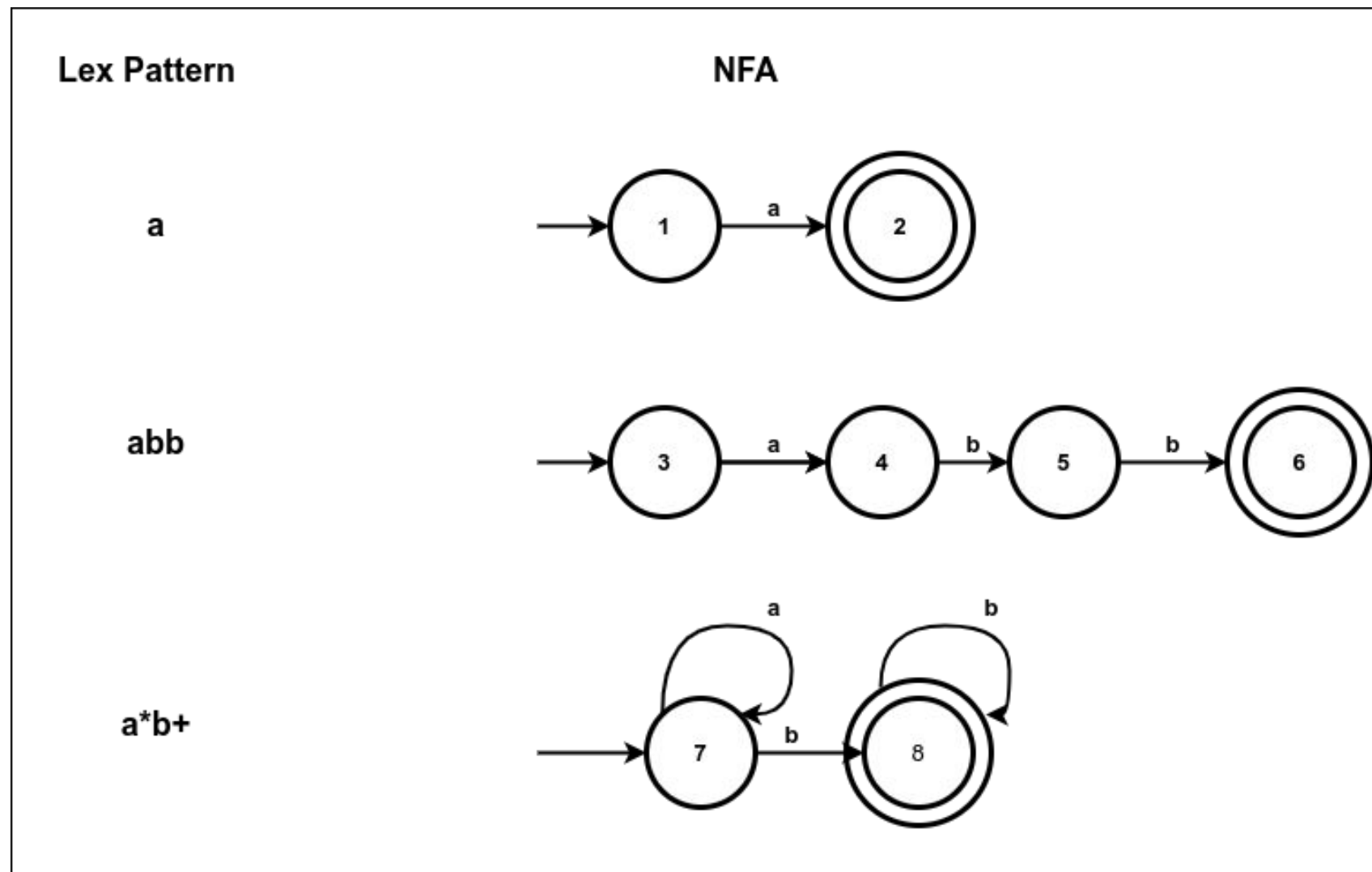
Given patterns and corresponding actions

a **A1**

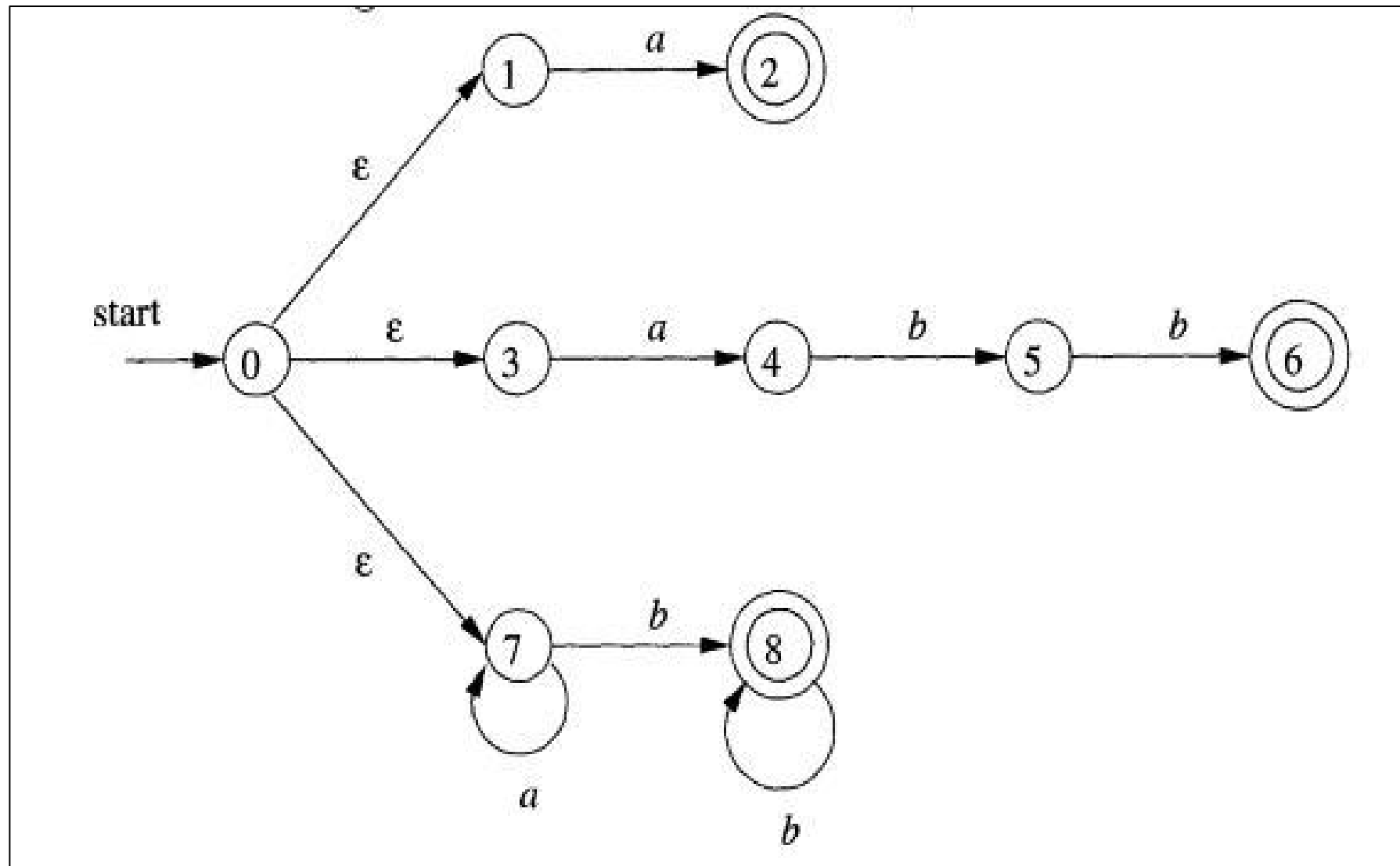
abb **A2**

a*b+ **A3**

Step 1 - Convert each lex pattern to an NFA

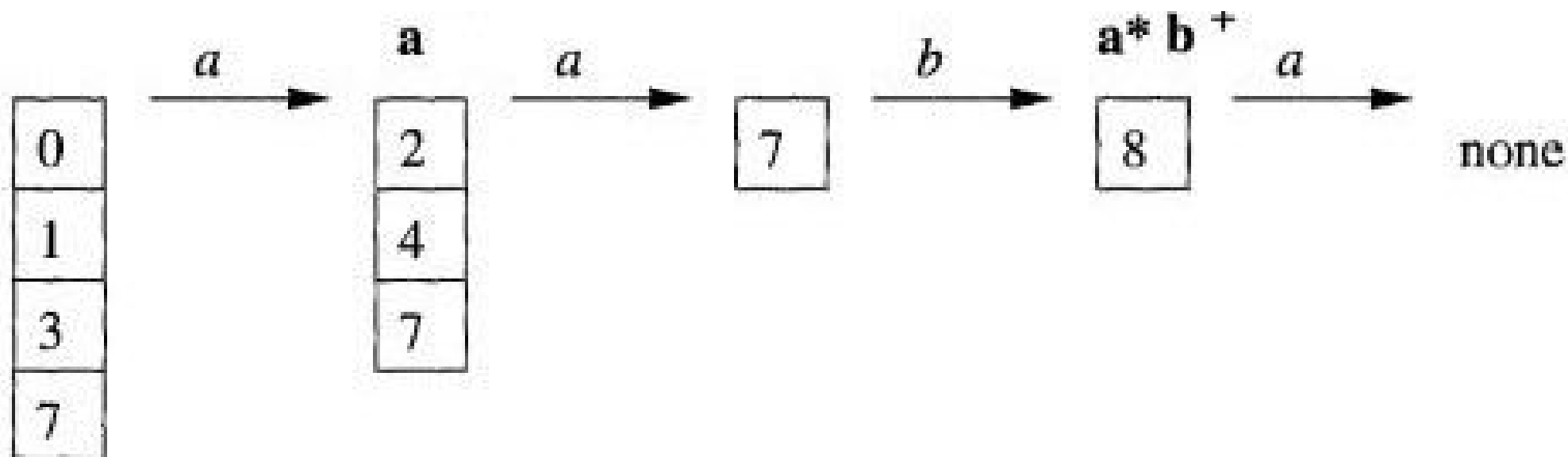


Step 2 - Construct a combined NFA by adding a new start state, and ϵ transactions to each start state (1,3,7)



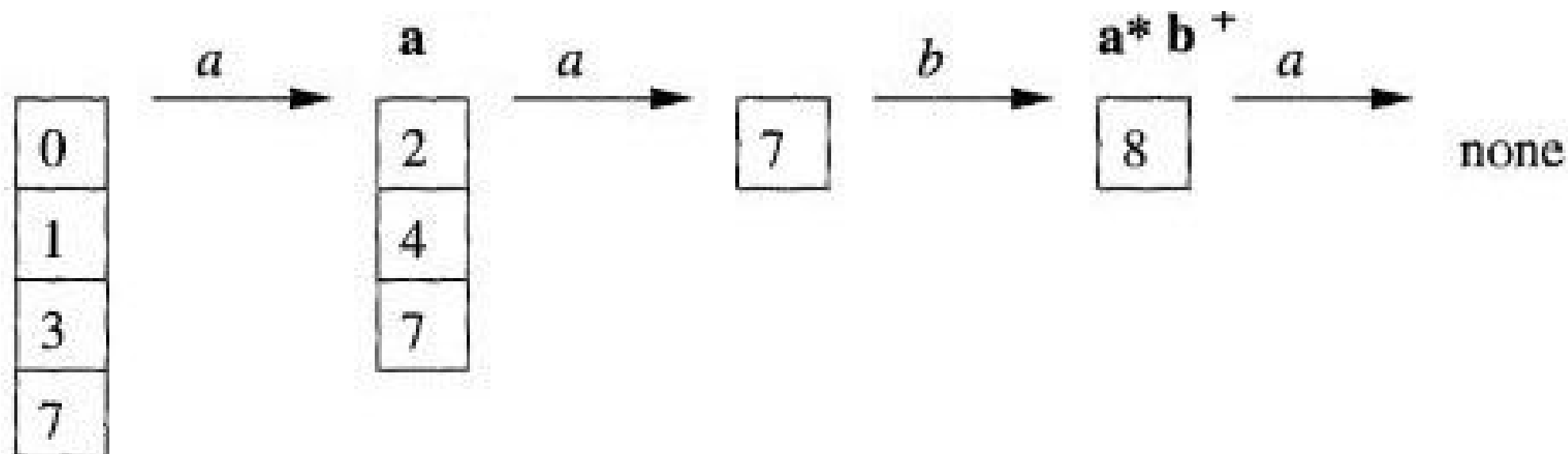
If the Lexical analyzer simulates the above NFA, pattern matching is done as follows:

- Let the input lexeme be **aaba**
- Initially, the combined NFA reads the input character pointed by **lexeme_begin**.
- The **forward** pointer moves ahead. Set of states is calculated at each point.
- When the NFA simulation reaches a point in the input where there are no next states, the set of states is empty (none).



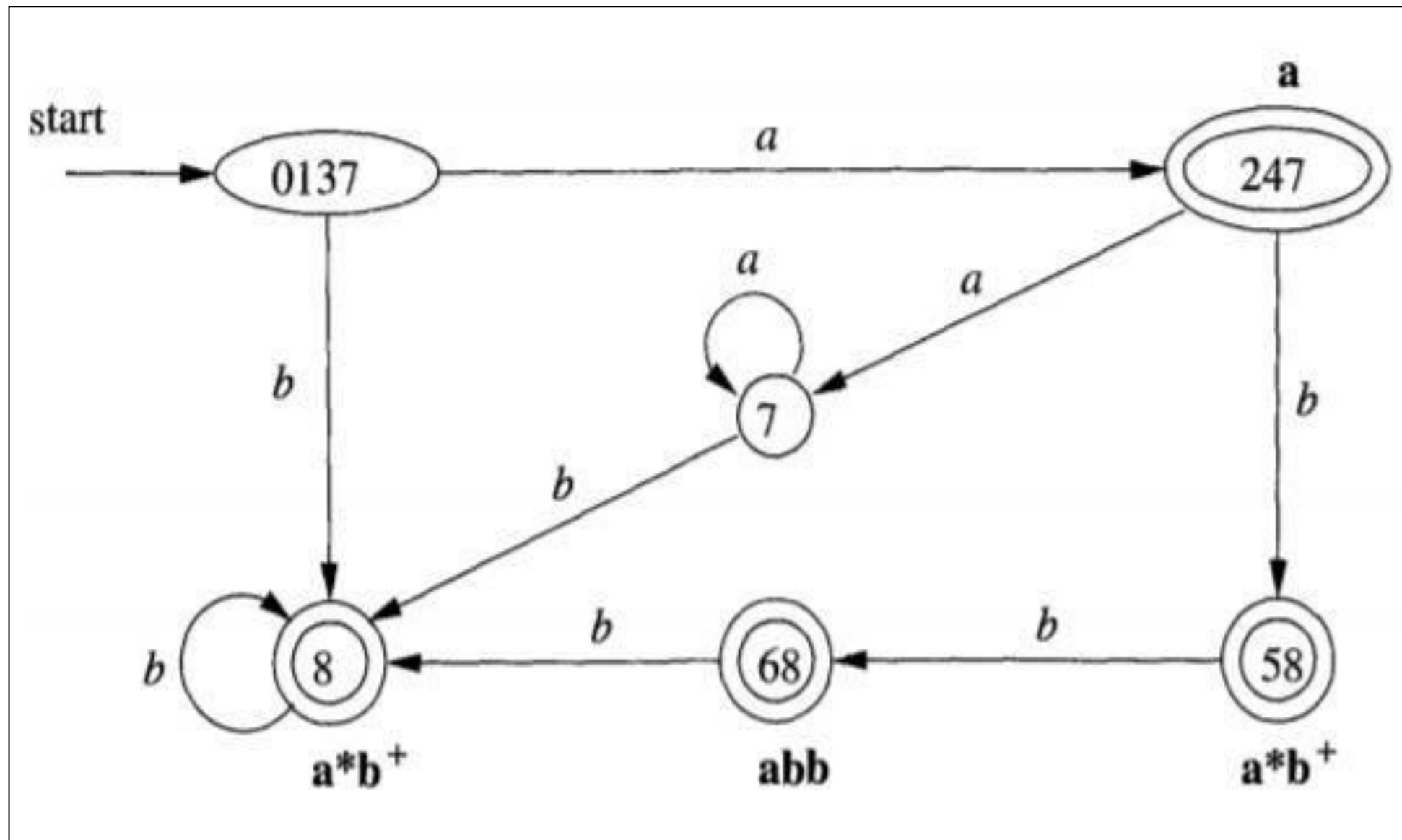
Sequence of
states entered
while processing
input aaba

- At this point, the longest prefix that matches the NFA can be identified,
- As indicated in the diagram, in state 8, **aab** matches the pattern **a*b**.
- Prefix **aab** is the longest prefix, and so it executes action **A3**.
- The forward pointer is moved to the end of the lexeme.
- Note - If there are several accepting states in the set, the action associated with the earliest pattern is executed.



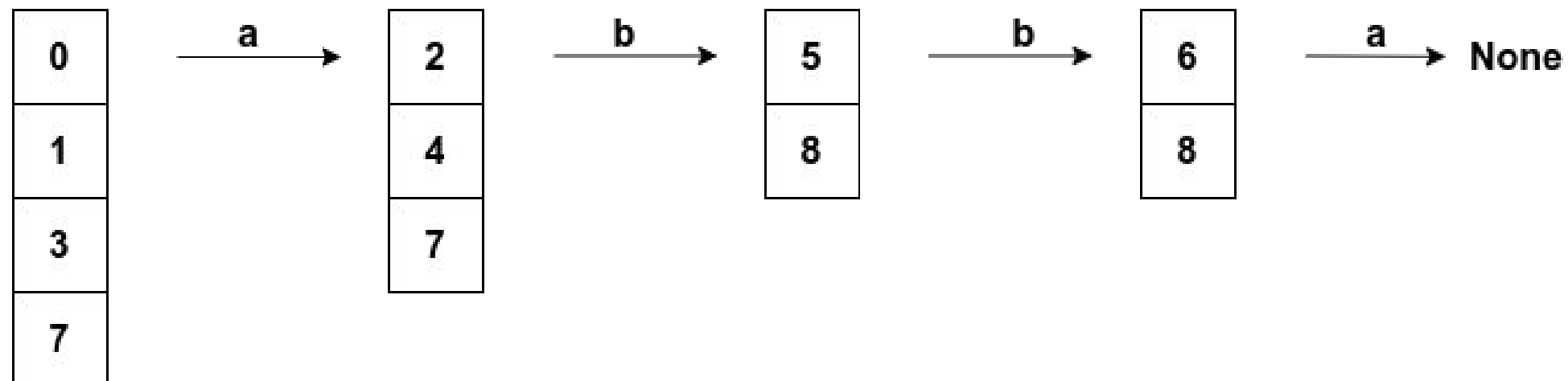
Sequence of
states entered
while processing
input aaba

Step 3 - Convert NFA to DFA



If the Lexical analyzer simulates the above DFA, pattern matching is done as follows -

- Let the input lexeme be **abba**
- Set of states is calculated at each point.
- When the DFA simulation reaches a point in the input where there are no next states, the action associated with the last accepting state is executed.
- The lexeme is **abb**, and the accepting state is 68
- This matches pattern **abb**, and action **A2** is executed



**Sequence of
states entered
while processing
input abba**

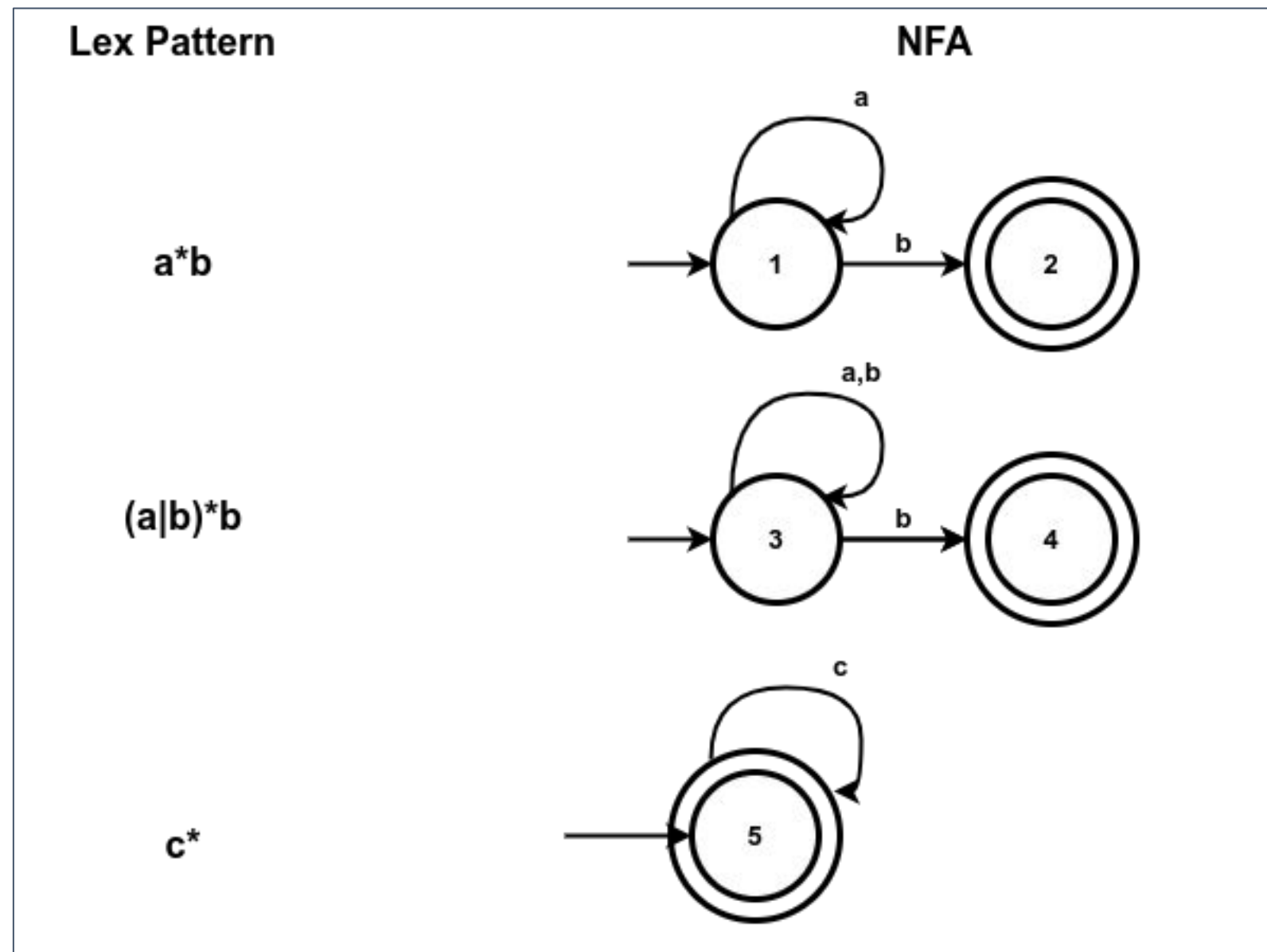
Given patterns and corresponding actions

a^*b **A1**

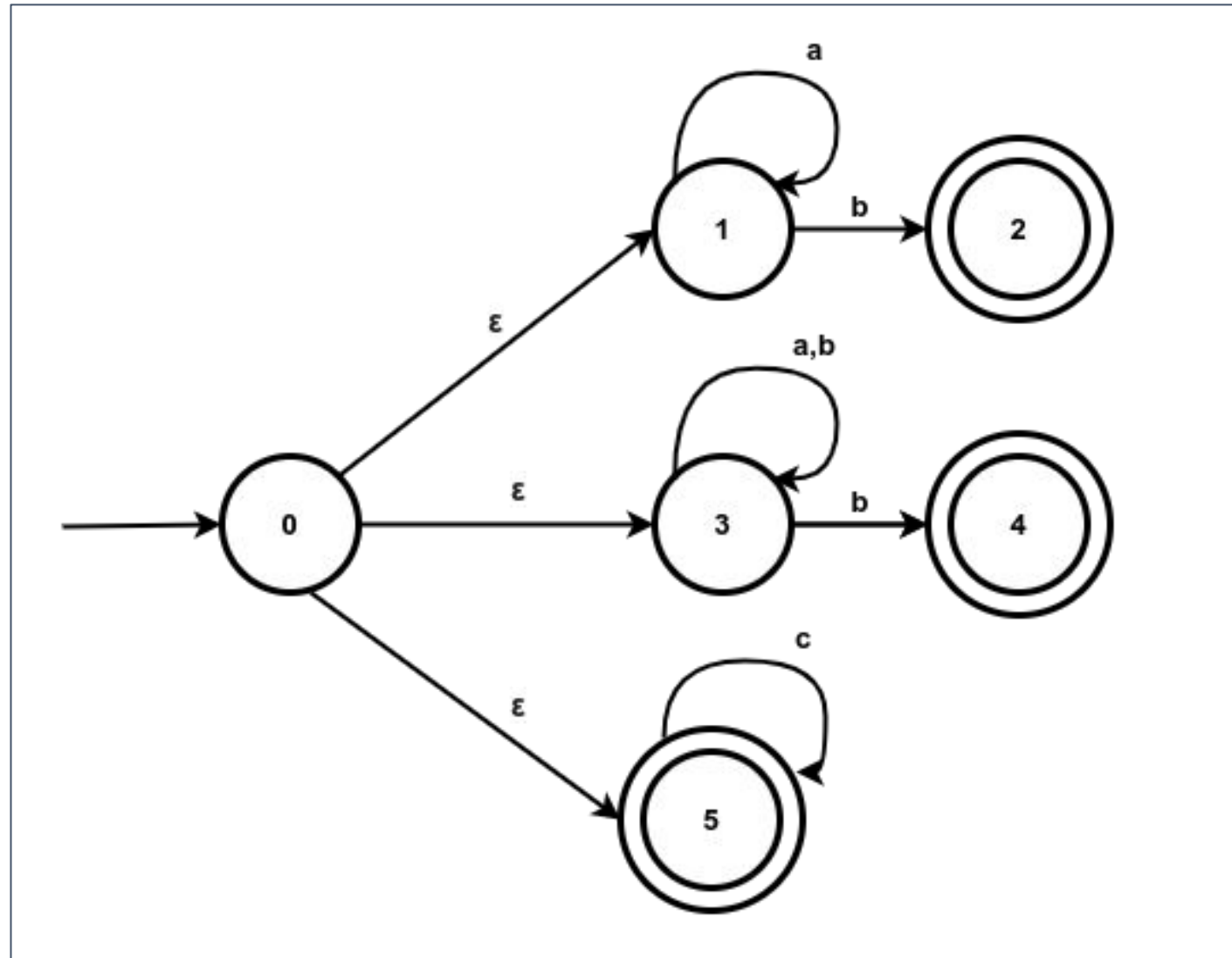
$(a|b)^*b$ **A2**

c^* **A3**

Step 1 - Convert each lex pattern to an NFA

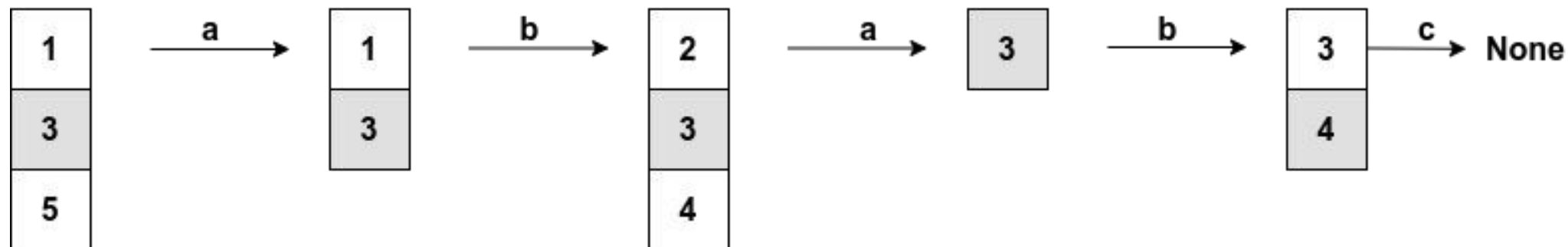


Step 2 - Construct a combined NFA by adding a new start state, and ϵ transactions to each start state (1,3,5)



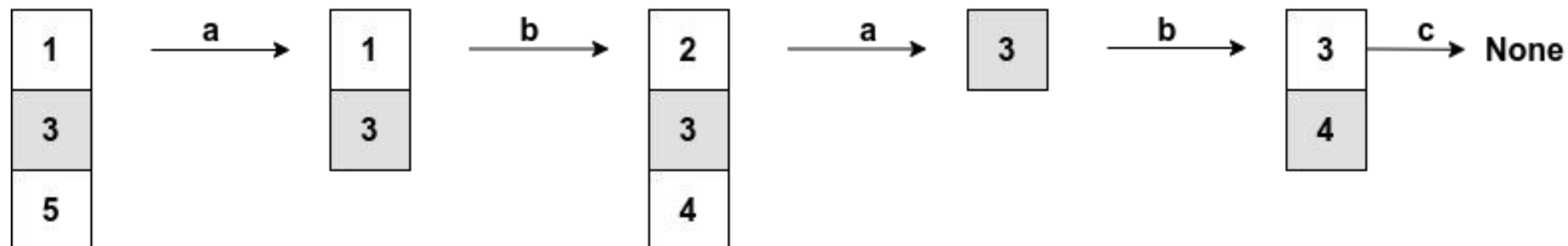
If the Lexical analyzer simulates the above NFA, pattern matching is done as follows -

- Let the input lexeme be **ababc**
- Initially, the combined NFA reads the input character pointed by **lexeme_begin**.
- The **forward** pointer moves ahead. Set of states is calculated at each point.
- When the NFA simulation reaches a point in the input where there are no next states, the set of states is empty (none).



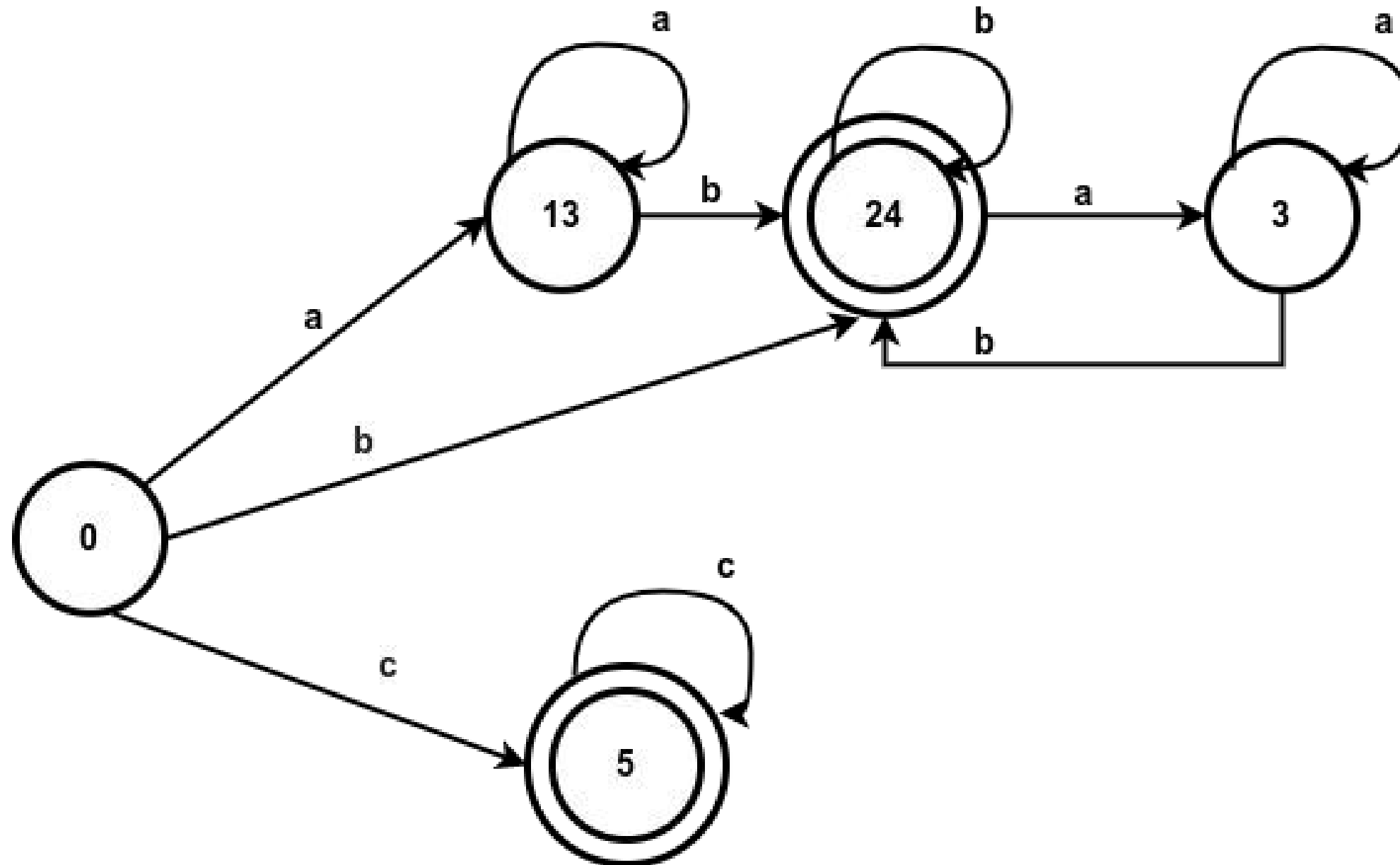
**Sequence of
states entered
while processing
input ababc**

- At this point, the longest prefix that matches the NFA can be identified,
- As indicated in the diagram, in state 8, **abab** matches the pattern **(a|b)*b**.
- Prefix **abab** is the longest prefix, and so it executes action **A2**.
- The forward pointer is moved to the end of the lexeme.
- Note - If there are several accepting states in the set, the action associated with the earliest pattern is executed.



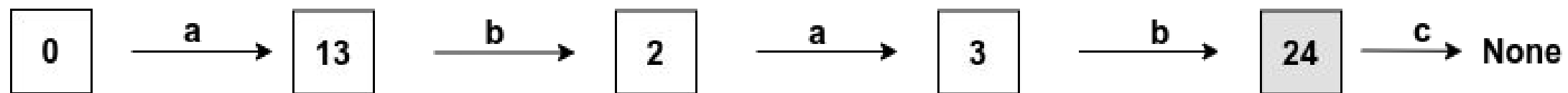
**Sequence of
states entered
while processing
input aabac**

Step 3 - Convert NFA to DFA



If the Lexical analyzer simulates the above DFA, pattern matching is done as follows -

- Let the input lexeme be **ababc**
- Set of states is calculated at each point.
- When the DFA simulation reaches a point in the input where there are no next states, the action associated with the last accepting state is executed.
- The lexeme is **abab**, and the accepting state is 24
- This matches pattern **$(a|b)^*b$** , and action **A2** is executed



Sequence of
states entered
while processing
input ababc

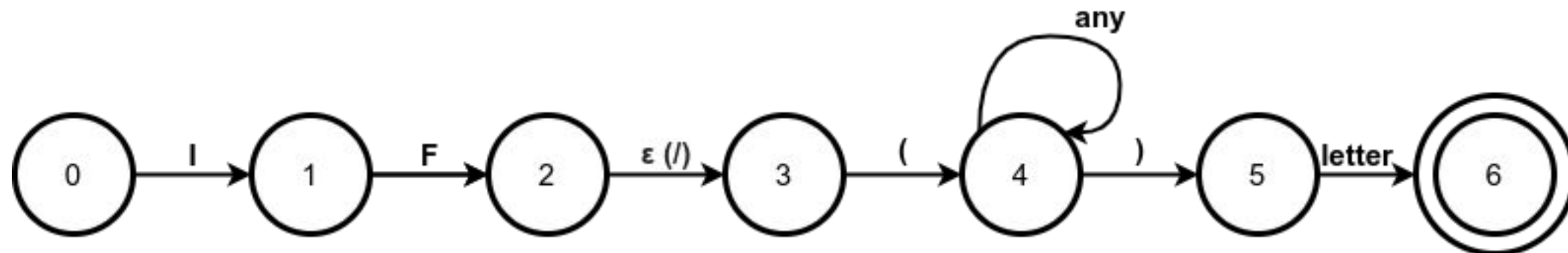
- When a certain pattern is required to be matched only when it is followed by certain other characters, the lookahead operator is / used
- $r1 / r2$
 - $r1$ - the pattern that matches the lexeme
 - $r2$ - the additional pattern that must be matched before deciding the token of the lexeme. Note - the characters that match $r2$ is NOT part of the lexeme.
- Example - FORTRAN IF statement of the form IF(condition) THEN ...
- In FORTRAN, keywords are not reserved

- IF is always followed by
 - Left parenthesis (
 - Some text
 - Right parenthesis)
 - Any letter
- The lex rule can be written as **IF/\(.*\){letter}**
- This lex code demonstrates use of / to detect a FORTRAN IF

```
letter [A-Za-z]
%{
#include<stdio.h>
%}
%%
IF/\(.*\){letter} printf("FORTRAN IF");
.;
%%
int yywrap(){return(1);}
int main()
{
    yyin = fopen("input.txt", "r");
    yylex();
    fclose(yyin);
    return 0;
}
```

Program file : lookahead.l

- When converting to NFA, the lookahead operator (/) is treated as an ϵ transaction.
- The below diagram represents NFA for FORTRAN IF using lookahead
- The ϵ transaction from state 2 to state 3 represents the lookahead operator.
- State 6 indicates presence of IF
- The lexeme IF is found by scanning backwards to the last occurrence of state 2 whenever state 6 is reached.
- Note - If the NFA has more than one ϵ (or λ) transition on / , then the problem of finding the correct state (e.g, state 2 in this example) is difficult.



```
digit  [0-9]
letter [A-Za-z]
%{
#include <stdio.h>
}%
%%
if|else|int|char|float  {printf("Keyword :\t %s",yytext);
{letter}({letter}|{digit})*  printf("Valid Identifier:\t %s",yytext);
.;
%%
int yywrap(){return(1);}
int main() {
    yyin = fopen("input.txt", "r");
    yylex();
    fclose(yyin);
}
```

Program file : keywords_and_identifiers.l

The rule
.;
is used to ignore
all other



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Gautham P Atreyas

Compiler Design

Unit 1

Specification and Recognition of Tokens

Preet Kanwal

Department of Computer Science &
Engineering

Compiler Design

Lecture Overview

In this lecture, you will learn about:

- **Revision**
 - Tokens, Lexemes and Patterns
 - Classes of tokens
 - Attributes
- **Implementation of a Lexer**
 - Handwritten
 - Using a tool
- **Handwritten Implementation**
 - Specification of Tokens
 - Regular Expressions
 - Transition Diagrams
 - Keywords vs Identifiers
 - How does the Lexical Analyzer implement all Transition Diagrams?

- **Token** : A token is a structure that holds a token name, and an optional attribute value. The token name is an abstract symbol representing a type of lexical unit (keywords, identifiers, etc). A token represents a sequence of characters that cannot be broken down further. It can be denoted as `**<token name, attribute value>**`.
- **Attribute value** is pointer to the symbol table entry.
- **Lexeme**: A lexeme is a sequence of characters in the source program which is identified by the lexical analyzer as an instance of a token.
- **Pattern**: A pattern is the description of the form that the lexemes of a token may take. It is a rule that defines the structure a token must follow.

regex101 is a very useful tool for writing and testing patterns

Token	Informal Description	Pattern	Lexemes
if	Characters `i` and `f` together as a whole word	if	if
else	Characters `e`,`l`,`s` and `e` together as a whole word	else	else
Comparison	`<` or `>` or `<=` or `>=` or `==` or `!=`	< > <= > = == !=	<=,!=
id	Letter followed by letters and digits	[a-zA-Z_]+[a-zA-Z0-9_]*	Sum, avg1, pi

- Tokens are required to classify substrings of source program according to their role.
- The parser relies on token distinctions.
- **getNextToken()** is the command sent from the Parser to the Lexical analyzer.
- On receipt of the command, the lexical analyzer scans the input until it determines the next token, and returns it.
- Input is read left to right.
- Lookahead is required to decide where one token ends.

Choosing good token classes is important, and is mostly language specific.

The most common classes of tokens are listed below:

- **Keywords** - One token for each keyword. The pattern for a keyword is the same as the keyword itself.
- **Operators** - Tokens either individually, or in classes for operators.
- **Identifiers** - One token representing all identifiers.
- **Constants** - One or more tokens representing constants, such as numbers and literal strings.
- **Punctuation Symbols** - Tokens for each punctuation symbol, such as left and right parentheses, comma and semicolon.

- When more than one lexeme can match a pattern, lexical analyzer must provide additional information about each lexeme matched.
- This additional information is stored in the attribute-value field of the token.
- The attribute-value field can include several pieces of information.
- Consider the example : **token ID** (an identifier)
 - Its attributes are the lexeme, the type and the location at which it is first found. This is added to the symbol table.
 - The attribute value is the pointer to the symbol table entry for that identifier.
- Lexical analyzer returns token name and attribute value to parser.
- The token name influences parsing decisions, whereas the attribute value influences translation of tokens after parsing.

There are two ways :

- **Hand-written Implementation** : One will write the lexer from scratch as a program. For example : The C compiler is hand-written. It is basically called loop-switch implementation.
- **Using a tool** : lex, PLY, Flex (Fast Lexical Analyzer Generator). They generally generate a lexer from a series of regular expressions.
- While tools like Flex generates lexers that are generally faster than a loop-switch, a handwritten lexer allows for better error diagnostics.

Hand-written Implementation

The specification of tokens is done using formal notations, i.e, regular expressions.

Some important definitions -

- **Symbol** : A letter, digit or punctuation
- **Alphabet** : A finite set of symbols, for e.g, {0,1} is a Binary alphabet
- **String over an alphabet** : A finite set of symbols drawn from the alphabet
- If **s** is the string, then **|s|** denotes the length of the string
- **Language** - A set of strings over a fixed alphabet
- **Abstract language** - An empty set, {ε}

- Regular expressions are special text strings that describe a search pattern.
- It is mainly used for pattern matching.
- It is a metalanguage: a form of language used for description or analysis of another language.
- It uses metacharacters: characters that have special meaning. For example,
 - ***** matches any number of characters
 - **.** matches single characters except newline
 - **** drops the special meaning of the next character
- A Regular language is a formal language that can be expressed using a regular expression.
- Regular expressions are the grammatical notations used for describing structure of tokens, i.e, **patterns**.

- For notational convenience, names can be given to regular expressions, and these names can be used to define other regular expressions.

- This is called a regular definition.

- Example 1 : Regular definition for Identifiers

letter → **[a-zA-Z_]**

digit → **[0-9]**

id → **letter(letter | digit)***

- Example 2: Regular definition for whitespace

blank → **" "**

tab → **\t**

newline → **\n**

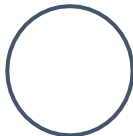
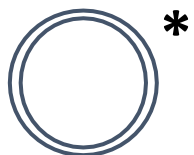
delim → **blank | tab | newline**

ws → **delim+**

Compiler Design

Transition Diagrams

- Patterns are converted to stylized flowcharts called Transition Diagrams to understand how a pattern traverses a given input.
- Transition diagrams consist of a collection of nodes called **states**.
- Each state represents a condition that summarises the characters seen.
- Two types of states:

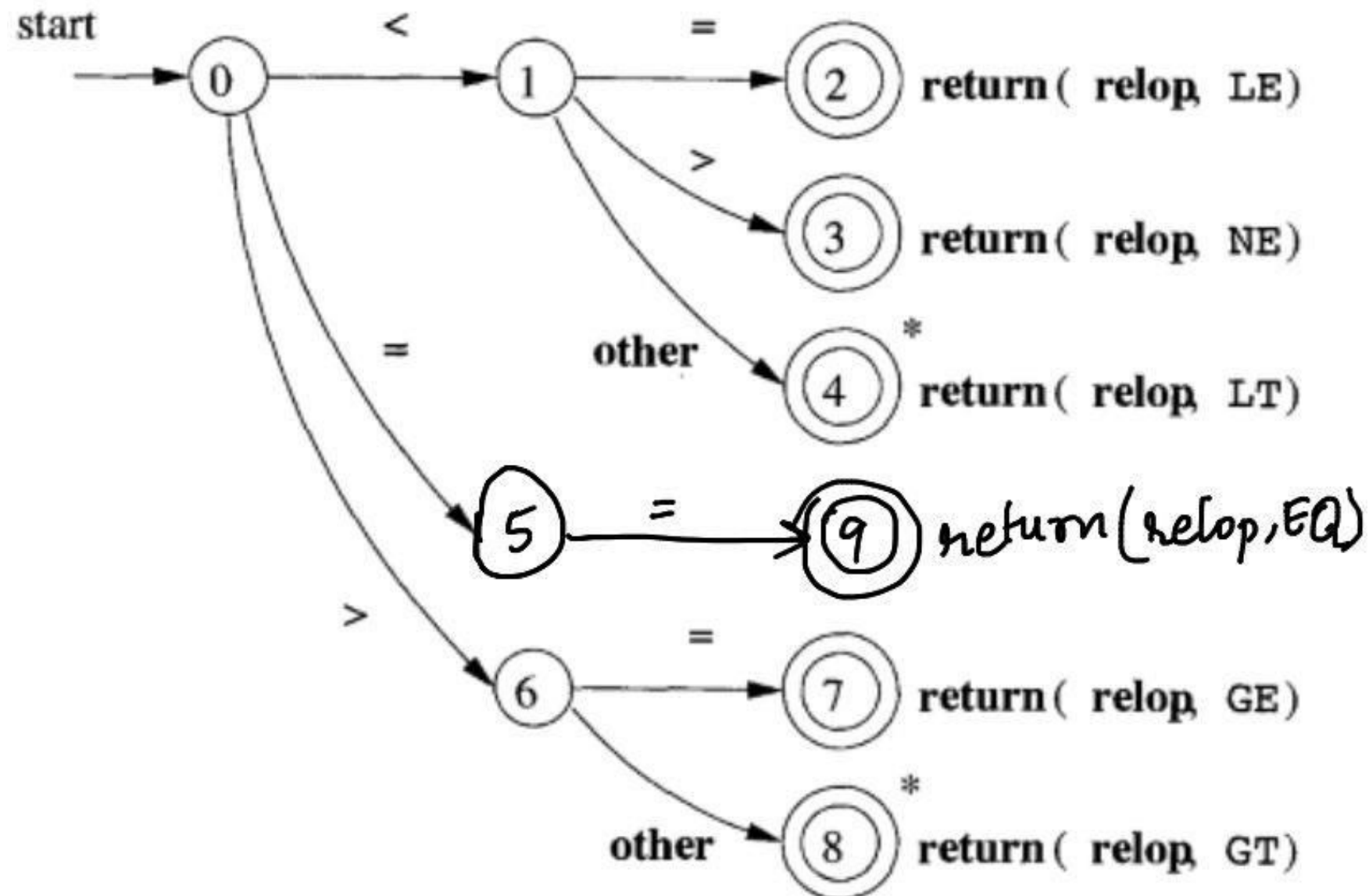
1. Final or Acception State 
 - a. Indicates that a lexeme has been found
 - b. An action is attached to this state: returning the token and attribute value to the parser
 - c. If necessary to retract the forward pointer by one position, a `\*` is placed next to the state
2. Start State  

- A variable **state** holds current state number
- A switch based on the value of **state** executes the code for that state.
- The following functions are used -
 - **getc()** or **nextChar()** - reads the next character from input.
 - **install_id()** - places the lexeme (identifier) in the symbol table if it's not present and returns a pointer to the same.
 - **install_num()** - similar to **install_id()**, enters the number in the table of numbers if not present, and returns a pointer to the same.
 - **retract()** - If the accepting state has a *****, this function is used to retract forward pointer.
 - **getToken()** - examines the symbol table entry for the lexeme found and returns the corresponding token names

- **fail()** - resets forward ptr to lexeme_begin to try another transition diagram.
 - What the **fail()** function does depends on the global error recovery strategy of the Lexical Analyzer
- **isalpha(c)** - returns true iff **c** is an alphabet
- **isdigit(c)** - returns true iff **c** is a digit
- **isalnum(c)** - returns true iff **c** is an alphabet/digit
- **isdelim(c)** - returns true iff **c** is a delimiter

Regular Expression -

relop **< | <= | > | >= | <> | ==**

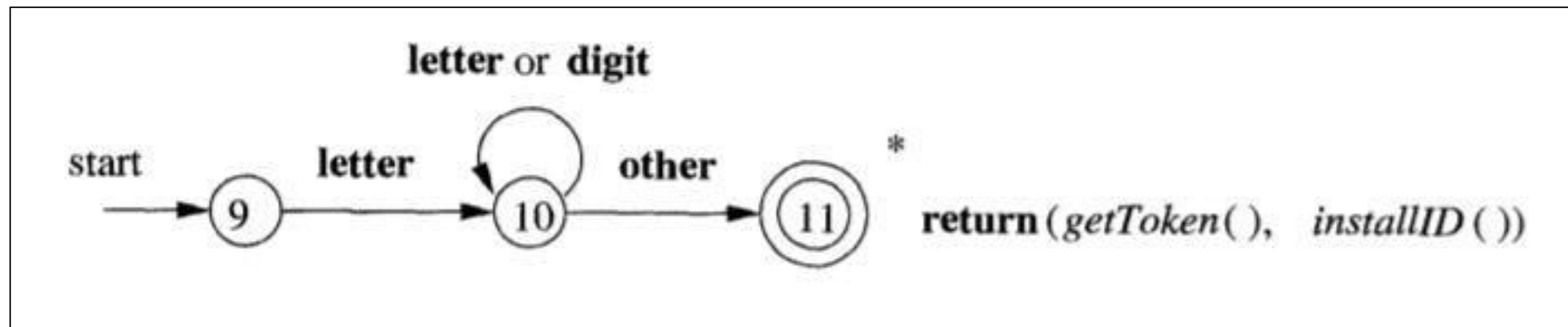


Regular Expression -

letter $\rightarrow$ [a-zA-Z]

digit $\rightarrow$ [0-9]

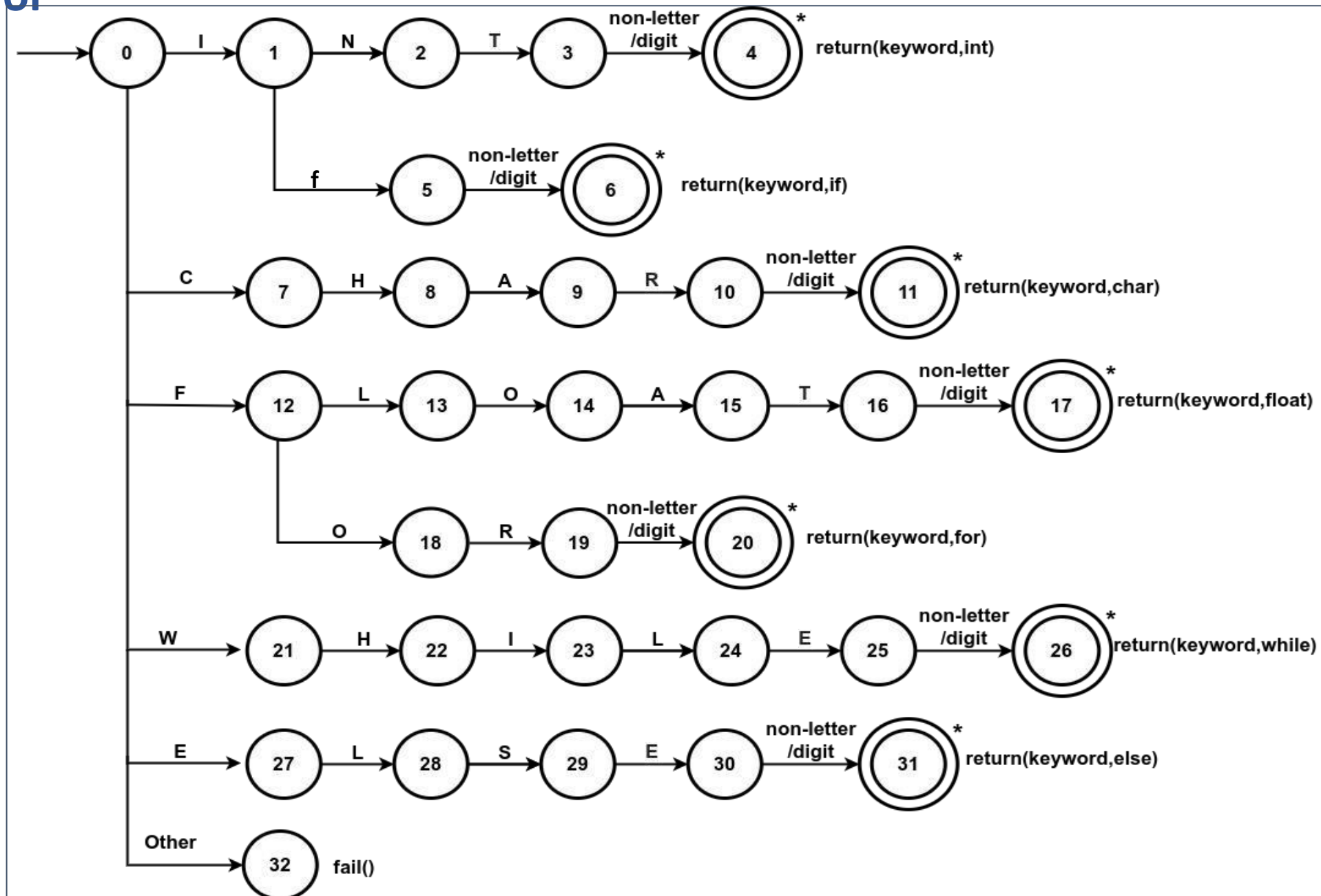
id $\rightarrow$ letter (letter | digit)*



For pseudo-code, please refer to the file:
identifier_loopswitch_pseudocode.c.txt

Note: This file is only a conversion of the transition diagram from the previous slide and is not executable without a proper interface and implementations of other functions like retract(), fail(), nextChar(), etc.

int | char | float | if | while | else |
for



non-letter/digit
simply means that
the transition
should be over a
non-letter, non-
digit character

- For pseudo-code, please refer to the file:
keyword_loopswitch_pseudocode.c

Note: This file is only a conversion of the transition diagram from the previous slide and is not executable without a proper interface and implementations of other functions like `retract()`, `fail()`, `nextChar()`, etc.

- For code that combines identification of keywords and identifiers, please refer to the file:
loopswitch.c

This code is an altered version of the code from:
<https://gist.github.com/luckyshq/6749155>

Regular Expression -

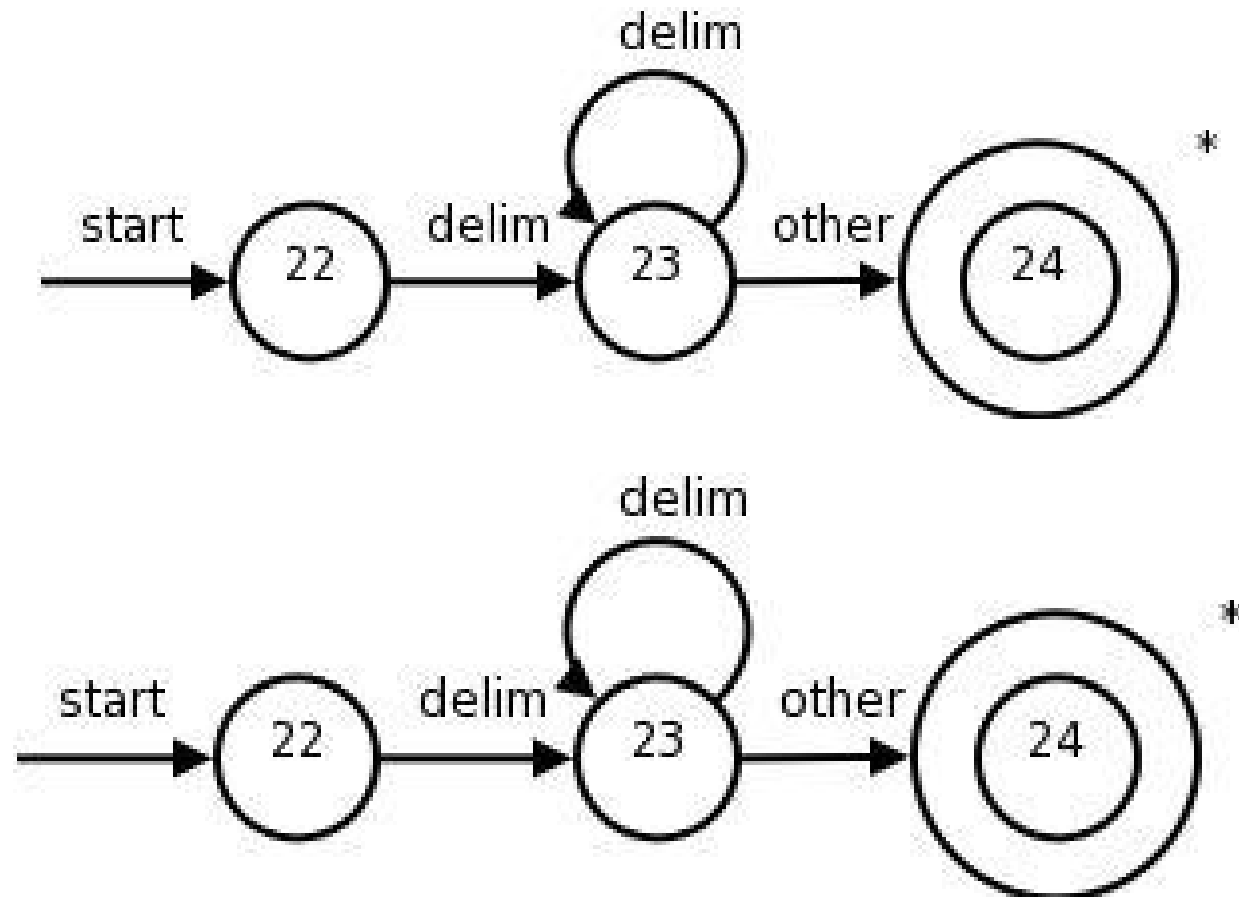
blank	→	" "
tab	→	"\t"
newline	→	"\n"
delim	→	(blank tab newline)
ws	→	delim+

Here, the input is retracted to begin at a non-whitespace

There is no return to parser.

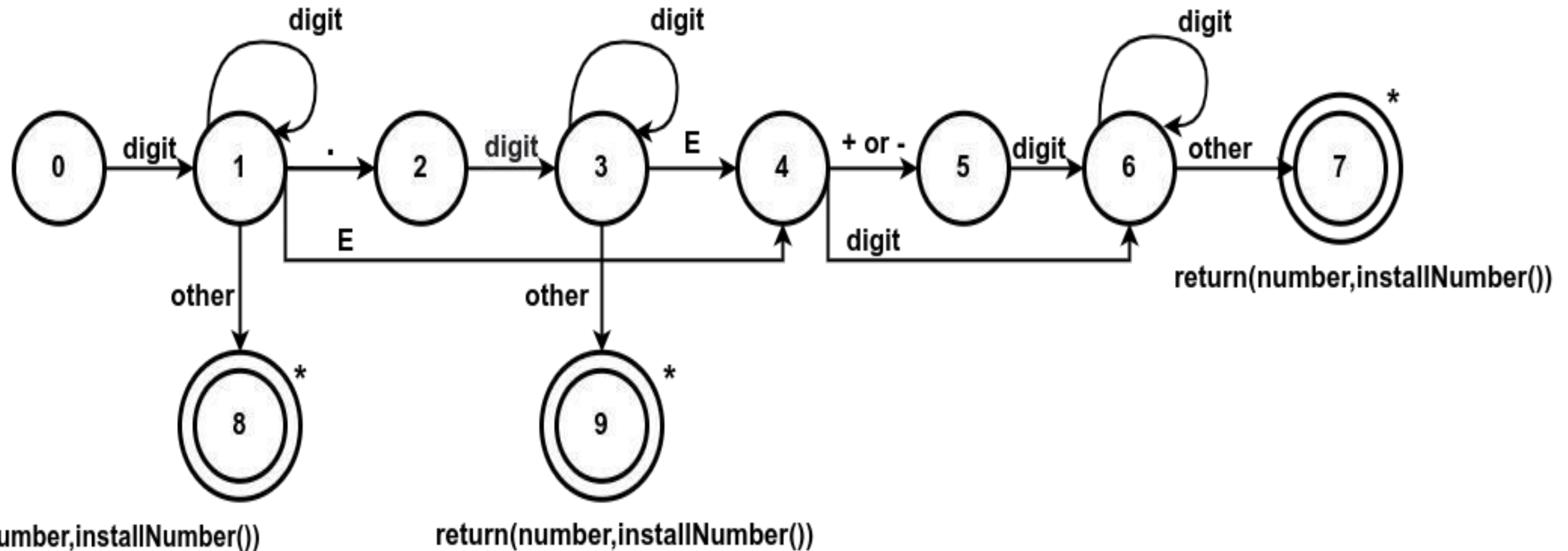
The process of lexical analysis is restarted after

whitespace



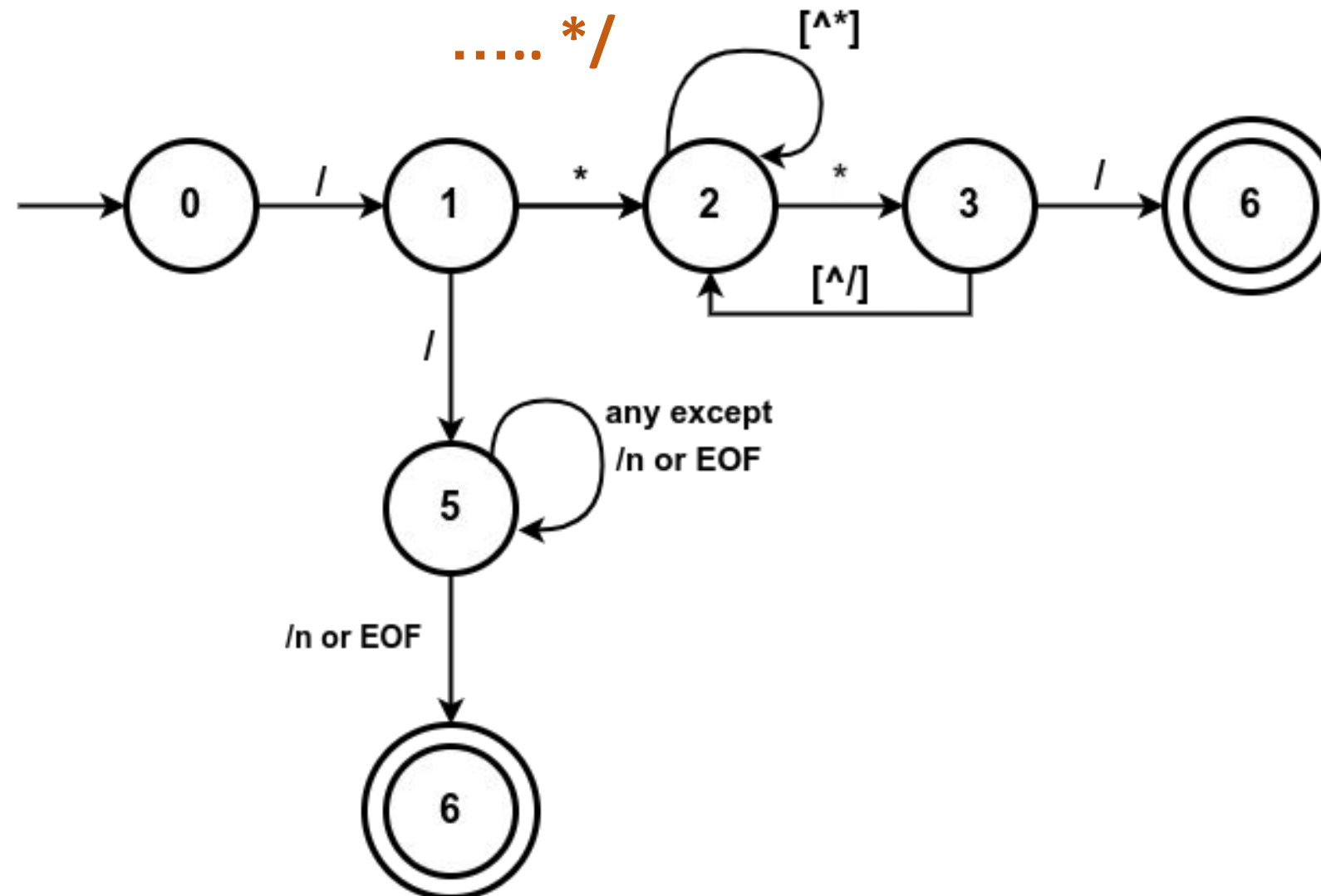
Regular Expression -

digit → [0-9]+
number → digit(.digit)?(E[+-]?digit)?



single-line comments are of the form `// ....`

multiline comments are of the form `/* .... */`



The transition diagram for identifiers will also recognise Keywords. There are two ways to handle keywords that look like identifiers:

1. Install the reserved words in the symbol table beforehand.
 - a. One field of the symbol table entry can indicate that these are reserved words (e.g. **token_name = keyword**)
 - b. Thus, when an identifier is found
 - i. **installID()** is called to place the identifier in the symbol table.
 - ii. Since keywords were preloaded in the symbol table, **installID()** returns the pointer to the keyword found.
 - c. **getToken()** can be used to return the **token_name**

2. Create a separate transition diagram for each keyword.
 - a. In this approach, each pattern for the keywords should be defined before the pattern for identifiers.
 - b. This enables reserved word tokens to be recognised when the lexeme matches both patterns.

There are three possible approaches -

- **Approach 1** - Arrange transition diagrams for each token to be tried sequentially.
- **Approach 2** - Run various transition diagrams in parallel
- **Approach 3** - Combine all transition diagrams into one.

- **Approach 1** - Arrange transition diagrams for each token to be tried sequentially.
 - In this case, each time the function fail() is called, it resets the forward pointer to lexeme_begin, and then tries the next transition diagram.
 - In this method, each keyword can have it's own transition diagram, but it must be defined before the transition diagram for identifiers.

- **Approach 2** - Run various transition diagrams in parallel
 - This involves feeding the net input character to all transition diagrams
 - If more than one transition diagram matches the lexeme, the usual strategy is to take the longest prefix.
- **Approach 3** - Combine all transition diagrams into one.
 - The transition diagram reads input until there is no possible next state.
 - The longest lexeme that matches a pattern is taken.
 - This is the preferred approach. However, the problem of combining several transition diagrams is more complex.

Question 1: Find the number of tokens

int a = 20;

Tokens		Count
int	keyword	1
a	Identifier	1
=	Operator	1
20	Constant	1
;	Special Symbol	1
Total		5

Question 2: Find the number of tokens

```
int main()  
{  
    //Compiler Design  
    int a, b;  
    a = 10;  
    b = 20;  
    return 0;  
}
```

Tokens		Count
int main	Keyword	2
()	Special symbol	2
{	Special symbol	1
//Compiler Design	Comment	0
int	Keyword	1
a	Identifier	1
,	Separator	1
b;	Identifier, symbol	2
a=10;	...	4
b=10;	...	4
return 0;	..	3
}	Special symbol	1
Total		22

E = M * C ** 2

- <id, Pointer to symbol table entry for E >
- <assign_op>
- <id , Pointer to symbol table entry for M>
- <mult-op>
- <id, Pointer to symbol table entry for C>
- <exp-op>
- <number, integer value 2>

Question 4: List all the tokens in the following code snippet

```
if(a > b)
{
    a = a + 1;
}
else
{
    b = b + 1;
}
```

Pattern	Lexemes
Keyword	if, else
Identifier	a, b
Relop	>
Assignop	=
Arithop	+
Number	1, 1
Punctuation	() { ; } { ; }

Question 4: List all the tokens in the following code snippet

```
if(a > b)
{
    a = a + 1;
}
else
{
    b = b + 1;
}
```

< keyword, if >

< punctuation, (>

<identifier, a >

< relop, > >

< identifier, b >

< punctuation,) >

< punctuation, { >

<identifier, a>

< Assignop, = >

< identifier, a>

< Arithop, + >

< number, 1 >

< punctuation, ; >

< punctuation, } >

< keyword, else >

< punctuation, { >

< identifier, b>

< Assignop, = >

< identifier, b>

< Arithop, + >

< number, 1 >

< punctuation, ; >

< punctuation, } >



**THANK
YOU**

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Sanket N

D

Compiler Design

Unit 1

Parsers and Error Recovery Strategies

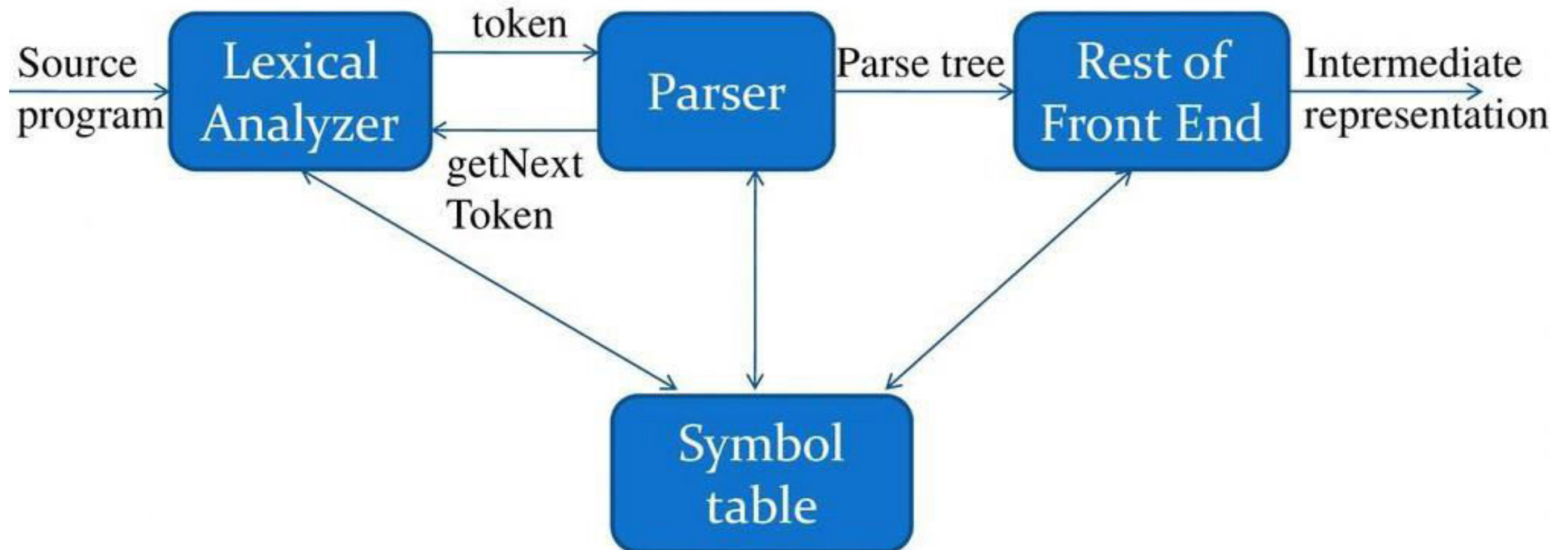
Preet Kanwal

Department of Computer Science &
Engineering

In this lecture, you will learn about -

- What are parsers?
- Role of a Parser
- Error Recovery Strategies in Parsers
- Overview of Types of Parsers
 - Top-down and Bottom-Up Parsers

- Parsing is the process of determining if a string of tokens can be generated by the grammar or not.
- Parser, also called a Syntax Analyzer, is a program which takes as input a grammar **G** and a string **w** to produce a parse tree for **w**, if the string **w** belongs to the language of grammar **G** else throws an error.
- Note - Input is read from left to right.



The Role of a parser is-

- To validate the syntax of the program.
- To construct a parse tree and pass it to the semantic analyzer in the compiler.
- To store information in the symbol table about tokens received from the lexer.
- Report syntax errors.

The compiler must report errors by generating messages of the following format -

- The message must pinpoint the error - for example, the line number of the statement containing the error.
- The message should be specific and must localise the problem.
 - Eg: Undeclared variable in function foo() at line 29.
- The message should be clear and understandable.
- Messages should not be redundant. For example, if an undeclared variable is referenced 20 times, the error should be displayed only once.

Compiler Design

Error Recovery

Strategies



Error Recovery Strategies

- There is no universally accepted strategy for error recovery in Parsers.
- Some simple strategies would be to quit with an informative error message (eg. python), or gather additional errors before quitting (eg. C), presuming it is possible to restore the parser state where it can continue processing.
- There are four commonly used error recovery strategies -
 1. Panic Mode Recovery
 2. Phrase level Recovery
 3. Error Productions
 4. Global Corrections

1. Panic Mode Recovery

- The parser will discard the input symbols until it finds a delimiter like ; or) or } etc, and then restart the parsing process.
- Such delimiters are called **synchronizing tokens**.
- Consider the following expression
 - **(1 + + 2) + 3**

In this case, we skip the input till) and then continue parsing.

2. Phrase level

recovery

- This involves performing local corrections in the program, in case it contains errors. input
- For example,
 - missing semicolon ; - insert it
 - missing closing bracket) - insert it
- The choice of the correction is left to the compiler designer. However, it needs to be done carefully.
 - Incorrectly inserting characters could cause the program to perform erroneously.

3. Error productions

- The idea here is to specify in the grammar known common mistakes.

For example,

- $S \rightarrow \text{if (cond) \{S\} else \{S\}}$

is grammar for valid if-else code.

- $S \rightarrow \text{fi (cond) \{S\} else \{S\}}$

can be written as an error production, with action being a more informative message about the error to the user.

- Disadvantage - It complicates the grammar

4. Global Correction

- Extremely hard to implement
- Takes the grammar and the input program (with errors) as input, and passes them to the algorithm to find a program y that is closest correct program to the original program x .
- The algorithm must have made minimal changes to x .
- Disadvantages
 - The closest correct program y may not exactly be what the programmer intended to write.

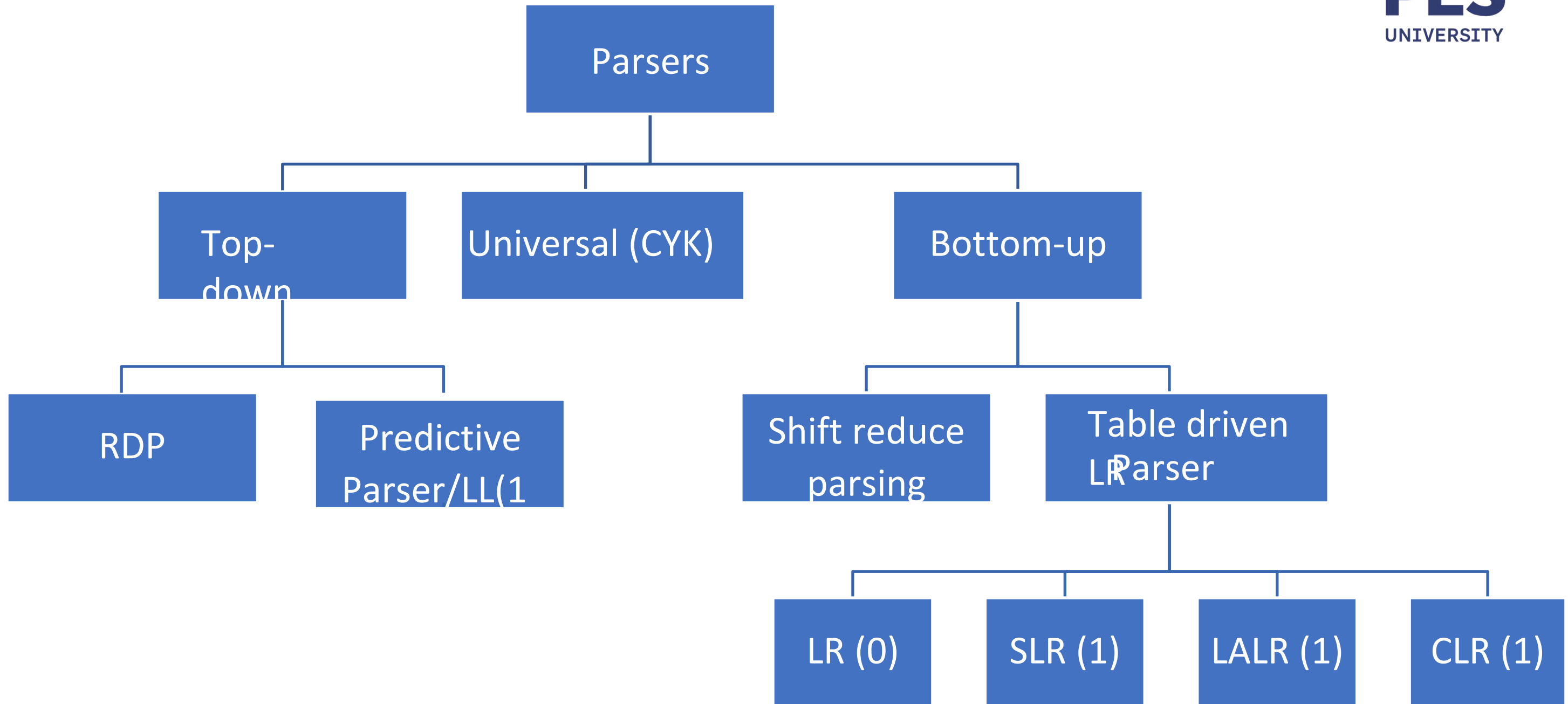
Compiler Design

Error Recovery

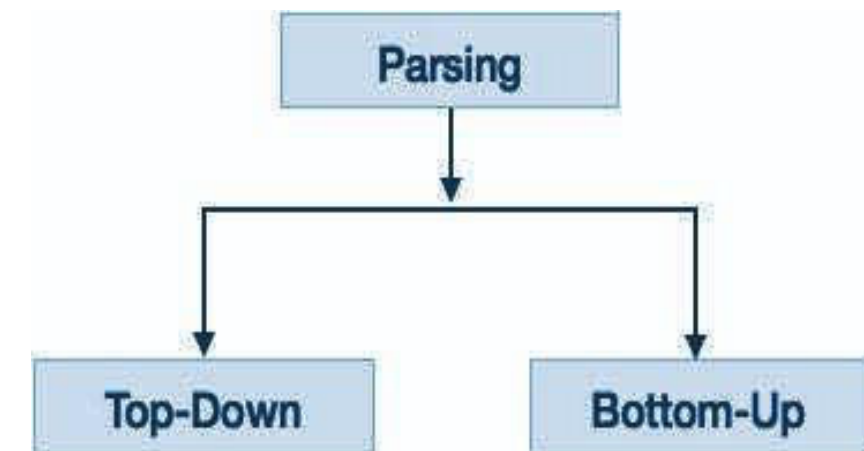
Strategies



Types of Parsers



- There are mainly two types of parsers - **top-down** parsers and **bottom-up** parsers.
- A **Top-down** Parser generates parse tree for the given input string with the help of grammar productions by expanding the non-terminals.
 - It starts from the start symbol and ends on the terminals.
 - It uses left most derivation.
- A **Bottom-up** Parser generates the parse tree for the given input string with the help of grammar productions by compressing the non-terminals
 - It starts from non-terminals and ends on the start symbol.
 - It uses the reverse of the rightmost derivation.





**THANK
YOU**

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Kavya P

K

Compiler Design

Unit 2

Recursive Descent Parsing without Backtracking

Preet Kanwal

Department of Computer Science &
Engineering

- Top down parsing involves constructing a parse tree from the start symbol and attempting to transform the start symbol to the input.
- Top down parsing is based on **Left Most Derivation** of input string.
- There are two ways to implement a top-down parser
 - RDP implementation
 - Table driven/Predictive parser implementation

- A few modifications are applied to grammars to make it compatible for TDP.
- This involves two steps -
 - **Left factoring** of the grammar
 - **Eliminating Left recursion**
- The two operations need not be done in any particular sequence.
- Left factoring can be done before or after eliminating left recursion.
- Eliminating left recursion once is sufficient.

1. Left factoring of grammar

- Factor out the common prefix present in the grammar, i.e, scan the common part once.
- The following example shows a grammar before and after left factoring is done.

$$S \rightarrow \underset{a}{i C t S e s} \quad | \quad i C t s \quad |$$
$$C \rightarrow b$$

On left factoring

-
$$S \rightarrow \underset{a}{i C t S X} \quad |$$

$$C \rightarrow b$$
$$X \rightarrow e S \quad | \quad \lambda$$

2. Elimination of Left Recursion

A left recursive grammar is typically of the form

$A \rightarrow A\alpha$, where $\alpha \in (V \cup T)^*$

[V = variable, T = terminal]

- Left recursion is eliminated to prevent top down parsers from entering into an infinite loop.
- Left recursion can be
 - Immediate - For example - $A \rightarrow A | b$
 - Indirect - Recursion occurs in two or more steps Example -

$S \rightarrow A a b$

$A \rightarrow S d \mid$

λ

In case of indirect left recursion, whenever there are multiple non-terminals in the grammar, specify the order of nonterminals, which is usually the order in which they are presented.

- This will convert indirect left recursion to immediate left recursion.
- Example -

Given,

$$S \rightarrow A a b$$
$$A \rightarrow S d \mid$$

- Here, S is replaced by $A a b$ in the second production.

$$S \rightarrow A a b$$
$$A \rightarrow A a b d \mid$$
$$\lambda$$

Algorithm to eliminate left recursion -

- Input: Grammar with no cycles or λ productions
- Output: Grammar without Left Recursion
 1. Arrange the non-terminals in some order
(usually the order in which they are presented)
 2. Replace each production of the form

$$A \rightarrow A \alpha_1 \mid A \alpha_2 \mid \dots \mid A \alpha_s \mid \beta_1 \mid \beta_2$$

with

$$\begin{aligned} A &\rightarrow \beta_1 A' \mid \beta_2 A' \\ A' &\rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_s A' \mid \lambda \end{aligned}$$

Consider the following grammar

-

$A \rightarrow Bxy \mid x$

$B \rightarrow CD$

$C \rightarrow A \mid c$

$D \rightarrow d$

Eliminate left
recursion.



Given -

$$\begin{aligned} A &\rightarrow Bxy \mid x \\ B &\rightarrow CD \\ C &\rightarrow A \mid c \\ D &\rightarrow d \end{aligned}$$

Step 1 - Replace indirect left recursion

$$\begin{aligned} A &\rightarrow Bxy \mid x \\ B &\rightarrow CD \\ C &\rightarrow Bxy \mid x \quad \Rightarrow \quad C \rightarrow CDxy \mid \\ &\quad \mid c \quad \quad \quad c \quad \mid x \\ D &\rightarrow d \end{aligned}$$

$$\begin{aligned} A &\rightarrow B x y \mid x \\ B &\rightarrow C D \\ C &\rightarrow C D x y \mid c \\ &\quad \mid x \\ D &\rightarrow d \end{aligned}$$

Step 2 - Eliminate immediate Left recursion

$$\begin{aligned} A &\rightarrow B x y \mid x \\ B &\rightarrow C D \\ C &\rightarrow x C' \mid c C' \\ C' &\rightarrow D x y C' \mid \lambda \\ D &\rightarrow d \end{aligned}$$

Consider the following grammar

- $S \rightarrow a A c B$
 $A \rightarrow A b \mid b \mid c$
 $B \rightarrow d$

Eliminate left
recursion.

Consider the following grammar

- $S \rightarrow a A c B$
 $A \rightarrow A b \mid b \mid b c$
 $B \rightarrow d$

Answer -

$S \rightarrow a A c B$
 $A \rightarrow b A' \mid b c A'$
 $A' \rightarrow b A' \mid \lambda$

Eliminate left recursion in the following grammar -

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow id \mid num \\ &\quad \mid (E) \end{aligned}$$

Given -

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow id \mid num \mid (E) \end{aligned}$$

Eliminate immediate left recursions for E and T productions using the algorithm.

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \mid \lambda \\ T &\rightarrow F T' \\ T' &\rightarrow * F T' \mid \lambda \\ F &\rightarrow id \mid num \mid (E) \end{aligned}$$

- It is a top-down parser that implements a set of recursive procedures to process the input without backtracking.
- Recursive Procedures are written for each non-terminal in the grammar.
- The recursive procedures can be accessible to write and adequately effective if written in a language that performs the procedure call effectively.

Consider the earlier grammar for parsing arithmetic expressions -

$$E \rightarrow T E'$$
$$E' \rightarrow + T E' \mid \lambda$$
$$T \rightarrow F T'$$
$$T' \rightarrow * F T' \mid \lambda$$
$$F \rightarrow \text{id} \mid \text{num} \mid (E)$$

Notice that the grammar is left factored, and left-recursion is removed.

The RDP implementation for this grammar is present at - [RDPwithoutBacktracking.c](#)

Consider the following grammar

-

$$\begin{aligned} S &\rightarrow i C t S X \mid b \mid a \\ C &\rightarrow c \end{aligned}$$

Notice that the grammar is left factored, and left-recursion is removed.



**THANK
YOU**

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Kavya
P K

Compiler Design

Unit 1

Predictive Parsers (LL(1))

Preet Kanwal

Department of Computer Science &
Engineering

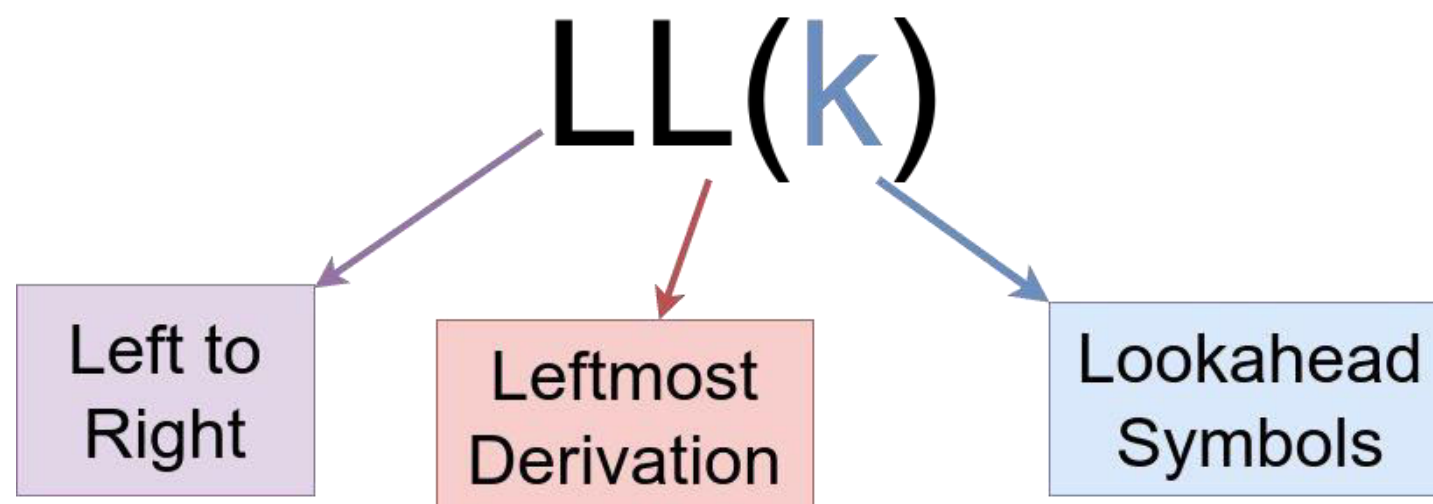
In this lecture, you will learn about -

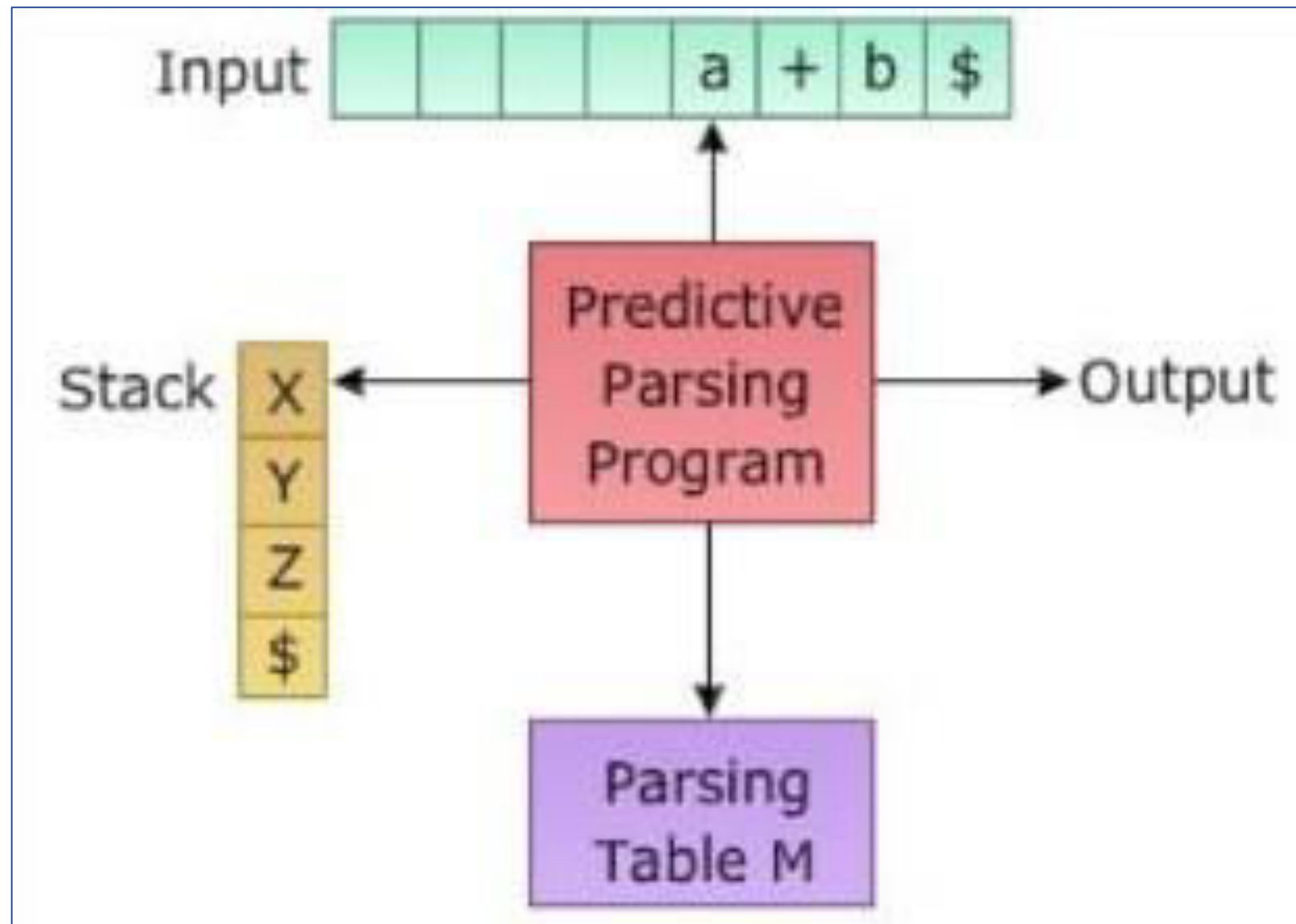
- What are Predictive Parsers?
- Model of a Table-driven Parser
- Computing First Sets
- Computing Follow Sets
- What is an LL(1) Parsing Table?
- Construction of an LL(1) Parsing Table

What are Predictive Parsers?

- Predictive Parser is a table driven top-down parser without backtracking.
- Class of grammars for which a table driven parser can be constructed is **LL(k)**.
- A predictive parser uses a lookahead pointer which points to the next input symbols.
- It uses a stack and a parsing table to parse the input and produce a parse tree.
- Both the stack and the input contains an end symbol **\$** to denote that the stack is empty and the input is consumed.

- Table driven parser can be constructed is for a class of grammars $LL(k)$ where
 - The first **L** indicates that input is parsed from left to right.
 - The second **L** indicates that the output of the parser, which is a parse tree, is the leftmost derivation of the input string.
 - **k** denotes the number of symbols used for lookahead.





- The **First()** of a non-terminal is the set of symbols that appear as the first symbol in a string that derives from the non-terminal.
- Rules to compute First set:
 1. **First(t) = { t }**, where **t** is a terminal
 2. If **X → λ**, then add **λ** to **First(X)**
 3. If **X → Y₁Y₂Y₃**
 - **First(X) = First(Y₁)**
 - If **λ ∈ First(Y₁)**, then **First(X) = { First(Y₁) - λ } ∪ { First(Y₂) }**
 - If **λ ∈ First(Y_i)** for all **1 ≤ i ≤ n**, then add **λ** to **First(X)**

Computing First set - Example 1

- Consider the following grammar:

$$E \rightarrow T E'$$
$$E' \rightarrow + T E' \mid \lambda$$
$$T \rightarrow F T'$$
$$T \rightarrow * F T' \mid \lambda$$
$$F \rightarrow \begin{array}{l} \text{id} \quad \mid \quad \text{num} \\ \quad \quad \mid \quad (E) \end{array}$$

Find the first set of non terminal

T

Computing First set - Example 1

- Production of T: $T \rightarrow FT'$
- According to Rule 3, $\text{First}(T) = \{ \text{First}(F) \}$
- Consider all productions of

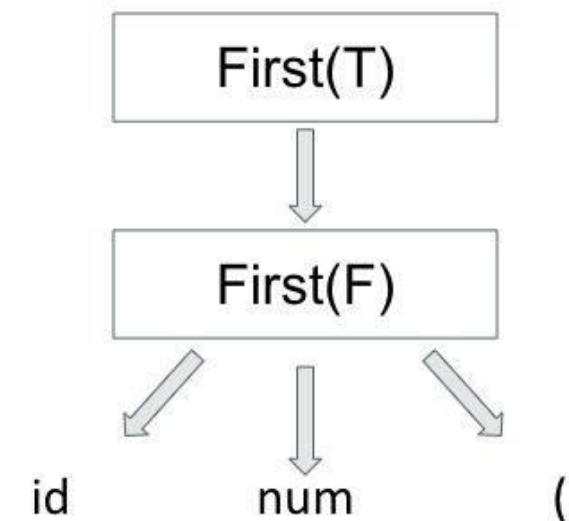
$F \rightarrow \text{id}$

$F \rightarrow \text{num}$

$F \rightarrow ($

- According to Rule 1, $\text{First}(F) = \{ \text{id}, \text{num}, (\}$
- Since F does not have any λ productions, we stop here.
Since $\text{First}(T) = \text{First}(F)$,

$\text{First}(T) = \{ \text{id}, \text{num}, (\}$

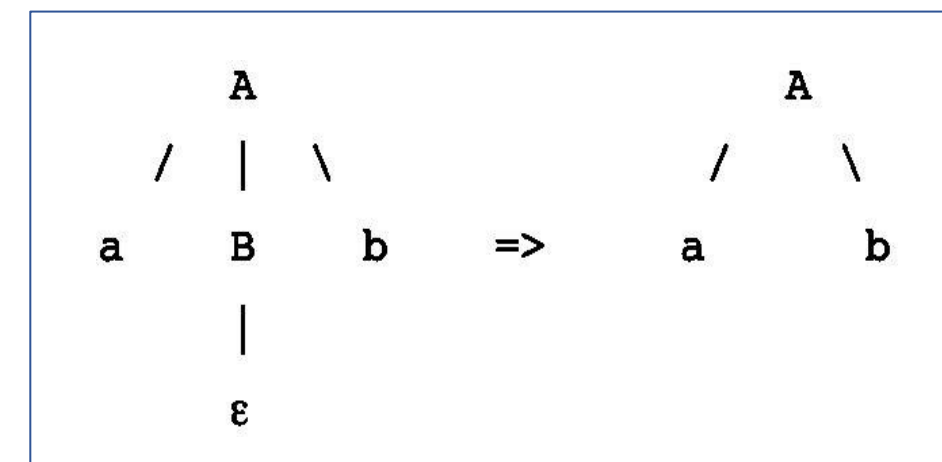
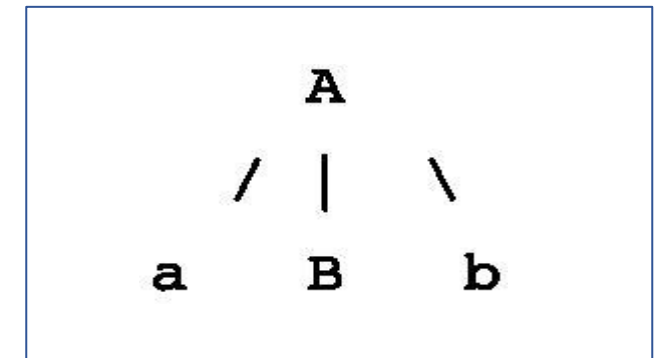


- Consider the partial parse tree on the right. Assume the following grammar:

$A \rightarrow aBb$

$B \rightarrow c \mid \lambda$

- When the input symbol is **b**, the non-terminal to derive is **B**. There is no production of **B** that starts with **b**, so the parser is now stuck.
- However, if the lambda production of **B**, $B \rightarrow \lambda$, is applied, **B** vanishes and the parser parses **ab** correctly.
- A non-terminal should vanish if the symbol that **follows** the non-terminal in the input is the same as the symbol that follows it in the production rule/in some



- Thus, when there are nullable non-terminals in a grammar, there is a need to calculate **follow sets** of each of the nullable non-terminal so that their lambda productions can be used using parsing.
- Follow(X) to be the set of terminals that can appear immediately to the right of Non-Terminal X in some sentential form.

Compiler Design

Computing Follow Sets



- Rules to compute Follow set:
 1. **Follow(S) = { \$ }**, where **S** is the start symbol
 2. If **A → αBβ**, where **λ ∈ First(β)**,
then **Follow(B) = { First(β) - λ } ∪ { Follow(A) }**
 3. If **A → αB**, **Follow(B) = Follow(A)**

where **α, β ∈ (V ∪ T)***

V = Variables (non

Follow Sets - Example

1

- Consider the following grammar:

$$E \rightarrow T E'$$
$$E' \rightarrow + T E' \mid \lambda$$
$$T \rightarrow F T'$$
$$T' \rightarrow * F T' \mid \lambda$$
$$F \rightarrow \text{num} \mid \text{id}$$

- Compute the Follow sets of all non-terminals in the grammar for each production.

Follow Sets - Example 1 - Solution

$E \rightarrow TE'$	$\text{Follow}(E) = \$$ $\text{Follow}(T) = \{ + \} \cup \text{Follow}(E)$ $\text{Follow}(E') = \text{Follow}(E)$
$E' \rightarrow + T$ E'	$\text{Follow}(T) = \{ + \} \cup \text{Follow}(E')$ $\text{Follow}(E') = \text{Follow}(E')$
$T \rightarrow FT'$	$\text{Follow}(F) = \{ * \} \cup \text{Follow}(T)$ $\text{Follow}(T') = \text{Follow}(T)$
$T' \rightarrow * F$ T'	$\text{Follow}(F) = \{ * \} \cup \text{Follow}(T')$ $\text{Follow}(T') = \text{Follow}(T')$
$F \rightarrow (E)$	$\text{Follow}(E) = \{) \}$

Steps to construct LL(1) Parsing Table

- Eliminate Left recursion (if present) and Left factor the grammar(if required).
- Calculate **First()** and **Follow()** sets for all non-terminals.
- Draw a table where the first row contains the Non-Terminals and the first column contains the Terminal Symbols.
- All the Null Productions of the Grammars will go under the elements of **Follow(LHS)**.
 - Reason - The null production must be used only when the parser knows that the character that follows the LHS in the production rule is same as the current input character.

Steps to construct LL(1) Parsing Table

For each production $A \rightarrow \alpha$

1. For each terminal in $\text{First}(\alpha)$, make entry $A \rightarrow \alpha$ in the table.
2. If $\text{First}(\alpha)$ contains λ , then for each terminal in $\text{Follow}(A)$ (i.e, $\text{Follow}(\text{LHS})$) make an entry $A \rightarrow \alpha$ in the table.
3. If the $\text{First}(\alpha)$ contains λ and $\text{Follow}(A)$ contains $\$$ as a terminal, then make an entry $A \rightarrow \alpha$ in the table under $\$$.

Constructing LL(1) Parsing Table - Example

1

Construct the parsing table for the earlier Grammar.

E	→	T E'			
E'	→	+ T E'		λ	
T	→	F T'			
T'	→	* F T'		λ	
F	→	num		id	(E)

Constructing LL(1) Parsing Table - Example 1

From the earlier examples, the first and follow tables for the given grammar is as follows -

First Table				
E	E'	T	T'	F
id	+	id	*	id
num	λ	num	λ	num
(		(		(

Follow Table				
E	E'	T	T'	F
\$	\$	+	+	*
)	)	\$	\$	+
		)	)	\$
				)

Constructing LL(1) Parsing Table - Example 1

Construct a table with terminals on the first row and Non-terminals on the first column.

	id	+	*	num	(	)	\$
E							
E'							
T							
T'							
F							

Constructing LL(1) Parsing Table -
Example 1

For each production in the grammar, entries to the
add appropriate locations.

	id	+	*	num	(	)	\$
E	E -> TE'			E -> TE'	E -> TE'		
E'							
T							
T'							
F							

E → TE'

Add the E → TE' under each element in First(T)

First(T) = {id, num and (}

Constructing LL(1) Parsing Table - Example 1

For each production in the grammar, add entries to the appropriate locations.

	id	+	*	num	(	)	\$
E	E -> TE'			E -> TE'	E -> TE'		
E'		E' -> +TE'				E' -> λ	E' -> λ
T							
T'							
F							

$E' \rightarrow +$
TE' Add $E' \rightarrow +TE'$ under +

$E' \rightarrow \lambda$
So, Add $E' \rightarrow \lambda$ under all elements in Follow(LHS), i.e, {), \$ }

Constructing LL(1) Parsing Table - Example

1

For each production in the grammar, add entries to the appropriate locations.

	id	+	*	num	(	)	\$
E	E -> TE'			E -> TE'	E -> TE'		
E'		E' -> +TE'				E' -> λ	E' -> λ
T	T -> FT'			T -> FT'	T -> FT'		
T'							
F							

T -> FT'

Check First(F).

First(F)={id,num,(}

Add T -> FT' under each element.

Constructing LL(1) Parsing Table -
Example 1

For each production in the grammar, add entries to the appropriate locations.

	id	+	*	num	(	)	\$
E	E -> TE'			E -> TE'	E -> TE'		
E'		E' -> +TE'				E' -> λ	E' -> λ
T	T -> FT'			T -> FT'	T -> FT'		
T'		T' -> λ	T' -> *FT'			T' -> λ	T' -> λ
F							

$T' \rightarrow *FT'$

Add $T' \rightarrow *FT'$ under $*$

$T' \rightarrow \lambda$

So add $T' \rightarrow \lambda$ under all elements of $\text{Follow}(T') = \{+,), \$\}$

Constructing LL(1) Parsing Table -
Example 1

For each production in the grammar, add entries to the appropriate locations.

	id	+	*	num	(	)	\$
E	E -> TE'			E -> TE'	E -> TE'		
E'		E' -> +TE'				E' -> λ	E' -> λ
T	T -> FT'			T -> FT'	T -> FT'		
T'		T' -> λ	T' -> *FT'			T' -> λ	T' -> λ
F	F -> id			F -> num	F -> (E)		

F → num | id | (E)
Add F → num
under num

F → id
Add F → id
under id

F → (E)
Add F → (E)
under (

Constructing LL(1) Parsing Table -
Example 1

Thus, this is the final LL(1) parsing table for the given grammar.

	id	+	*	num	(	)	\$
E	E -> TE'			E -> TE'	E -> TE'		
E'		E' -> +TE'				E' -> λ	E' -> λ
T	T -> FT'			T -> FT'	T -> FT'		
T'		T' -> λ	T' -> *FT'			T' -> λ	T' -> λ
F	F -> id			F -> num	F -> (E)		

LL(1) Parsing Table - Conclusions

- The LL(1) parsing table can now be used to parse an input string. This will be demonstrated in the next class.
- To check whether a grammar belongs to LL(1), check whether each box in the LL(1) parser table has atmost 1 production.
- If a grammar is not Left Factored, or Left Recursion is not removed, it will not belong to LL(1).
- The first table will never contain \$
- The follow table will never contain λ



**THANK
YOU**

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Kavya P

K

Compiler Design

Unit 1

Predictive Parsers (LL(1))

Preet Kanwal

Department of Computer Science &
Engineering

In this lecture, you will learn about -

- **LL(1) Parsing Algorithm**
- **Examples on LL(1) parsing algorithm**
- **Identify whether a grammar belongs to LL(1)**
- **LL(k) Grammars**

Parsing using an LL(1) Parsing Table

- Given an LL(1) Parsing table, how do we parse a given input string?
- Initially,
 - Stack contains start symbol **S**
 - Input buffer contains the input string **w**

Stack	Input Buffer	Action
S \$	w \$	

- Parsing Algorithm is applied.
- Finally, if the string is accepted, the stack and input buffer contain **\$**, indicating that they are empty.

Stack	Input Buffer	Action
\$	\$	accept

//Let a be the current input symbol and M be the parsing table

//Let X be the current stack

top. while($X \neq \$$) {

 if($X == a$) { //If X is a terminal

 stack.pop()

 Delete 'a' from input buffer

 }

 else if($M[X,a] = X \rightarrow Y_1Y_2...Y_k$) { //If X is a non terminal

 stack.pop()

 stack.push($Y_1, Y_2, ... Y_k$) //Y1 is the stack top now

 }

else

 error()

}

LL(1) Parsing Algorithm - Example 1

- Given the following LL(1) Parsing table M,
parse the input string
“id + id * id”

LL(1) PARSING TABLE

	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

LL(1) Parsing Algorithm - Example 1

- Given the following LL(1) Parsing table M,
parse the input string:

“id + id * id”

Initially, the stack and input buffer contain -

Stack	Input Buffer	Action
E \$	id + id * id \$	

LL(1) Parsing Algorithm - Example 1

Stack	Input Buffer	Action
E \$	id + id * id \$	M[E,id] = E->TE' pop E from stack push TE'
T E' \$	id + id * id \$	

Now, find the action for the stack top E and current input symbol id in M -
 $M[E, id] = E \rightarrow TE'$

Based on the parsing algorithm, pop E from stack push TE'

Current stack top changes to T.

Compiler Design

LL(1) Parsing Algorithm - Example 1

Stack	Input Buffer	Action
E \$	id + id * id \$	M[E,id] = E->TE' pop E from stack push TE'
T E' \$	id + id * id \$	M[T,id] = T->FT' pop T from stack push FT'
F T' E' \$	id + id * id \$	

M[T,id] = T->FT'
pop T from
stack push FT'

Current stack top changes to
F.

LL(1) Parsing Algorithm - Example 1

Stack	Input Buffer	Action
E \$	id + id * id \$	M[E,id] = E->TE' pop E from stack push TE'
T E' \$	id + id * id \$	M[T,id] = T->FT' pop T from stack push FT'
F T' E' \$	id + id * id \$	M[F,id] = F->id pop F from stack push id
id T' E' \$	id + id * id \$	

M[F,id] = F->id
pop F from
stack push id

Current stack top changes to
id.

LL(1) Parsing Algorithm - Example 1

Stack	Input Buffer	Action
E \$	id + id * id \$	M[E,id] = E->TE' pop E from stack push TE'
T E' \$	id + id * id \$	M[T,id] = T->FT' pop T from stack push FT'
F T' E' \$	id + id * id \$	M[F,id] = F->id pop F from stack push id
id T' E' \$	id + id * id \$	M[id,id] pop id from stack delete id from input buffer
T' E' \$	+ id * id \$	

M[id,id] - case 1, id==id -
True pop id from stack
delete id from input buffer

Current stack top changes to
T'.

LL(1) Parsing Algorithm - Example 1

Stack	Input Buffer	Action
E \$	id + id * id \$	M[E,id] = E->TE' pop E from stack push TE'
T E' \$	id + id * id \$	M[T,id] = T->FT' pop T from stack push FT'
F T' E' \$	id + id * id \$	M[F,id] = F->id pop F from stack push id
id T' E' \$	id + id * id \$	M[id,id] pop id from stack delete id from input buffer
T' E' \$	+ id * id \$	M[T',+] = T -> λ pop T' from stack Current stack top changes to E'.
E' \$	+ id * id \$	

M[T',+] = T' -> λ
pop T' from stack
Current stack top changes to E'.

LL(1) Parsing Algorithm - Example 1

Stack	Input Buffer	Action
...		...
E' \$	+ id * id \$	M[E',+] = E' -> +TE' pop E' from stack push +TE' to stack
+ T E' \$	+ id * id \$	

M[E',+] = E' -> +TE'
pop E' from stack
push +TE' to
stack
Current stack top changes to
+.

LL(1) Parsing Algorithm - Example 1

Stack	Input Buffer	Action
...		...
E' \$	+ id * id \$	M[E',+] = E' -> +TE' pop E' from stack push +TE' to stack
+ T E' \$	+ id * id \$	M[+,+] = match pop + from stack remove + from input buffer
T E' \$	id * id \$	

M[+,+] = match
pop + from stack
remove + from input buffer
Current stack top changes to T.

LL(1) Parsing Algorithm - Example 1

Stack	Input Buffer	Action
...		...
E' \$	+ id * id \$	M[E',+] = E' -> +TE' pop E' from stack push +TE' to stack
+ T E' \$	+ id * id \$	M[+,+] = match pop + from stack remove + from input buffer
T E' \$	id * id \$	M[T,id] = T->FT' pop T from stack push FT' to stack
F T' E' \$	id * id \$	

M[T,id] = T->FT'
pop T from stack
push FT' to stack
Current stack top changes to F.

LL(1) Parsing Algorithm - Example 1

Stack	Input Buffer	Action
...		...
E' \$	+ id * id \$	M[E',+] = E' -> +TE' pop E' from stack push +TE' to stack
+ T E' \$	+ id * id \$	M[+,+] = match pop + from stack remove + from input buffer
T E' \$	id * id \$	M[T,id] = T->FT' pop T from stack push FT' to stack
F T' E' \$	id * id \$	M[F,id] = F->id pop F from stack push id to stack

M[F,id] = F->id
pop F from stack
push id to stack
Current stack top changes to F.

LL(1) Parsing Algorithm - Example 1

Stack	Input Buffer	Action
...		...
id T' E' \$	id * id \$	M[id,id] = match pop id from stack remove id from i/p buffer
T' E' \$	* id \$	

M[id,id] = match
pop id from
stack
remove id from i/p buffer
Current stack top changes to
F.

LL(1) Parsing Algorithm - Example 1

Stack	Input Buffer	Action
...		...
id T' E' \$	id * id \$	M[id,id] = match pop id from stack remove id from i/p buffer
T' E' \$	* id \$	M[T',*] = T'-> λ pop T' from stack push *FT' to stack
* F T' E' \$	* id \$	

M[T',*] = T'-> *FT'
pop T' from stack
push *FT' to
stack
Current stack top changes to
E'.

LL(1) Parsing Algorithm - Example 1

Stack	Input Buffer	Action
...		...
id T' E' \$	id * id \$	M[id,id] = match pop id from stack remove id from i/p buffer
T' E' \$	* id \$	M[T',*] = T'-> λ pop T' from stack push *FT' to stack
* F T' E' \$	* id \$	M[*,*] = match
F T' E' \$	id \$	

M[*,*] = match
Current stack top changes to F.

LL(1) Parsing Algorithm - Example 1

Stack	Input Buffer	Action
...		...
id T' E' \$	id * id \$	M[id,id] = match pop id from stack remove id from i/p buffer
T' E' \$	* id \$	M[T',*] = T'-> λ pop T' from stack push *FT' to stack
* F T' E' \$	* id \$	M[*,*] = match
F T' E' \$	id \$	M[F,id] = F->id pop F from stack push id to stack

M[F,id] = F->id
pop F from
stack push id to
stack
Current stack top changes to
id.

LL(1) Parsing Algorithm - Example 1

Stack	Input Buffer	Action
...		...
id T' E' \$	id \$	M[id,id] = match
T' E' \$	\$	M[T', \$] = T' -> λ pop T' from stack
E' \$	\$	M[E', \$] = E' -> λ
\$	\$	Accept

M[id,id] = match
Current stack top changes to T'.

M[T', \$] = T' -> λ
pop T' from stack

M[E', \$] = E' -> λ
pop E' from stack

Thus, the string **id+id*id** is accepted by the parser.

LL(1) Parsing Algorithm - Example 1 - Homework

Parse the string (id *
id)

LL(1) Parsing Algorithm - Example 2

Construct the LL(1) Parsing table for the given grammar

-

$S \rightarrow a \mid (L)$

$L \rightarrow L, S \mid S$

Then, demonstrate parsing using the input string
(a,a).

LL(1) Parsing Algorithm - Example 2

Given -

$$S \rightarrow a \mid (L)$$
$$L \rightarrow L, S \mid S$$

First, remove left recursion.

- Arrange non-terminals -

$$S \rightarrow a \mid (L)$$
$$L \rightarrow L, S \mid a \mid (L$$

- Replacing productions with left recursion

-
$$S \rightarrow a \mid (L)$$

$$L \rightarrow a L' \mid (L) L'$$
$$L' \rightarrow , S L' \mid \lambda$$

LL(1) Parsing Algorithm - Example 2

Construct first and follow tables for the modified grammar

- $S \rightarrow a \mid (L)$
 $L \rightarrow aL' \mid (L)L'$
 $L' \rightarrow ,SL' \mid \lambda$

First Table		
S	L	L'
a	a	,
(	(	λ

Follow Table		
S	L	L'
\$	)	)
,		
)		

LL(1) Parsing Algorithm - Example 2

Construct the LL(1) parsing table.

	a	(	)	,	\$
S	S->a	S->(L)			
L	L->aL'	L->(L)L'			
L'			L'-> λ	L'-> ,SL'	

LL(1) Parsing Algorithm - Example 2

Stack	Input Buffer	Action
S \$	(a , a) \$	M[S,(] = S -> (L)
(L) \$	(a , a) \$	M[(,(] = match
L) \$	a , a) \$	M[L,a] = L->aX
a X) \$	a , a) \$	M[a,a] = match
X) \$	, a) \$	M[X,,] = X->,SX
, S X) \$	, a) \$	M[,,,] = match
S X) \$	a) \$	M[S,a] = S->a
a X) \$	a) \$	M[a,a] = match
X) \$	) \$	M[X,)] = X->λ
) \$	) \$	M[),)] = match
\$	\$	Accept

Parse the input string (a,a) Parsing successful.

Identify whether a grammar belongs to LL(1)

- If the grammar is not left factored and/or left recursion is not eliminated, it does not belong to LL(1).
- If the LL(1) Table has more than one entry in any cell, the grammar does not belong to LL(1) class of grammars.
- Formally, to check whether a grammar belongs to LL(1) without constructing the table, the following rules are checked.
- A grammar is NOT in LL(1) if
 1. For $A \rightarrow a \mid b$, i.e., there are two or more alternatives for a Non-terminal
If $\text{First}(a) \cap \text{First}(b) \neq \Phi$, i.e., if they have something in common.
 1. $A \rightarrow a \mid \lambda$

Identify whether a grammar belongs to LL(1) - Example

3

Is the Given grammar in LL(1) ? Also, Compute first and follow sets.

$S \rightarrow A a A b \quad | \quad B b B a$

$A \rightarrow \lambda$

$B \rightarrow \lambda$

Identify whether a grammar belongs to LL(1) - Example 3

$S \rightarrow A a A b \quad | \quad B b B a$

$A \rightarrow \lambda$

$B \rightarrow \lambda$

Compute First and Follow Sets

-

First Table		
S	A	B
a	λ	λ
b		

Follow Table		
S	A	B
\$	a	a
	b	b

Identify whether a grammar belongs to LL(1) - Example 3

$S \rightarrow A a A b \quad | \quad B b B a$

$A \rightarrow \lambda$

$B \rightarrow \lambda$

$\text{First}(AaAb) = \{a\}$

$\text{First}(BbBa) = \{b\}$

$\text{First}(AaAb)$ and $\text{First}(BbBa)$ are disjoint, hence, the grammar is LL(1).

Identify whether a grammar belongs to LL(1) - Example

4

Is the Given grammar in LL(1) ? If not, modify and re-check.

$$S \rightarrow i C t S \mid i C t S e S \mid a$$
$$C \rightarrow b$$

Identify whether a grammar belongs to LL(1) - Example

2

- If we construct the parse table M for the following

grammar,
 $S \rightarrow i C t S \mid i C t S e S \mid a$

$$C \rightarrow b$$

- $M[S,i]$ will have 2

productions

$$S \rightarrow i C t S$$
$$S \rightarrow i C t S e S$$

- Therefore, it is not in LL(1).
- To modify, left factor the above grammar

-

$$S \rightarrow i C t S X \mid a$$
$$X \rightarrow e S \mid \lambda$$
$$C \rightarrow b$$

Identify whether a grammar belongs to LL(1) - Example

4

- Is the modified grammar in LL(1)

?

$S \rightarrow i C t S X \mid a$

$X \rightarrow e S \mid \lambda$

$C \rightarrow b$

- Notice the production

$X \rightarrow e S \mid \lambda$

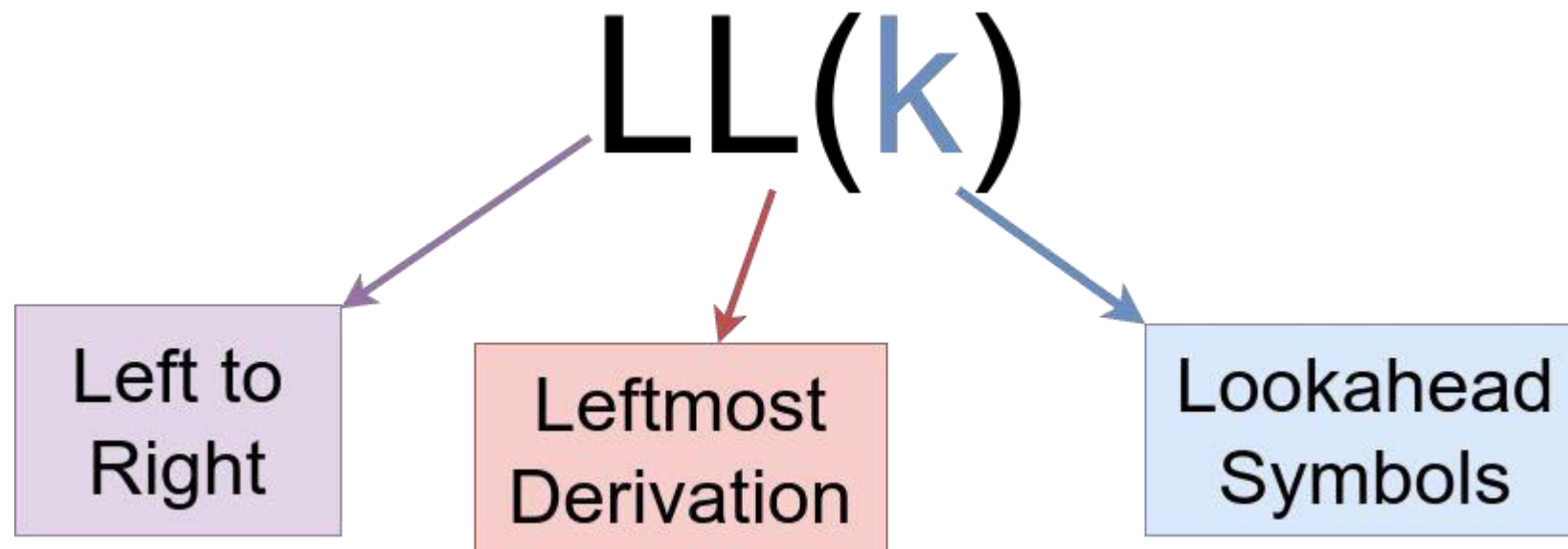
is of the form

$A \rightarrow a \mid \lambda$

- $\text{First}(e) = e$
- $\text{Follow}(X) = \{ e, \$ \}$
- Thus, $\text{First}(e) \cap \text{Follow}(X) = e \neq \Phi$
- Therefore, the modified grammar is also not in LL(1).

$k > 1$

As mentioned
previously,



- When $k = 1$, we can choose the next production rule using 1 lookahead symbol.
- That is, by reading 1 input character.
- For example,

$S \rightarrow a \mid b$

$k > 1$

- For $k > 1$, we can choose the next production after reading k input characters.
- For example,
$$S \rightarrow ab \mid ac \mid ad$$
- Here, the production can be chosen only after reading 2 input characters.
- Therefore, the above grammar belongs to **LL(2)** class of parsers.
- $LL(k) \subset LL(k+1)$
- If the LL(k) Table has more than k entries in any cell, the grammar does not belong to LL(k) class of grammars, where $k > 1$.

Identify whether a grammar belongs to LL(k) - Examples

Are the following grammars in LL(k)? If yes, find k.

1. $S \rightarrow i C t S \mid i C t S e S \mid a$
 $C \rightarrow b$

2. $S \rightarrow a a B \mid a a C$
 $B \rightarrow b$
 $C \rightarrow c$

Identify whether a grammar belongs to LL(k) - Solution

Are the following grammars in LL(k)? If yes, find k.

1. $S \rightarrow i C t S \mid i C t S e S \mid a$
 $C \rightarrow b$

Answer - No. Fixed k is not possible.

Answer: No for any fixed finite k. NOT LL(k) for any fixed finite k.

Because to decide whether an “else” (i.e., e in grammar) follows, you need to know where the inner **S** ends, and **S** generates arbitrarily long strings, no finite k suffices (i.e., unlimited lookahead is required).

2. $S \rightarrow a a B \mid a a C$
 $B \rightarrow b$
 $C \rightarrow c$

Answer - Yes. It belongs to LL(3).

Identify whether a grammar belongs to LL(k) - Solution

Exercise:

$$E \rightarrow T + E \mid T$$
$$T \rightarrow \text{id} \mid \text{id} * T \mid (E)$$

Note: The grammar generates the language of all arithmetic expressions using identifiers, +, *, and parentheses, with right-associative multiplication and addition.

Can a predictive parser decide **which production of E to use** with a finite lookahead k?

Answer: No for any fixed finite k. NOT LL(k) for any fixed finite k.

To check whether a grammar is LL(k) for any finite k.

Productions: $E \rightarrow T + E \mid T$

Both alternatives begin with T, and $\text{FIRST}(T) = \{ \text{id}, '(' \}$

To distinguish: $T + E$ vs T

you must look ahead past T, which is unbounded.

T can generate: $\text{id}, \text{id} * \text{id}, \text{id} * \text{id} * \text{id} * \dots, (E), (T + E), (\text{id} * (\text{id} + \text{id} * \dots))$

The length of T is unbounded.

There is no fixed number k of lookahead symbols that always brings you to the +.

You might have: $\text{id} * \text{id} * \text{id} * \text{id} * \text{id} + \dots$

You don't know you must choose production $E \rightarrow T + E$ until after arbitrarily many * expansions.

Thus, the above grammar is not LL(k) for any finite k.



**THANK
YOU**

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Kavya P

K

Compiler Design

Unit 1

Predictive Parsers (LL(1))

Preet Kanwal

Department of Computer Science &
Engineering

In this lecture, you will learn about -

- Additional examples on LL(1) parsing algorithm
- LL(k) Grammars

LL(1) Parsing Algorithm - Example 5

Construct the LL(1) Parsing table for the given grammar

$S \rightarrow AB$

$A \rightarrow a \mid \lambda$

$B \rightarrow b \mid \lambda$

Then, demonstrate parsing using the input string λ .

LL(1) Parsing Algorithm - Example 5

Given -

$$S \rightarrow AB$$
$$A \rightarrow a \mid \lambda$$
$$B \rightarrow b \mid \lambda$$

No left recursion present, and grammar is left factored.

LL(1) Parsing Algorithm - Example 5

Construct first and follow tables for the grammar

- $S \rightarrow AB$

$A \rightarrow a \mid \lambda$

$B \rightarrow b \mid \lambda$

First Table		
S	A	B
a	a	b
b	λ	λ
λ		

Follow Table		
S	A	B
\$	b	\$
	\$	

LL(1) Parsing Algorithm - Example 5

Construct the LL(1) parsing table.

	a	b	\$
S	$S \rightarrow AB$	$S \rightarrow AB$	$S \rightarrow AB$
A	$A \rightarrow a$	$A \rightarrow \lambda$	$A \rightarrow \lambda$
B		$B \rightarrow b$	$B \rightarrow \lambda$

LL(1) Parsing Algorithm - Example 5

Stack	Input Buffer	Action
S\$	\$	$M[S, \$] = S \rightarrow AB$
AB\$	\$	$M[A, \$] = A \rightarrow \lambda$
B\$	\$	$M[B, \$] = B \rightarrow \lambda$
\$	\$	Accept

Parse the input string
 λ
Parsing successful.

LL(1) Parsing Algorithm - Example 6

Is the Given grammar in LL(1) ? Also, Compute first and follow sets. Parse the input string **cde**

$S \rightarrow ABCDE$

$A \rightarrow a \mid \lambda$

$B \rightarrow b \mid \lambda$

$C \rightarrow c$

$D \rightarrow d \mid \lambda$

$E \rightarrow e \mid \lambda$

LL(1) Parsing Algorithm - Example 6

S $\rightarrow$ **ABCDE**

A $\rightarrow$ **a** | λ

B $\rightarrow$ **b** | λ

C $\rightarrow$ **c**

D $\rightarrow$ **d** | λ

E $\rightarrow$ **e** | λ

Compute First and Follow Sets -

First Table					
S	A	B	C	D	E
a	a	b	c	d	e
b	λ	λ		λ	λ
c					

Follow Table					
S	A	B	C	D	E
\$	b	c	d	e	\$
	c		e	\$	
			\$		

LL(1) Parsing Algorithm - Example 6

Construct the LL(1) parsing table.

	a	b	c	d	e	\$
S	S→ABCDE	S→ABCDE	S→ABCDE			
A	A→a	A→λ	A→λ			
B		B→b	B→λ			
C			C→c			
D				D→d	D→λ	D→λ
E					E→e	E→λ

LL(1) Parsing Algorithm - Example 6

Stack	Input Buffer	Action
S \$	cde\$	$M[S,c] = S \rightarrow ABCDE$
ABCDE\$	cde\$	$M[A,c] = A \rightarrow \lambda$
BCDE\$	cde\$	$M[B,c] = B \rightarrow \lambda$
CDE\$	cde\$	$M[C,c] = C \rightarrow c$
cDE\$	cde\$	$M[c,c] = \text{match}$
DE\$	de\$	$M[D,d] = D \rightarrow d$
dE\$	de\$	$M[d,d] = \text{match}$
E\$	e\$	$M[E,e] = E \rightarrow e$
e\$	e\$	$M[e,e] = \text{match}$
\$	\$	Accept

Parse the input string
cde Parsing successful.

Identify whether a grammar belongs to LL(k) -

Examples

Are the following grammars in LL(k)? If yes, find k.

1. $S \rightarrow \quad Abbx \mid Bbby$

$A \rightarrow x$

$B \rightarrow \quad x$

2. $S \rightarrow Z$

$Z \rightarrow aMa \mid bMb \mid aRb \mid bRa$

$M \rightarrow c$

$R \rightarrow c$

Identify whether a grammar belongs to LL(k) - Solution

Are the following grammars in LL(k)? If yes, find k.

1. $S \rightarrow \text{Abbx} \mid \text{Bbby}$

$A \rightarrow x$

$B \rightarrow x$

Answer - Yes. It belongs to LL(4).

2. $S \rightarrow Z$

$Z \rightarrow aMa \mid bMb \mid aRb \mid bRa$

$M \rightarrow c$

$R \rightarrow c$

Answer - Yes. It belongs to LL(3).



**THANK
YOU**

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Kavya P

K

Compiler Design

Unit 1

Predictive Parsers (LL(1))

Preet Kanwal

Department of Computer Science &
Engineering

In this lecture, you will learn about

-

- Error recovery in LL(1) Parser
- RDP implementation vs LL(1)

- When an action $M[X,Y]$ is blank in the Parsing Table, and it is encountered during parsing of an input string, it indicates **syntax error**.
- How can we handle errors in LL(1) parsers?
- One of the most commonly used Error recovery strategies in Parsers is **Panic Mode Recovery**.
- Recall that in Panic mode Recovery,
 - The parser will discard the input symbols until it finds a delimiter, and then restart the parsing process.
 - Such delimiters are called **synchronizing tokens**.

How is Panic Mode Recovery implemented in LL(1) Parsers?

- For each Non-terminal X in the parsing table M , add a synchronising token **sync** under the elements of $\text{Follow}(X)$ if the cell is blank.
- Parsing procedure -

If $M[X,a] == \text{blank}$ // i.e, syntax error ignore the
input symbol a

If $M[X,a] == \text{sync}$

If X is the only symbol in the Stack Ignore input

symbol a

Error Recovery in LL(1) Parser - Example

1

Implement the parsing table for the following grammar with panic mode recovery. Also, parse the string “)id*+id”

E	→	T E'		
E'	→	+ T E'		λ
T	→	F T'		
T'	→	* F T'		λ
F	→	num		id (E)

Error Recovery in LL(1) Parser - Example

1

Given,

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \lambda$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid \lambda$

$F \rightarrow \text{num} \mid \text{id} \mid (E)$

First Table				
E	E'	T	T'	F
id	+	id	*	id
num	λ	num	λ	num
(		(		(

Follow Table				
E	E'	T	T'	F
\$	\$	+	+	*
)	)	\$	\$	+
		)	)	\$
				)

Error Recovery in LL(1) Parser - Example

1

The LL(1) parsing table without panic mode recovery-

	id	+	*	num	(	)	\$
E	E -> TE'			E -> TE'	E -> TE'		
E'		E' -> +TE'				E' -> λ	E' -> λ
T	T -> FT'			T -> FT'	T -> FT'		
T'		T' -> λ	T' -> *FT'			T' -> λ	T' -> λ
F	F -> id			F -> num	F -> (E)		

Error Recovery in LL(1) Parser - Example

1

Adding sync tokens under elements of Follow(X), where X is a non-terminal.

	id	+	*	num	(	)	\$
E	E -> TE'			E -> TE'	E -> TE'	sync	sync
E'		E' -> +TE'				E' -> λ	E' -> λ
T	T -> FT'	sync		T -> FT'	T -> FT'	sync	sync
T'		T' -> λ	T' -> *FT'			T' -> λ	T' -> λ
F	F -> id	sync	sync	F -> num	F -> (E)	sync	sync

Follow

Table				
E	E'	T	T'	F
\$	\$	+	+	*
)	)	\$	\$	+
		)	)	\$
				)

Error Recovery in LL(1) Parser - Example

1

Now, using the new LL(1) Parsing table M, parse the input string

) id * + id

Initially, the stack and input buffer contain -

Stack	Input Buffer	Action
E \$	) id * + id \$	

LL(1) Parsing Algorithm - Example 1

Stack	Input Buffer	Action
E\$	)id*+id\$	M[E,)] = sync skip
E\$	id*+id\$	

M[E,)] = sync

Based on the new parsing algorithm, Since E is the only symbol in the stack, ignore the current input symbol)

LL(1) Parsing Algorithm - Example 1

Stack	Input Buffer	Action
E \$	) id * + id \$	M[E,)] = sync skip
E \$	id * + id \$	M[E,id] = E -> TE'
T E' \$	id * + id \$	M[T,id] = T ->FT'
F T' E' \$	id * + id \$	M[F,id] = F->id
id T' E' \$	id * + id \$	M[id,id] = match
T' E' \$	* + id \$	M[T',*] = T'->*FT'
* F T' E' \$	* + id \$	M[*,*] = match
F T' E' \$	+ id \$	M[F,+] = sync Pop F
T' E' \$	+ id \$	

Continue parsing.

M[F, +] = sync

F is not the only symbol in the stack. So, Pop F from stack.

The current stack top is T'

LL(1) Parsing Algorithm - Example 1

Stack	Input Buffer	Action
...	...	...
T' E' \$	+ id \$	M[T',+] = T' ->λ
E' \$	+ id \$	M[E',+] = +TE'
+ T E' \$	+ id \$	M[+,+] = match
T E' \$	id \$	M[T,id] = T -> FT'
F T' E' \$	id \$	M[F,id] = F->id
id T' E' \$	id \$	M[id,id] = match
T' E' \$	\$	M[T',\$] = T'->λ
E' \$	\$	M[E',\$] = E'->λ
\$	\$	Accept

Continue parsing.

Thus, the input string is parsed till the end even though it contained several syntax errors.

Homework

Parse the erroneous string **id (+) id**

RDP without Backtracking vs Predictive Parsers

RDP without Backtracking	Predictive Parsers// LL(1) parser
It uses mutually recursive procedures for every non-terminal entity to parse strings.	It maintains an explicit stack data structure.
It accepts LL(*) grammars	It accepts only a LL(1) grammars.

Thus, RDP without Backtracking is more powerful, as it accepts a larger set of grammars.



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Kavya P

K

Compiler Design

Unit 1

Predictive Parsers (LL(1))

Preet Kanwal

Department of Computer Science &
Engineering

In this lecture, you will learn about -

- Additional examples on Error recovery in LL(1) Parser

Error Recovery in LL(1) Parser - Example

2

Implement the parsing table for the following grammar with panic mode recovery. Also, parse the string **a(a**)

$$S \rightarrow a \quad | \quad (L)$$
$$L \rightarrow a L' \quad | \quad (L) L'$$
$$L' \rightarrow , S L' \quad | \quad \lambda$$

Error Recovery in LL(1) Parser - Example

2

Construct first and follow tables for the grammar

- - $S \rightarrow a \mid (L)$
 - $L \rightarrow aL' \mid (L)L'$
 - $L' \rightarrow ,SL' \mid \lambda$

First Table		
S	L	L'
a	a	,
(	(	λ

Follow Table		
S	L	L'
\$	)	)
,		
)		

Error Recovery in LL(1) Parser - Example

2

The LL(1) parsing table without panic mode recovery is -

	a	(	)	,	\$
S	S->a	S->(L)			
L	L->aX	L->(L)L'			
L'			L'-> λ	L'-> ,SL'	

Error Recovery in LL(1) Parser - Example 2

Adding **sync** tokens under elements of Follow(X), where X is a non-terminal.

	a	(	)	,	\$
S	S->a	S->(L)	sync	sync	sync
L	L->aX	L->(L)L'	sync		
L'			L'-> λ	L'->,SL'	

Follow Table		
S	L	L'
\$	)	)
,		
)		

Error Recovery in LL(1) Parser - Example

1

Now, using the new LL(1) Parsing table M, parse the input string

a(a) Initially, the stack and input buffer contain -

Stack	Input Buffer	Action
s\$	a (a) \$	

LL(1) Parsing Algorithm - Example 2

Stack	Input Buffer	Action
S\$	a (a) \$	M[S,a] = S->a
a\$	a (a) \$	M[a,a] = match
\$	(a) \$	

LL(1) Parsing Algorithm - Example 2 - Homework

Parse the erroneous string
 $\wedge(,a,a)$



**THANK
YOU**

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu